

INTRODUCTION TO THE SCENARIO APPROACH



MOS-SIAM Series on Optimization

This series is published jointly by the Mathematical Optimization Society and the Society for Industrial and Applied Mathematics. It includes research monographs, books on applications, textbooks at all levels, and tutorials. Besides being of high scientific quality, books in the series must advance the understanding and practice of optimization. They must also be written clearly and at an appropriate level for the intended audience.

Editor-in-Chief

Katya Scheinberg
Lehigh University

Editorial Board

Santanu S. Dey, *Georgia Institute of Technology*
Maryam Fazel, *University of Washington*
Serge Gratton, *INP-ENSEEIH*
Andrea Lodi, *Polytechnique Montréal*
Arkadi Nemirovski, *Georgia Institute of Technology*
David B. Shmoys, *Cornell University*
Stefan Ulbrich, *Technische Universität Darmstadt*
Stephen J. Wright, *University of Wisconsin*

Series Volumes

Campi, Marco C. and Garatti, Simone, *Introduction to the Scenario Approach*
Beck, Amir, *First-Order Methods in Optimization*
Trelaký, Tamás, Anjos, Miguel F., and Ahmed, Shabbir, editors, *Advances and Trends in Optimization with Engineering Applications*
Todd, Michael J., *Minimum-Volume Ellipsoids: Theory and Algorithms*
Bienstock, Daniel, *Electrical Transmission System Cascades and Vulnerability: An Operations Research Viewpoint*
Koch, Thorsten, Hiller, Benjamin, Pfetsch, Marc E., and Schewe, Lars, editors, *Evaluating Gas Network Capacities*
Corberán, Ángel, and Laporte, Gilbert, *Arc Routing: Problems, Methods, and Applications*
Toth, Paolo and Vigo, Daniele, *Vehicle Routing: Problems, Methods, and Applications, Second Edition*
Beck, Amir, *Introduction to Nonlinear Optimization: Theory, Algorithms, and Applications with MATLAB*
Attouch, Hedy, Buttazzo, Giuseppe, and Michaille, Gérard, *Variational Analysis in Sobolev and BV Spaces: Applications to PDEs and Optimization, Second Edition*
Shapiro, Alexander, Dentcheva, Darinka, and Ruszczyński, Andrzej, *Lectures on Stochastic Programming: Modeling and Theory, Second Edition*
Locatelli, Marco and Schoen, Fabio, *Global Optimization: Theory, Algorithms, and Applications*
De Loera, Jesús A., Hemmecke, Raymond, and Köppe, Matthias, *Algebraic and Geometric Ideas in the Theory of Discrete Optimization*
Blekherman, Grigoriy, Parrilo, Pablo A., and Thomas, Rekha R., editors, *Semidefinite Optimization and Convex Algebraic Geometry*
Delfour, M. C., *Introduction to Optimization and Semidifferential Calculus*
Ulbrich, Michael, *Semismooth Newton Methods for Variational Inequalities and Constrained Optimization Problems in Function Spaces*
Biegler, Lorenz T., *Nonlinear Programming: Concepts, Algorithms, and Applications to Chemical Processes*
Shapiro, Alexander, Dentcheva, Darinka, and Ruszczyński, Andrzej, *Lectures on Stochastic Programming: Modeling and Theory*
Conn, Andrew R., Scheinberg, Katya, and Vicente, Luis N., *Introduction to Derivative-Free Optimization*
Ferris, Michael C., Mangasarian, Olvi L., and Wright, Stephen J., *Linear Programming with MATLAB*
Attouch, Hedy, Buttazzo, Giuseppe, and Michaille, Gérard, *Variational Analysis in Sobolev and BV Spaces: Applications to PDEs and Optimization*
Wallace, Stein W. and Ziemba, William T., editors, *Applications of Stochastic Programming*
Grötschel, Martin, editor, *The Sharpest Cut: The Impact of Manfred Padberg and His Work*
Renegar, James, *A Mathematical View of Interior-Point Methods in Convex Optimization*
Ben-Tal, Aharon and Nemirovski, Arkadi, *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*
Conn, Andrew R., Gould, Nicholas I. M., and Toint, Philippe L., *Trust-Region Methods*

INTRODUCTION TO THE SCENARIO APPROACH

Marco C. Campi

University of Brescia
Brescia, Italy

Simone Garatti

Polytechnic University of Milan
Milan, Italy



Society for Industrial and Applied Mathematics
Philadelphia



Mathematical
Optimization Society

Mathematical Optimization Society
Philadelphia

Copyright © 2018 by the Society for Industrial and Applied Mathematics and the Mathematical Optimization Society

10 9 8 7 6 5 4 3 2 1

All rights reserved. Printed in the United States of America. No part of this book may be reproduced, stored, or transmitted in any manner without the written permission of the publisher. For information, write to the Society for Industrial and Applied Mathematics, 3600 Market Street, 6th Floor, Philadelphia, PA 19104-2688.

Trademarked names may be used in this book without the inclusion of a trademark symbol. These names are used in an editorial context only; no infringement of trademark is intended.

MATLAB is a registered trademark of The MathWorks, Inc. For MATLAB product information, please contact The MathWorks, Inc., 3 Apple Hill Drive, Natick, MA 01760-2098 USA, 508-647-7000, Fax: 508-647-7001, info@mathworks.com, www.mathworks.com.

Publications Director	Kivmars H. Bowling
Acquisitions Editor	Paula Callaghan
Developmental Editor	Gina Rinelli Harris
Managing Editor	Kelly Thomas
Production Editor	Lisa Briggeman
Copy Editor	Claudine Dugan
Production Manager	Donna Witzleben
Production Coordinator	Cally A. Shrader
Compositor	Cheryl Hufnagle
Graphic Designer	Doug Smock

Library of Congress Cataloging-in-Publication Data

Names: Campi, Marco, author. | Garatti, Simone, 1976- author.

Title: Introduction to the scenario approach / Marco Campi, Simone Garatti.

Description: Philadelphia : Society for Industrial and Applied Mathematics : Mathematical Optimization Society, [2018] | Series: MOS-SIAM series on optimization ; 26 | Includes bibliographical references and index.

Identifiers: LCCN 2018033507 (print) | LCCN 2018038454 (ebook) | ISBN 9781611975444 | ISBN 9781611975437 (print)

Subjects: LCSH: Uncertainty-Mathematical models. | Decision making-Mathematical models.

Classification: LCC QA273 (ebook) | LCC QA273 .C245 2018 (print) | DDC 519.2-dc23

LC record available at <https://lcn.loc.gov/2018033507>



Contents

Preface	vii
1 Introduction: Uncertainty in optimization and the scenario approach	1
1.1 Principles in optimization with uncertainty	1
1.2 The scenario approach	6
1.3 Generalizations	20
2 Problems in control	21
2.1 Robust control problems	21
2.2 Disturbance compensation	27
3 Theoretical results and their interpretation	33
3.1 Mathematical setup	33
3.2 The generalization theorem of the scenario approach	38
3.3 Scenario optimization with constraint removal	44
4 Probabilistic vs. deterministic robustness	49
4.1 Sure statements vs. statements with a probabilistic validity	49
4.2 Beyond the use of models	53
5 Proofs	55
5.1 Support constraints and fully supported problems	55
5.2 Proof of the generalization Theorem 3.7	58
5.3 Proof of Theorem 3.9	64
6 Region estimation models	67
6.1 Observations and estimation models	67
6.2 Beyond Chebyshev layers	76
7 Application to statistical learning	79
7.1 An introduction to statistical learning and classification	79
7.2 The algorithm GEM	81
7.3 Application to breast tumor diagnosis	86
8 Miscellanea	89
8.1 Probability box	89
8.2 L_1 -regularization	91
8.3 Fast algorithm for the scenario approach	93
8.4 Wait-and-judge	95
8.5 Expected shortfall	99
8.6 Nonconvex scenario optimization	101
Bibliography	109
Index	115

Preface

This book is about optimizing in the presence of uncertainty. Due to uncertainty, one needs to exercise caution, and optimization must accommodate the uncertain elements that are present in the problem. A scenario is an instance of uncertainty, and a scenario program is an optimization program that incorporates a sample of scenarios. The fact that only a finite sample of scenarios is considered makes the problem tractable, while theoretical results establish rigorous ties between scenario solutions and the original uncertainty problem.

The scenario approach has been given a solid mathematical foundation in recent years, and it is rapidly evolving at the time this book is being written. Our goal here is not completeness, instead, we mean to provide the reader with easy access to the scenario methodology, leaving to the literature the task of providing more-in-depth presentations. Many pointers to the literature will be given along the way.

Scenario optimization can be applied across a variety of fields, including machine learning, quantitative finance, and control, so this book targets a vast audience. It is intended not only for practitioners, who can find here “easy-to-use recipes,” but also for theoreticians, who will benefit from a rigorous treatment of the theoretical foundations of the method.

This book has a bottom-up structure. We start with a broad presentation that covers a rapid exposure to many aspects relating to the scenario approach, and then we guide the reader through various chapters that present material that is more detailed. This material covers a selected set of theoretical results, some applications, and a discussion on the interpretation of the method.

In order to keep the book focused, only a sample of fundamental proofs is presented. At times, we shall just hint at various issues, rather than fully developing them. The chapters are linked to one another according to the scheme presented in Figure 1, where an arrow indicates that the reading of a chapter is best done after the reading of the parent chapters has been completed.

This book was partly motivated by the lectures that we taught for four consecutive years, from 2012 to 2015, at Supélec—Orsay, Paris Sud—as part of the EECI International Graduate School on Control. We are gratefully indebted to Françoise Lamnabhi-Lagarrigue for her commitment towards promoting and organizing this school. Later, presentations of portions of this book were delivered at the NASA—Langley Research Center, USA, at the Politecnico di Milano, Italy, at Texas A&M University, USA, and at the University of Melbourne, Australia. We thank all the people who have contributed to improving and consolidating this material through their comments and suggestions.

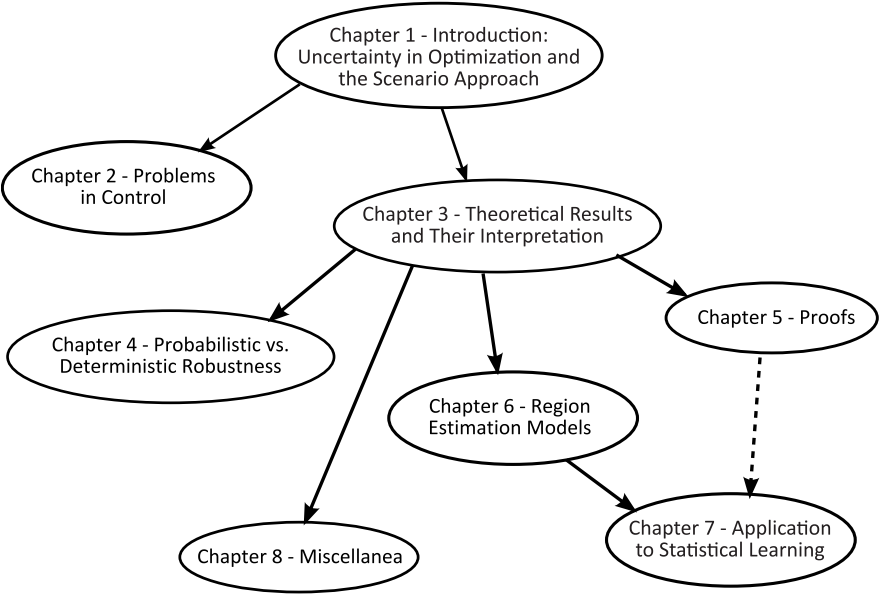


Figure 1. *The book structure.*

Chapter 1

Introduction: Uncertainty in optimization and the scenario approach

Optimization is everywhere. Any time we make a choice based on some criterion of preference, we try to meet that criterion at best. In a word, we “optimize.” Examples are found in system design, controller synthesis, portfolio selection, and management, to cite but a few examples. Along the way, however, a second element often comes into play, and this is uncertainty. Because of uncertainty, one does not aim at just solving a nominal problem; instead one wishes to exercise caution and come up with a design that is robust against the uncertainty elements that are present in the problem. The scenario approach copes with the presence of uncertainty and relies on a finite sample of occurrences of the uncertainty, possibly obtained via experiments. In the first part of this chapter, we briefly review the most common approaches to dealing with uncertainty in optimization. The presentation is not comprehensive, and our goal is that of providing the reader with background knowledge to put the scenario approach in context. The scenario approach is then presented in the second part of the chapter.

1.1 ■ Principles in optimization with uncertainty

Throughout, an uncertain element is indicated with the symbol δ , while Δ is the range set for δ . $\nu \in \mathbb{R}^{d-1}$ is the vector of design variables. ν is in $\mathbb{R}^{d-1}$ rather than $\mathbb{R}^d$ because, as we shall see, the role of the d th variable is played by the cost that we optimize. ν can, for instance, contain the parameters of a system that is being designed, can be the percentage of capital allocated on various assets when a portfolio is selected, or can describe how resources are deployed in the management of a company. The cost function to be optimized is written as $\ell(\nu, \delta)$. ℓ stands for “loss,” and throughout, “optimizing” is “minimizing.” If one wishes to maximize, s/he has simply to place a minus in front of the function to be maximized, and then minimize it. In real world applications, experience shows that by applying the same design twice, one seldom gets the same result. This is because the result also depends on extra elements besides our own decision, and the simultaneous presence of ν and δ as arguments of $\ell(\cdot, \cdot)$ reflects this state of things.

An optimization problem with uncertainty consists of minimizing $\ell(\nu, \delta)$ with respect to ν , and one is attempted to write

$$\min_{\nu} \ell(\nu, \delta). \quad (1.1)$$

As stated in (1.1), however, the optimization problem is not well defined because (1.1) does not describe how the uncertain element δ should be accounted for when ℓ is minimized. Addressing this issue requires one to be more specific about the role of uncertainty, and various approaches arise depending on the adopted formulation.

1.1.1 ■ The worst-case approach

The notion of uncertainty is inescapably linked to the notion of set since there cannot be uncertainty without a set of possible uncertainty outcomes. Without any further structure given to uncertainty apart from the fact that the uncertain element δ belongs to a set Δ , a natural way to pose the problem is along a worst-case approach:

$$\min_{\nu \in \mathbb{R}^{d-1}} \left[\max_{\delta \in \Delta} \ell(\nu, \delta) \right]. \quad (1.2)$$

In control, the worst-case philosophy was initially adopted out of concerns for stability in [97], where H_∞ control was used as the robust alternative to linear-quadratic-Gaussian (LQG) control, a method that has no guaranteed stability margins, as shown in [48]. Since then, the worst-case approach has become mainstream in robust control, and it has been applied to a variety of control problems. References [58, 99, 37] provide book-length presentations of the worst-case approach in control. In other fields, the worst-case approach is not as popular as it is in control. The articles [11, 12, 15] and the book [10] have had a significant role in promoting the worst-case approach in optimization.

1.1.2 ■ The average approach

When a more structured, probabilistic point of view in the description of uncertainty is adopted, the average approach can be used. In a probabilistic description, Δ is endowed with a probability distribution $\mathbb{P}$, and this $\mathbb{P}$ can be given various interpretations, depending on the user's standpoint, and in light of the problem at hand. Sometimes, $\mathbb{P}$ describes the chance with which various outcomes of the uncertain element δ occur, while at other times $\mathbb{P}$ is more simply a descriptor of the relative importance given to the various uncertainty outcomes. No matter which interpretation is adopted, $\mathbb{P}$ can be used to weigh the δ 's, leading to an average cost optimization problem,

$$\min_{\nu \in \mathbb{R}^{d-1}} \mathbb{E}_\Delta [\ell(\nu, \delta)] = \min_{\nu \in \mathbb{R}^{d-1}} \int_\Delta \ell(\nu, \delta) d\mathbb{P}.$$

In control, this framework is often adopted when uncertainty is associated with disturbances [6, 60, 14]. A typical example is quadratic stochastic control, where the average cost in discrete time over a finite horizon is written as

$$\mathbb{E}_\Delta [\ell(\nu, \delta)] = \mathbb{E}_\Delta \left[\sum_{t=0}^T \{x_t^T Q x_t + u_t^T R u_t\} \right].$$

In this formula, x_t indicates the system state, u_t is the input, and expectation is taken with respect to the realizations of the disturbance acting on the system. Thus, δ corresponds to a noise realization, and Δ is the set of all possible noise realizations, whereas the probability $\mathbb{P}$ specifies the noise distribution. If, for example, the noise is white and Gaussian, $\mathbb{P}$ is the product probability measure of T

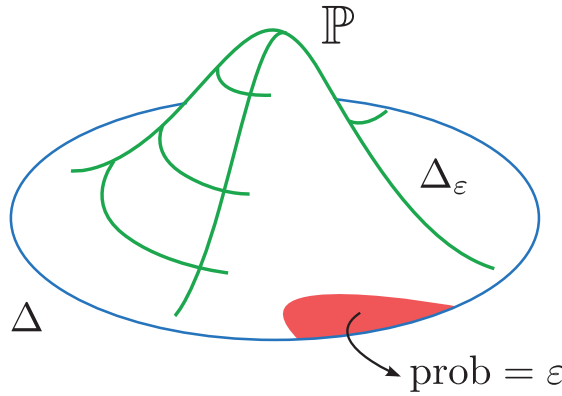


Figure 1.1. Set where the performance level is not attained. The red region is the set where the performance level is not attained, and the probability of this set is ϵ .

independent Gaussian random variables. The average approach is also quite common in stochastic optimization, especially for multistage problems and, in general, for all those problems where the same decision is repeatedly made under different conditions over a certain period of time; see, e.g., [78, 82].

1.1.3 ■ The chance-constrained approach

Averaging is not the only use one can make of probability. An alternative approach consists of using probability to quantify the chance that a performance level is not attained. Referring to Figure 1.1, one aims at minimizing the max cost with max taken over a reduced set $\Delta_\epsilon \subset \Delta$ having probability $\mathbb{P}\{\Delta_\epsilon\} = 1 - \epsilon$, namely,

$$\min_{\nu \in \mathbb{R}^{d-1}, \Delta_\epsilon} \left[\max_{\delta \in \Delta_\epsilon} \ell(\nu, \delta) \right]. \quad (1.3)$$

Indicating by $(\nu_\epsilon^*, \Delta_\epsilon^*)$ the optimal solution of problem (1.3) and by ℓ_ϵ^* its optimal value, we have that $\ell_\epsilon^* = \max_{\delta \in \Delta_\epsilon^*} \ell(\nu_\epsilon^*, \delta)$, that is, ℓ_ϵ^* is guaranteed against all uncertainty outcomes in Δ_ϵ^* , a set having probability $1 - \epsilon$.

The reason for leaving out an ϵ -probability set is that one wants to reduce the optimal value ℓ_ϵ^* as compared to the worst-case approach. The level of robustness depends on ϵ , and, for a given ϵ , the set Δ_ϵ in (1.3) must be selected so that the reduction of the value is maximized. This justifies the presence of Δ_ϵ as an optimization variable in the min quantifier.

To make the above explanation more concrete, think of the problem of portfolio selection. An investor comes to see a market analyst. The analyst asks the investor about the level of risk s/he is willing to take in the investment. In responding, say, 2%, the investor is not interested in the circumstances under which s/he will hit a shortfall. It can either be that a crisis will affect the stock market, or that national bonds will lose value in response to a bad political policy. What the investor is interested in is the fact that the risk is 2%. It is then the analyst's business to decide which 2% portion of the uncertainty set Δ is best left out to maximize the reward in the remaining 98% probability set.

Interestingly, the parameter ϵ can be varied and used as a tuning knob. The larger ϵ , the better the guaranteed performance level, but the higher the risk of not attaining it. This quite naturally leads to an approach where the level of robustness

is adjustable, a fundamental feature in practical problems. The same flexibility is not germane to the worst-case and average approaches.

The approach of (1.3) has a long history in optimization theory, where it is known under the name of “chance-constrained” optimization. It dates back at least to the 1950s [35], and countless contributions have appeared in the literature; see, e.g., [73, 74, 75, 42, 82]. In systems and control, instead, this approach is certainly newer, and it has only been considered in a few papers [62, 20, 1, 16, 36]. This is mainly due to tradition: When dealing with stochastic disturbances, the approach commonly adopted in control is the average approach, while the worst-case approach is mainstream for structural uncertainty.

Chance-constrained optimization is very hard to solve in general, and this fact has by and large hindered its applicability to real problems. The scenario approach addresses optimization along the chance-constrained framework and offers a viable route to find approximate chance-constrained solutions under very general conditions. Due to its generality, the scenario approach can be applied across diverse fields.

Before delving into the scenario approach, in the next subsection we pause for a moment and look at the three optimization paradigms—worst-case, average, and chance-constrained—from a different angle, that of the optimization domain of the variable (ℓ, ν) .

1.1.4 ■ Worst-case, average, and chance-constrained in the optimization domain

For one given δ , $\ell(\nu, \delta)$ is a function of the optimization variable ν only; a graphical visualization of one such function is given in Figure 1.2. As δ is varied, functions $\ell(\nu, \delta)$ form a bundle called the “performance bundle”; see again Figure 1.2. The various paradigms to uncertain optimization can be described by referring to this bundle.

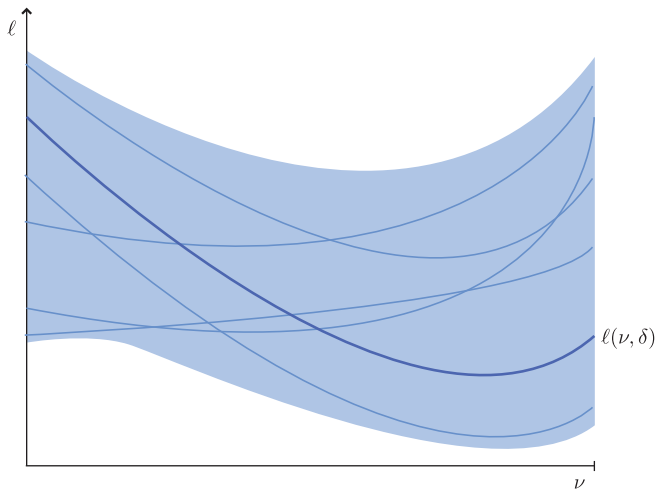


Figure 1.2. The performance bundle. The dark blue solid curve represents the cost function $\ell(\nu, \delta)$ for a given outcome of uncertainty δ . As δ is varied, functions $\ell(\nu, \delta)$ form the performance bundle.

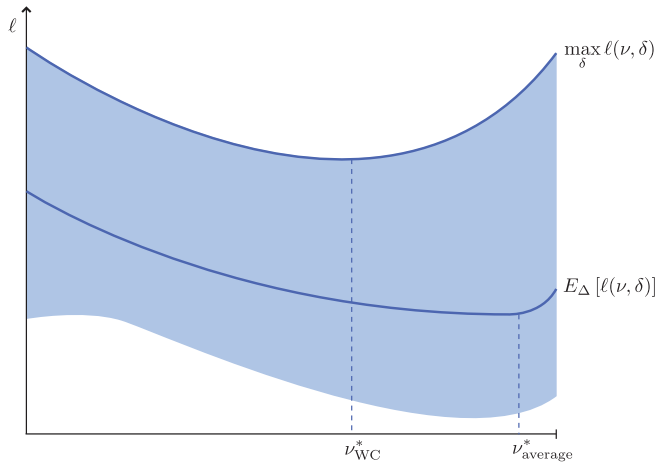


Figure 1.3. Worst-case and average cost functions. The worst-case cost function is the top border of the performance bundle, while the average cost function is the curve in the barycentric position.

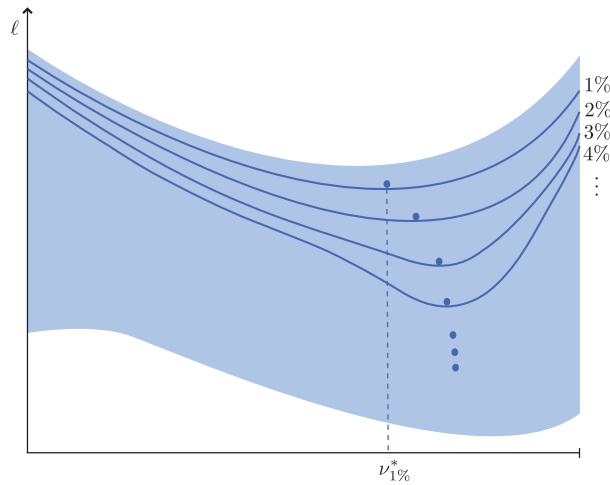


Figure 1.4. Chance-constrained cost functions. The curves marked with 1%, 2%, ... are called the chance-constrained cost functions at risk 1%, 2%, ..., and represent, ν by ν , the best possible cost value that can be achieved by leaving out 1%, 2%, ... of the loss functions.

In Figure 1.3, the top border of the performance bundle represents the worst-case cost function $\max_{\delta \in \Delta} \ell(\nu, \delta)$, and ν_{WC}^* is the worst-case minimizer.

The average cost optimization problem instead corresponds to cutting the performance bundle along the vertical direction corresponding to each value of ν , and then averaging over the δ 's to determine $\int_{\Delta} \ell(\nu, \delta) d\mathbb{P}$; see again Figure 1.3. As ν varies, $\int_{\Delta} \ell(\nu, \delta) d\mathbb{P}$ gives the average cost function, whose minimizer $\nu_{average}^*$ is called the average minimizer.

Finally, chance-constrained optimization is depicted in Figure 1.4. The curves marked with 1%, 2%, ..., called the chance-constrained cost functions at risk 1%, 2%, ..., represent, ν by ν , the best possible cost value that can be achieved by leaving out

1%, 2%, ... of the loss functions. Corresponding to each given ν , the best cost value is obtained by moving down from the top border of the bundle until 1%, 2%, ... of the loss functions are left above. By minimizing the curve marked with, say, 1%, the minimizer $\nu_{1\%}^*$ is obtained, which carries a risk of 1% that the corresponding optimal value is not achieved.

Clearly, as the risk is increased, the value improves. Moreover, one should note that each risk leads to a different design. This is not surprising: For example, in portfolio optimization a risk seeker ready to run a risk of 20% will probably invest most of her/his capital in the stock market, while a more prudent investor who chooses a risk of 1% will place most of her/his savings in a bank.

1.2 ■ The scenario approach

The scenario approach is introduced here under the assumption that the function $\ell(\nu, \delta)$ is convex in the optimization variable ν for any given δ . Later in this chapter, we shall extend this setup to also include uncertain constraints, which will also be assumed to be convex sets in the optimization domain. Thus, convexity in ν is a standing assumption in this book. While convexity defines a very broad framework, and indeed many problems are convex or can be equivalently recast so that they become convex, convexity still sets a limitation on the use of the scenario approach. Attempts to extend it beyond convexity have produced interesting results, some of which will be mentioned in Chapter 8. The dependence of $\ell(\nu, \delta)$ on δ is instead totally arbitrary.¹

We recall that a function $f(\nu)$, $\nu \in \mathbb{R}^{d-1}$, is convex if, for all ν' , ν'' , and $\alpha \in [0, 1]$, it holds that

$$f(\alpha \nu' + (1 - \alpha) \nu'') \leq \alpha f(\nu') + (1 - \alpha) f(\nu'').$$

Graphically, convexity of f means that the line segment between every two points on the graph of f lies above the graph of f ; see Figure 1.5.

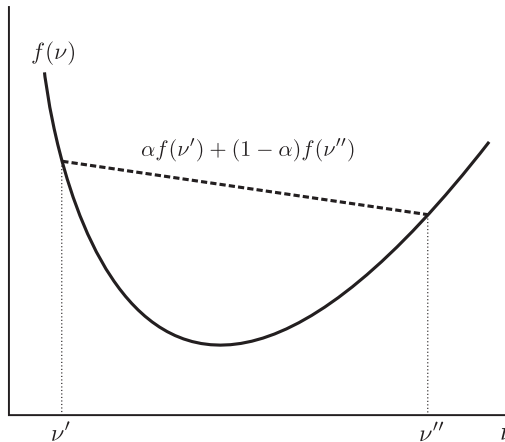


Figure 1.5. A convex function. The line segment between every two points on the graph of a convex function f lies above the graph of f .

¹This is up to measurability issues, which are glossed over throughout this book.

The scenario algorithm is quite simple to describe.

Let $\delta_1, \delta_2, \dots, \delta_N$ be N outcomes of the uncertainty variable, hereafter called *scenarios*, sampled independently² of one other from Δ according to the probability measure $\mathbb{P}$. The scenarios are used to approximate Δ , where the word “approximate” has a very precise meaning that will become clear in light of the scenario theory, where we also discuss how large N must be. Approximating Δ , which has normally an infinite cardinality, with a finite set of N elements is convenient to develop practically implementable algorithms.

In its simplest formulation, the scenario approach consists of applying the worst-case philosophy when only $\delta_1, \delta_2, \dots, \delta_N$ are considered. This gives the program

$$\min_{\nu \in \mathbb{R}^{d-1}} \left[\max_{i=1, \dots, N} \ell(\nu, \delta_i) \right] \quad (1.4)$$

(see Figure 1.6 for a pictorial representation), which is called a scenario program with N scenarios (SP_N). Since max of convex functions is a convex function, SP_N is a convex program that can be solved by means of standard optimization tools, such as the openly distributed CVX [57, 56], or YALMIP [63]. The solution to SP_N is denoted by ν^* , while its optimal value is ℓ^* .

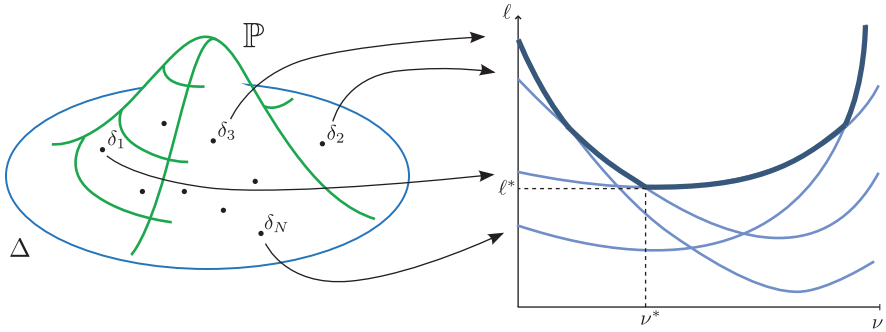


Figure 1.6. A pictorial representation of the scenario approach. The scenario program is based on a sample of N scenarios from Δ .

ν^* defines a design or a choice. It can, e.g., be the parameters defining a filter, or it can determine the allocation of resources on various tasks. If the corresponding worst-case cost ℓ^* is satisfactory, then one may be willing to adopt ν^* . However, when ν^* is applied to a new situation δ , the incurred cost is given by $\ell(\nu^*, \delta)$, a value that has not been experienced before, and it is natural to ask what the probability is that $\ell(\nu^*, \delta) > \ell^*$. This is indeed a deep question that relates the observed situations $\delta_1, \delta_2, \dots, \delta_N$ to what has not been experienced yet. In other words, what we are asking for is a *generalization* theory. Before presenting this theory, we are well advised to make our discussion more concrete through some examples.

1.2.1 ■ Example: Feed-forward noise compensation

Consider the autoregressive moving average (ARMA) system

$$y_{t+1} = a y_t + b u_t + c w_t + d w_{t-1}, \quad (1.5)$$

²Independence is a fundamental assumption that applies throughout.

where u_t and y_t are input and output, and w_t is a white noise with zero mean and unitary variance $WN(0, 1)$. a , b , c , and d are parameters whose values are not precisely known.

The noise w_t is measured by a sensor, and the objective is to design a feed-forward compensator with structure

$$u_t = \nu_1 w_t + \nu_2 w_{t-1}$$

that minimizes the steady-state variance of y_t ; see Figure 1.7.

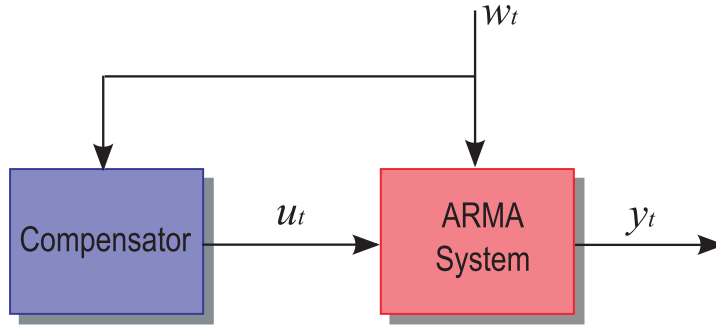


Figure 1.7. Feed-forward compensation scheme. The disturbance source w_t is measured and processed by the compensation unit that generates the control action u_t with the objective of minimizing the effect of w_t on y_t .

If the system parameters a , b , c , and d were known, an optimal compensator would be easily found. Indeed, substituting $u_t = \nu_1 w_t + \nu_2 w_{t-1}$ in (1.5) gives

$$y_{t+1} = a y_t + (c + b \nu_1) w_t + (d + b \nu_2) w_{t-1},$$

from which the expression for the steady-state variance of y_t is computed as

$$E[y_t^2] = \frac{(c + b \nu_1)^2 + (d + b \nu_2)^2 + 2a(c + b \nu_1)(d + b \nu_2)}{1 - a^2},$$

as seen in [84]. Hence, if $b \neq 0$ (controllability condition), the values of ν_1 and ν_2 minimizing $E[y_t^2]$ are

$$\nu_1 = -\frac{c}{b}, \quad \nu_2 = -\frac{d}{b}, \quad (1.6)$$

resulting in $E[y_t^2] = 0$.

However, the system parameter values are not always available in practical situations. More realistically, the parameters are only partially known, and the choice of the compensator parameters ν_1 and ν_2 has to be made taking into account the various dynamical behaviors that the system can possibly have. Thus, in this example we have $\nu = [\nu_1 \ \nu_2]^T$, $\delta = [a \ b \ c \ d]^T$, and $\ell(\nu, \delta) = E[y_t^2]$.

We do not carry on this example by assigning Δ and $\mathbb{P}$ at this stage. We will do so later in this chapter when we resume this example after the theoretical results are presented. Here, we just say that the scenario paradigm consists of sampling N scenarios $\delta_i = [a_i \ b_i \ c_i \ d_i]^T$ according to probability $\mathbb{P}$,³ and then solving

³References [22, 21, 86] provide algorithms to perform random sampling for various $\mathbb{P}$ and Δ .

program (1.4), which in this case is written as

$$\min_{\nu_1, \nu_2} \left[\max_{i=1, \dots, N} \frac{(c_i + b_i \nu_1)^2 + (d_i + b_i \nu_2)^2 + 2a_i(c_i + b_i \nu_1)(d_i + b_i \nu_2)}{1 - a_i^2} \right].$$

The probability that $\ell(\nu^*, \delta) > \ell^*$ is interpreted here as the probability that one more system will attain an output variance larger than ℓ^* when the feed-forward unit has parameters ν_1^* and ν_2^* .

1.2.2 ■ More examples: Data-driven optimization

In the example of the previous section, uncertainty was described by the couple $(\Delta, \mathbb{P})$, which was chosen by the user and used to generate scenarios. Hence, selecting $(\Delta, \mathbb{P})$ was part of the modeling of the problem, and the issue arises as to how a suitable $(\Delta, \mathbb{P})$ should be selected. This issue of obtaining a reliable description of uncertainty is common to problems that are found in diverse fields. Consider, for example, portfolio optimization, where the optimization variables are the percentages of capital placed on the assets, while the rates of return are uncertain owing to an abundance of diverse and somehow elusive causes.

It is important to remark on the fundamental fact that for the scenario approach to be applied, one is only required to be in possession of enough scenarios, no matter how the scenarios have been obtained. Thus, it can well be the case that scenarios are collected as observations, a situation that is relevant to a vast range of applications. This allows us to move into the realm of inductive methods. In this section, we briefly present a few examples to clarify this use of the scenario approach. Some of these examples will be resumed and discussed more in depth in subsequent chapters.

The first example is estimation with layers. Suppose we are given N points $(u_i, y_i) \in \mathbb{R}^2$ coming from an unknown distribution $\mathbb{P}$, sampled independently of each other. Our goal is to construct the thinnest layer centered around a quadratic polynomial with tunable coefficients that contains all the points; see Figure 1.8.

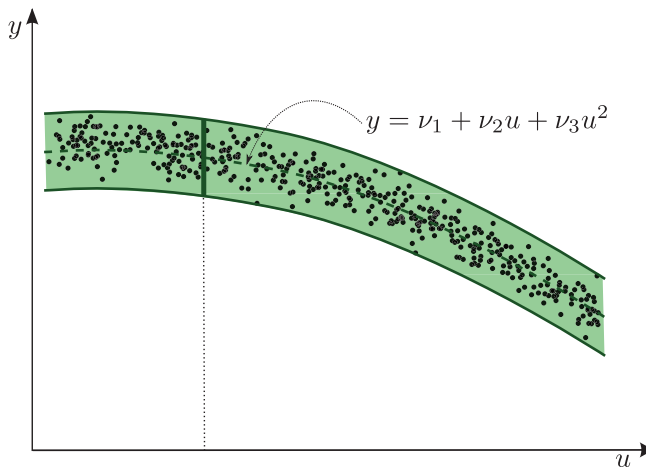


Figure 1.8. Estimation with a layer. The thinnest layer that contains all observations is used for estimation purposes.

This task amounts to solving the scenario program

$$\min_{\nu_1, \nu_2, \nu_3} \left[\max_{i=1, \dots, N} |y_i - [\nu_1 + \nu_2 u_i + \nu_3 u_i^2]| \right]. \quad (1.7)$$

The interpretation is that u is an explanatory variable and y is the variable to be estimated. For example, u can be the outcome of a medical inspection and y the degree of a disease. $\mathbb{P}$ describes a population, and $\delta_i = (u_i, y_i)$ are observed members of the population. The next time a patient $\delta = (u, y)$ is given a medical inspection and outcome u is determined, the degree of the disease is estimated to lie in the interval obtained by intersecting the layer with the vertical line generated from u ; see again Figure 1.8. The probability that $\ell(\nu^*, \delta) = |y - [\nu_1^* + \nu_2^* u + \nu_3^* u^2]| > \ell^*$ is the probability that δ is outside the layer; that is, the degree of disease of the patient is misevaluated. Estimation with layers is considered in more detail in Chapter 6.

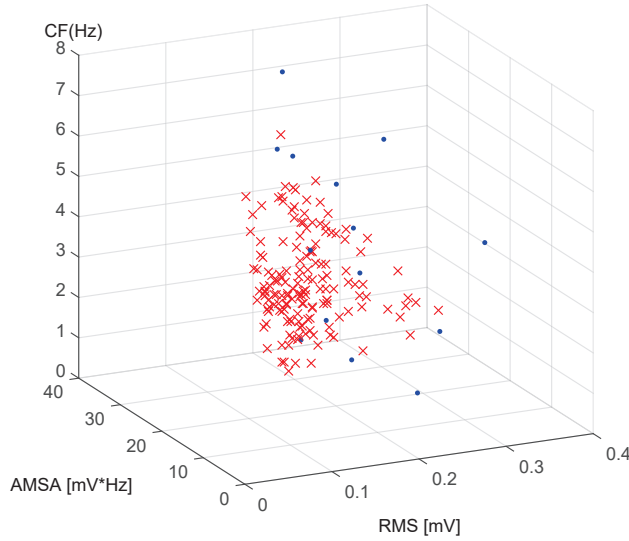


Figure 1.9. *Patients with cardiac arrest. Each patient is represented as a point in the 3D space of amplitude spectral area (AMSA), root mean square (RMS), and centroid frequency (CF). Blue dot/red cross = successful/unsuccessful resuscitation.*

Next, we consider an example in machine learning. In Figure 1.9, data are shown for 170 patients with out-of-hospital cardiac arrest treated by the emergency medical services of Spedali Civili of Brescia, Italy, between January and December 2007. Each patient is represented as a point in a 3D space, where the coordinates are attributes of the electrocardiographic trace recorded during the cardiac arrest. These attributes are amplitude spectral area (AMSA), root mean square (RMS), and centroid frequency (CF). These patients were treated with a defibrillator. The patient was labeled blue (blue dot in the figure) if the first electrical shock resulted in the restoration of an organized electrical activity of the heart; otherwise the patient was labeled red (red cross in the figure). Giving an unsuccessful shock affects the patient's conditions negatively. If one knew in advance that the shock will be unsuccessful, one should undertake a different therapy first, e.g., give a cardiac massage or inject appropriate substances. Similarly to estimation with layers, the goal

here is prediction: We want to predict whether the heartbeat will be restored by the electrical shock. In practice, a properly designed predictor can be embedded in an “intelligent defibrillator,” which, when placed on a patient in cardiac arrest, records a short electrocardiographic trace, e.g., 4 seconds long, extracts the three attributes AMSA, RMS, and CF, and predicts the effectiveness of the electrical shock providing fundamental information on the therapy to follow. In contrast with estimation with layers where the output was an interval, here the predictor has a binary output: The electrical shock is or is not effective. One such predictor is called a *classifier*. Figure 1.10 shows a classifier designed using the data in Figure 1.9. A patient whose attributes are in the blue region is predicted to react positively to the electrical shock.

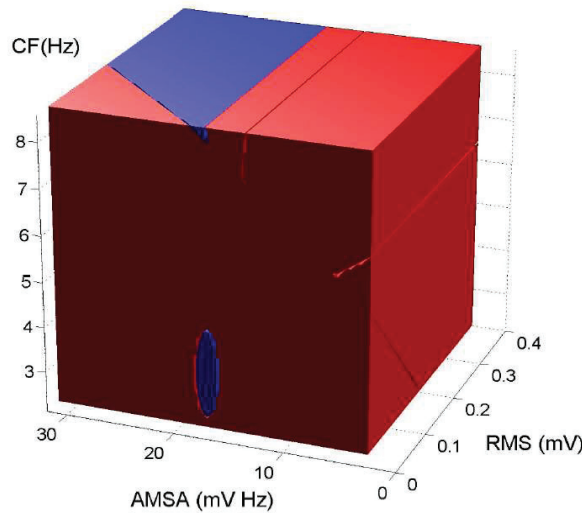


Figure 1.10. A classifier. A patient who falls in the blue region is classified as having a fibrillation condition that is resolved after the electrical shock is applied; the opposite holds for the red region.

We will not discuss the details of this application here. Suffice to say that the classifier is constructed with the scenario approach, and the probability of $\ell(\nu^*, \delta) > \ell^*$ is the probability of a misclassification. Machine learning applications are the subject of Chapter 7.

A third example is value-at-risk portfolio selection. Suppose we have collected the daily rates of return

$$R_i^k = \frac{P_i^{c,k} - P_i^{o,k}}{P_i^{o,k}}$$

of $k = 1, \dots, d-1$ assets over $i = 1, \dots, N$ past days, where $P_i^{c,k}$ is the closing price of asset k on day i , and $P_i^{o,k}$ is the opening price on the same day. The scenario program is

$$\min_{\nu_1, \dots, \nu_{d-1}} \left[\max_{i=1, \dots, N} \left(- \sum_{k=1}^{d-1} \nu_k R_i^k \right) \right], \quad (1.8)$$

where $\delta_i = [R_i^1 \dots R_i^{d-1}]^T$ and $\sum_{k=1}^{d-1} \nu_k R_i^k$ is the portfolio return over day i . Selecting $\nu = [\nu_1 \dots \nu_{d-1}]^T$ as indicated by formula (1.8) corresponds to choosing the investment strategy that, if it had been applied over the past N days, would have produced the minimal loss on the worst day. This approach certainly has a drawback in that it is very conservative, and one would probably like to reduce the conservatism in favor of a higher return. How this can be done within the scenario approach is discussed in Section 1.2.5. Here, we only remark that the probability of $\ell(\nu^*, \delta) = -\sum_{k=1}^{d-1} \nu_k^* R^k > \ell^*$ is the probability of incurring tomorrow a loss bigger than the largest loss that would have been experienced in the past with the chosen portfolio.

1.2.3 ■ A theoretical result

As anticipated, and as shown by the examples, a generalization theory is required to make the user aware of how well the scenario solution will behave when applied to a new situation. Precisely, the scenarios $\delta_1, \delta_2, \dots, \delta_N$ are visible uncertainty outcomes—outcomes we know and that are used in the scenario program SP_N to construct a solution. ℓ^* is the worst-case loss over $\delta_1, \delta_2, \dots, \delta_N$ incurred by the solution, and the question is whether this ℓ^* is an upper bound to the loss paid for other δ 's. This generalization question points to inferring the “invisible” from the “visible,” the yet unseen δ 's from the seen scenarios δ_i 's.

An answer to this generalization question is provided by the next theorem, which is quoted here without proof. Full rigor is sacrificed in favor of readability. For example, conditions for the existence and uniqueness of the solution are not discussed. Chapter 3 contains a rigorous and more comprehensive theory from which the result given here follows.

Theorem 1.1. *For any $\epsilon \in (0, 1)$ (risk parameter) and $\beta \in (0, 1)$ (confidence parameter), if the number of scenarios N satisfies $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, then, with probability $\geq 1 - \beta$, it holds that ℓ^* is ϵ -risk guaranteed, that is, $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$.*

The statement of this theorem deserves some explanation. In the expression $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$, $\mathbb{P}$ refers to the occurrence of a new instance of the uncertain element δ , and $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\}$ must be interpreted as the probability of observing δ such that $\ell(\nu^*, \delta) > \ell^*$ for fixed values of ν^* and ℓ^* . Given $\delta_1, \delta_2, \dots, \delta_N$, the solution ν^* and the value ℓ^* are obtained by solving the scenario program SP_N , and, for a given ϵ , condition $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$ may or may not be satisfied depending on which $\delta_1, \delta_2, \dots, \delta_N$ have been seen. In other words, for different samples $(\delta_1, \delta_2, \dots, \delta_N)$, ν^* and ℓ^* are different so that whether or not $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$ depends on the sample $(\delta_1, \delta_2, \dots, \delta_N)$. The theorem states that if $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, the probability of seeing a sample $(\delta_1, \delta_2, \dots, \delta_N)$ such that $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$ is at least $1 - \beta$.

Since the theorem contains two levels of probability, ϵ and β , one can find it somehow unfriendly; we now go through its statement again and explain it gradually to increase readability. Let us pretend for a moment that the second level of probability expressed by β does not exist. Then the theorem reads “For any $\epsilon \in (0, 1)$, if the number of scenarios N satisfies $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, then it holds that ℓ^* is ϵ -risk guaranteed, that is, $\mathbb{P}\{\delta \in \Delta : \ell(\nu^*, \delta) > \ell^*\} \leq \epsilon$.”

In this form, the theorem provides an explicit expression for N so that ℓ^* is an upper bound to $\ell(\nu^*, \delta)$ with probability $1 - \epsilon$. Put differently, the chance-constrained cost function at risk ϵ in correspondence of ν^* is no more than ℓ^* . However, this statement without β cannot possibly be true. In fact, the random sample $(\delta_1, \delta_2, \dots, \delta_N)$ may be a poor descriptor of Δ , in which case no guarantee of generalization can be given. To clarify this point, think of the example of estimation with layers from Section 1.2.2: There it may happen that all N points lay on, or in the vicinity of, a quadratic function, yet the probability distribution that has generated the points is spread. If so, the layer covers a small amount of the probabilistic mass, and the prediction is unreliable. Parameter β accounts for such a possibility. In theory, β plays an important role, and pushing β down to zero makes N rise to infinity. However, from a practical point of view, the role of β is fairly marginal. This is seen in the sample complexity expression $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, where β shows up under the sign of logarithm, so that β can be made very small, say 10^{-7} , without causing an excessive increase of N . When β is assigned such a small value, one can in practice neglect it.

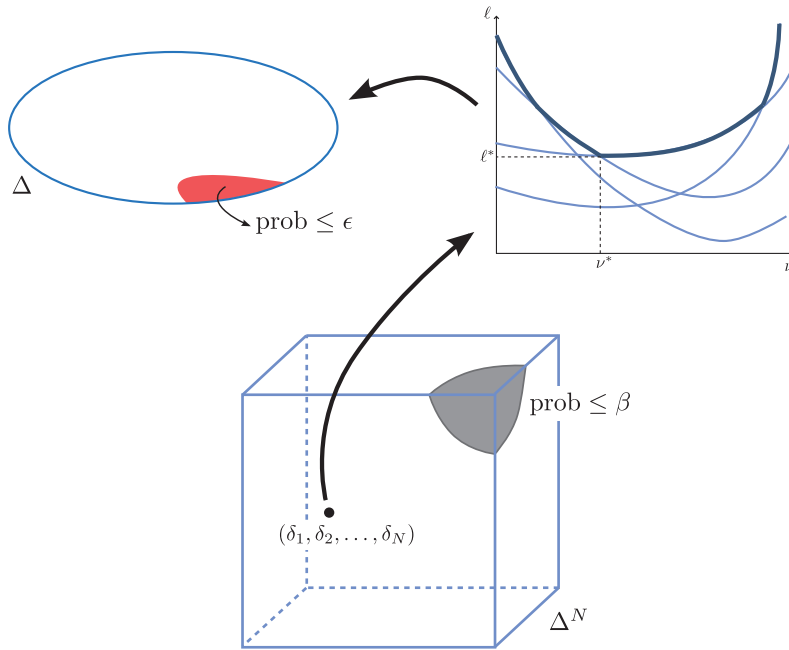


Figure 1.11. Illustration of Theorem 1.1. SP_N maps a sample $(\delta_1, \delta_2, \dots, \delta_N)$ into a convex program that has chance-constrained properties.

The theorem is graphically illustrated further in Figure 1.11. SP_N maps a sample $(\delta_1, \delta_2, \dots, \delta_N) \in \Delta^N$ into a convex program. In Δ^N there is a “bad set,” the grey region in the picture, such that, if the sample falls within the bad set, the theorem says nothing. However, the bad set is tiny, with probability, say, 10^{-7} . Any time the sample belongs to the rest of Δ^N , solving SP_N , which is an ordinary convex program, returns a solution ν^* for which cost ℓ^* is guaranteed for all other δ ’s but an ϵ fraction of them at most. Thus, the scenario approach offers a viable way to robustify any nominal design up to level ϵ .

A few remarks are in order.

- (i) When scenarios are artificially generated, new scenarios can be easily obtained to perform a tight evaluation of $\mathbb{P}\{\delta \in \Delta : \ell(\gamma^*, \delta) > \ell^*\}$. In this case, the theory of this section can be used as a guideline to perform optimization so that the solution is bound to attain desired levels of reliability. When instead scenarios are data and represent a limited and costly resource, they are better used for optimization purposes, and one is willing to obtain an evaluation of $\mathbb{P}\{\delta \in \Delta : \ell(\gamma^*, \delta) > \ell^*\}$ without splitting the set of scenarios in two: an optimization set and a validation set. The theory of this section is the tool to obtain such an evaluation since it allows one to make the claim that $\mathbb{P}\{\delta \in \Delta : \ell(\gamma^*, \delta) > \ell^*\} \leq \epsilon$ with a high confidence $1 - \beta$.
- (ii) The theorem contains a generalization result that is quite general. In fact, it is applicable to any convex uncertain optimization problem.
- (iii) The sample complexity $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$ is easy to compute, and it depends on the problem through d only. N increases with d and decreases with ϵ .
- (iv) Importantly, N is independent of $\mathbb{P}$. Before, we saw that no knowledge of $\mathbb{P}$ was required to apply the scenario algorithm. All we need are the scenarios. Here, we see that knowledge of $\mathbb{P}$ is not required at a theoretical level because the theorem has *universal validity*, that is, it is applicable to any $\mathbb{P}$. Hence, when the scenario theory is applied, one has to accept the validity of the conceptual model that the data are generated according to a probability distribution, but the actual knowledge of the distribution is not necessary.
- (v) The fact that the scenario theory does not require knowledge of $\mathbb{P}$ does not mean that prior knowledge of the problem is not a valuable source of information. In fact, designing a suitable optimization problem calls for domain knowledge and familiarity with the framework of operation. For instance, in the estimation example with layers from Section 1.2.2, one may know that the phenomenon is fundamentally linear, in which case s/he sets out to use a linear center line for the layer as opposed to a quadratic function as in Figure 1.8. These two sources of information, prior knowledge and data, are shown in Figure 1.12. Interestingly, the solution γ^* is characterized by two features: the value ℓ^* and the probability to exceed it, i.e., the risk. The former, ℓ^* , becomes known to the user after the optimization procedure is completed, so that an imperfect prior knowledge that adversely affects the solution becomes manifest through the optimization value (if, for example, data are not well described by a linear model, a layer built around a linear center line becomes large). On the other hand, the risk also depends on $\mathbb{P}$, which is not accessible, so that the risk is not directly measurable. Hence, it is reassuring that our prior knowledge, which can be imperfect, does not affect the evaluation of the risk.

Since the optimization value is known after the optimization procedure is completed, while the risk is guaranteed regardless of prior information, one can design a procedure where the value is used as a feedback variable to adjust the optimization program. For example, a layer with a low-order center line can first be built. If it turns out that the layer is large, one moves on to

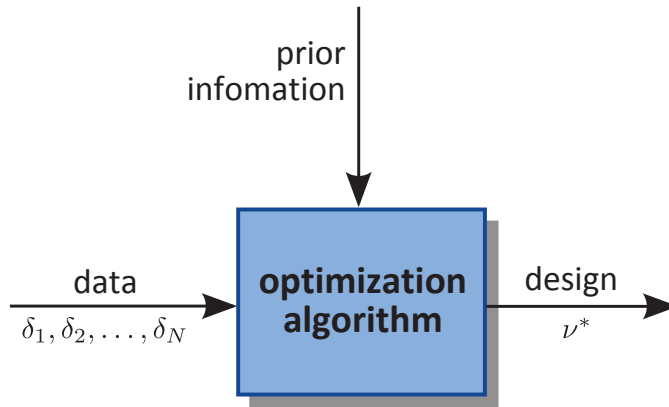


Figure 1.12. The two sources of information, prior knowledge and data, in scenario optimization.

a quadratic center line and so on until a preassigned maximum order. Along the way, one compares the layer thickness with the risk and chooses a suitable compromise between the two.⁴

1.2.4 ■ Example: Feed-forward noise compensation continued

For the example in Section 1.2.1, suppose that a , b , c , and d are assigned as functions of two parameters, σ_1 and σ_2 , that range in $[-1, 1] \times [-1, 1]$ with uniform probability. These functions are

$$\begin{aligned}
 a &= \frac{3.5\sigma_1^2 - 0.2}{3\sigma_1^2 + 0.3} \cdot (0.32\sigma_1 + 0.6), \\
 b &= 1 + \frac{\sigma_1\sigma_2^2}{10}, \\
 c &= \frac{-0.01 + (\sigma_1 + \sigma_2^2)^2}{0.02 + (\sigma_1 + \sigma_2^2)^2} \cdot \left(1 - \frac{(\sigma_1 - 1)(\sigma_2 - 1)}{2}\right), \\
 d &= \frac{0.05}{0.025 + (\sigma_1 + \sigma_2 - 2)^2}.
 \end{aligned}$$

We selected $\epsilon = 0.5\%$, $\beta = 10^{-7}$ which, when used in Theorem 1.1, gives $N = 7248$. The interpretation is that, with high confidence $1 - 10^{-7}$, the designed feed-forward unit generates an output whose variance is below ℓ^* with probability at least 99.5%. Upon solving the scenario program

$$\min_{\nu_1, \nu_2} \left[\max_{i=1, \dots, 7248} \frac{(c_i + b_i \nu_1)^2 + (d_i + b_i \nu_2)^2 + 2a_i(c_i + b_i \nu_1)(d_i + b_i \nu_2)}{1 - a_i^2} \right],$$

⁴To make this approach fully rigorous, one has to keep in mind that the confidence parameter has to be multiplied by the total number of possible attempts one makes, as one might in principle always stop the procedure at the step where the risk hits its higher value. However, as we have already noticed, β can be made very small because it appears under the sign of logarithm in the sample complexity formula, so that the “inflation” of the parameter β does not represent a serious disadvantage in practice. A broader discussion on this point is provided in Section 6.1.5.

we obtained

$$\nu_1^* = -0.9022, \quad \nu_2^* = -0.9028, \quad \ell^* = 5.8.$$

Figure 1.13 shows the domain for σ_1 and σ_2 , where the brown surface is the output variance corresponding to each plant operated with the designed compensation unit, and the plane is at level $\ell^* = 5.8$. It can be seen that the output variance is below the plane except for a small portion at the top-right corner. This portion has probability smaller than $\epsilon = 0.5\%$.

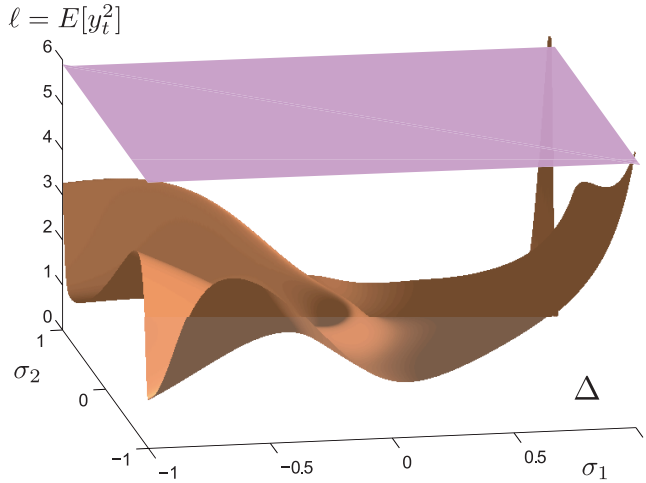


Figure 1.13. Feed-forward noise compensation example. The brown surface is the output variance of the ARMA systems when the compensation unit with parameters ν_1^* and ν_2^* is in place. The flat plane indicates the value $\ell^* = 5.8$.

1.2.5 ■ Risk-performance trade-off

As we have seen, an important feature of the chance-constrained approach is that it allows one to tune the risk against the guaranteed performance level. In the preceding section, we encountered a problem where setting $\epsilon = 0.5\%$ in the scenario approach gave a guaranteed performance level of $\ell^* = 5.8$. If this performance level is not satisfactory, one natural question to ask is whether it can be *a posteriori* improved by accepting an increase of risk. It is a fact that the scenario approach is perfectly suitable for this purpose, and how this can be achieved is presented in this section.

Once the scenarios $\delta_1, \delta_2, \dots, \delta_N$ have been obtained, the scenario program SP_N is completely defined and can be implemented on a computer. After ν^* and ℓ^* have been determined, we can further inspect SP_N for active scenarios, i.e., those such that $\ell(\nu^*, \delta_i) = \ell^*$, and deliberately discard one of these scenarios. After the new solution without the discarded scenario is computed, we can proceed further and discard another active scenario for the current program, and so on. As we discard 1, 2, $\dots$, scenarios, the solution changes to $\nu_1^*, \nu_2^*, \dots$, and the associated performance level improves to $\ell_1^*, \ell_2^*, \dots$, as shown in Figure 1.14. We can also draw a graph like the

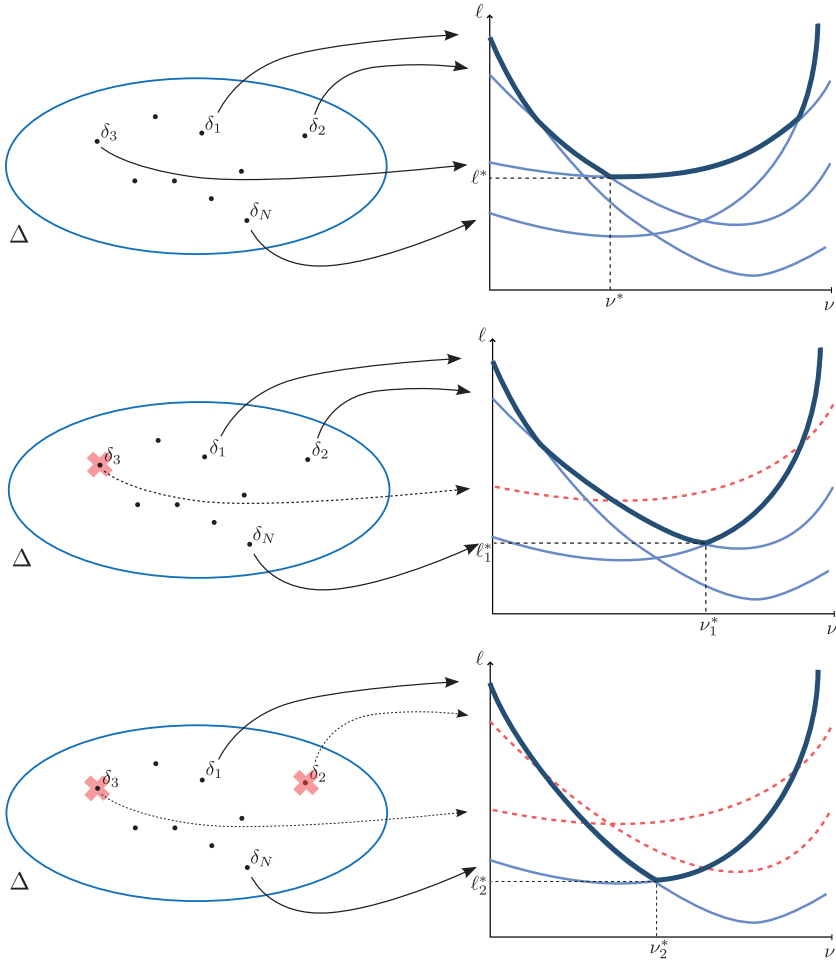


Figure 1.14. *The scenario approach with discarded scenarios. The solution performance level improves as scenarios are discarded.*

one in Figure 1.15, where the solution performance level improvement is displayed against the number k of scenarios that have been discarded. Note that this graph depends on the scenario program at hand so that a different graph is obtained in different problems.

Deliberately discarding scenarios sounds a bit like “cheating,” and it is natural to ask whether we can still provide guarantees on the risk of not attaining the obtained performance level. The answer is indeed positive, and the theorem below formalizes the result. In the theorem, ν_k^* and ℓ_k^* are the design and the associated performance level obtained after discarding k scenarios.⁵

⁵To be precise, in order for the theorem to hold, the k scenarios that have been discarded must be violated by the solution, i.e., $\ell(\nu_k^*, \delta_i) > \ell_k^*$ for all scenarios δ_i that have been discarded. This can be achieved easily: If a discarded scenario turns out not to be violated, this scenario is reinstated and another scenario is discarded in its place. The reader is referred to Chapter 3 for a more complete description of this procedure.

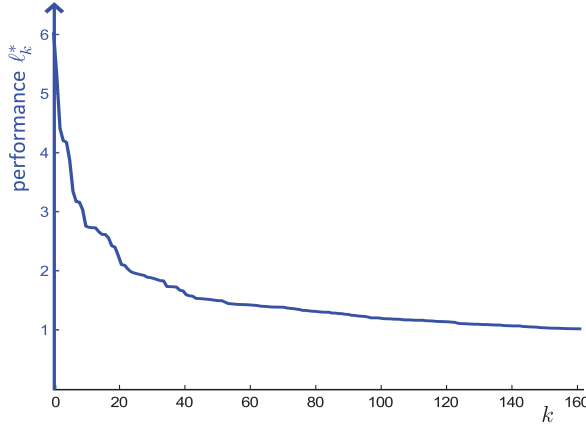


Figure 1.15. The scenario approach with discarded scenarios. The figure profiles the solution performance level as a function of the number of scenarios that have been discarded.

Theorem 1.2. With probability $\geq 1 - \beta$, it holds that ℓ_k^* is ϵ_k -risk guaranteed, that is, $\mathbb{P}\{\delta \in \Delta : \ell(\nu_k^*, \delta) > \ell_k^*\} \leq \epsilon_k$, where

$$\epsilon_k = \frac{k}{N} + \left[\frac{\sqrt{k}}{N} + \frac{\sqrt{k}+1}{N} \left((d-1)\ln(k+d-1) + \frac{d-1}{\sqrt{k}} + \ln \frac{1}{\beta} \right) \right]. \quad (1.9)$$

A few remarks clarify the result.

- (i) The expression for ϵ_k contains two terms: $\frac{k}{N}$ and the term in square brackets. $\frac{k}{N}$ is the “empirical risk,” the ratio between the number of violated scenarios over the total number of scenarios. We cannot expect the true risk to be bounded by the empirical risk. For one thing, there is stochastic fluctuation, and we demand that the result be valid with high confidence $1 - \beta$. Moreover, ν_k^* is the solution of an optimization problem, and this generates a bias towards selecting points with higher risk. The term in square brackets is a margin that accounts for these two effects. Let us inspect the term in square brackets more closely. Suppose that N and k increase so that k/N is kept constant at a value α . Then, the term in square brackets goes to zero as $O(\ln N / \sqrt{N})$, which is just slightly slower than $O(1/\sqrt{N})$. This is a remarkable result: $O(1/\sqrt{N})$ is the convergence rate of the law of large numbers in the context of estimating the probability p of a given event as the ratio between the number of independent trials that fall in the event over the total number of trials; the result in the theorem shows that if the event is not given but instead obtained through optimization, this convergence rate worsens only very marginally.
- (ii) The result in Theorem 1.2 holds true irrespective of the algorithm according to which scenarios are discarded. One can, e.g., think of discarding scenarios optimally, that is, one removes those scenarios whose removal produces the largest possible improvement. The drawback of this approach is that it is often computationally intractable. Alternatively, a greedy algorithm can be applied. Other procedures can be used as well, and in all cases, Theorem 1.2

applies. In a real problem, the procedure for scenario removal is often dictated by computational limits; at the end of the procedure one inspects the performance level achieved against the theoretical guarantees on the risk of not attaining it provided by the theorem. If the performance level risk compromise is judged to be satisfactory, the solution is adopted.

- (iii) By repeatedly applying the theorem, one can also guarantee that the risk ϵ_k is not exceeded for many values of k simultaneously. The exact procedure is illustrated in Section 3.3. Following this approach, one finally constructs a performance-risk plot like the one in Figure 1.16. Based on this plot, one selects her/his favorite compromise between performance level and risk.

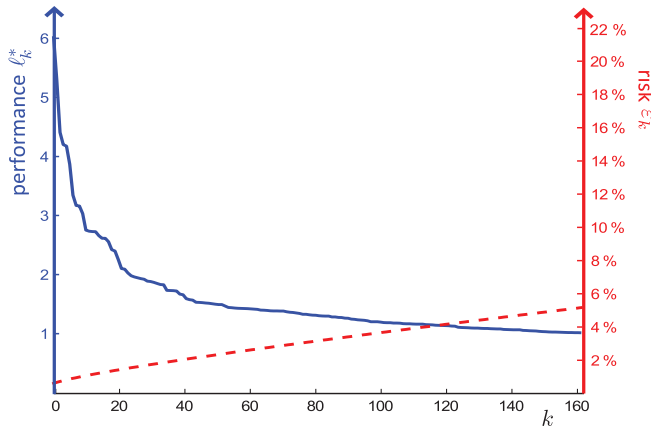


Figure 1.16. The performance-risk plot. To each k , there corresponds a performance level given by the solid blue curve and a risk given by the dashed red curve.

In actual fact, the performance-risk plot of Figure 1.16 refers to the feed-forward noise compensation example from Section 1.2.1. By an inspection of the curves, we decided to select $k = 60$, a choice that is subjective; others may have opted for a different choice. With $k = 60$, we have $\epsilon_{60} = 2.5\%$ and $\ell_{60}^* = 1.42$. This indicates an improvement of 75% over the initial value of 5.8 that had been obtained for $k = 0$. The compensator parameters are $\nu_{1,60}^* = -0.24$ and $\nu_{2,60}^* = -0.59$. Figure 1.17(a) depicts the value of $E[y_t^2]$

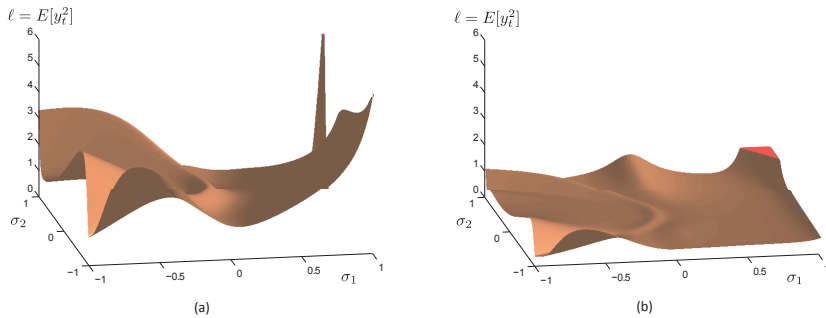


Figure 1.17. Output variance for the ARMA systems. (a) Compensator obtained for $k = 0$. (b) Compensator obtained for $k = 60$.

achieved for the ARMA systems by the compensator corresponding to $k = 0$; Figure 1.17(b) gives the same graph for $k = 60$. In the two figures, the flat red zones correspond to the regions where the cost value is above 5.8 and 1.42, respectively.

- (iv) For $k = 0$, Theorem 1.2 covers the same situation as Theorem 1.1. The reason they look different is simply because in Theorem 1.1 an explicit formula with respect to N is given, while in Theorem 1.2 the choice was made to make it explicit with respect to ϵ_k . Both formulas follow from a general result given in Chapter 3.

1.3 ■ Generalizations

Up until now, we have been dealing with the problem of minimizing a loss function $\ell(\nu, \delta)$ that incorporates an uncertainty parameter δ . We now turn briefly to the more general problem of minimizing a linear cost function $c^T \theta$, $\theta \in \Theta \in \mathbb{R}^d$, with constraints Θ_δ parameterized by an uncertainty parameter δ . Note that this latter problem is more general than the former, since the former can be rewritten by an epigraphic reformulation in the format of the second problem. To this end, introduce the auxiliary variable ℓ , and minimize ℓ subject to $\ell \geq \ell(\nu, \delta)$. Here, $\theta = (\nu, \ell)$,⁶ $c = [0 \cdots 0 \ 1]^T$, and Θ_δ is the set where ℓ is greater than or equal to $\ell(\nu, \delta)$. Also, note that linearity of the cost function $c^T \theta$ is with no loss of generality within a convex setup. In fact, should the cost be a convex function $f(\theta)$ instead of a linear function $c^T \theta$, again by an epigraphic reformulation one might minimize the auxiliary variable h over the augmented domain $\{(\theta, h) : h \geq f(\theta)\} \cap \Theta \times \mathbb{R}$ with the constraints parameterized by δ written as $(\theta, h) \in \Theta_\delta \times \mathbb{R}$.

For the problem of minimizing $c^T \theta$ with constraints $\theta \in \Theta_\delta$, where Θ_δ are convex sets, a scenario program is written as

$$\begin{aligned} \text{SP}_N : \min_{\theta \in \Theta} c^T \theta \\ \text{subject to } \theta \in \bigcap_{i=1, \dots, N} \Theta_{\delta_i}. \end{aligned} \quad (1.10)$$

Letting θ^* be the solution of (1.10), Theorem 1.1 generalizes to cover this context and becomes the following.

Theorem 1.3. *For any $\epsilon \in (0, 1)$ (violation parameter) and $\beta \in (0, 1)$ (confidence parameter), if the number of scenarios N satisfies $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, then, with probability $\geq 1 - \beta$, it holds that $\mathbb{P}\{\delta \in \Delta : \theta^* \notin \Theta_\delta\} \leq \epsilon$.*

In this context, ϵ is referred to as a violation parameter since it bounds the probability that the scenario solution violates a new constraint $\theta \in \Theta_\delta$.

Scenario removal procedures can be applied in the context of (1.10) (in this case, discarding a scenario corresponds to discarding a constraint, a thorough discussion of which will be given in Chapter 3). Similarly to Theorem 1.1, Theorem 1.2 straightforwardly generalizes to the new setup as well.

⁶ (ν, ℓ) has $(d-1) + 1 = d$ variables, which justify that $\theta \in \mathbb{R}^d$ in this section.

Chapter 2

Problems in control

Many problems in control encompass uncertain elements due to partial knowledge of the plant dynamics or, more simply, to the presence of external disturbance signals acting on the plant. In this chapter, we consider some of these problems and show that the scenario approach offers an attractive route to find suitable solutions.

2.1 ■ Robust control problems

All of the problems in this section deal with structural uncertainty, i.e., uncertainty arising from imprecise knowledge of the plant dynamics. Later, uncertainty arising from external disturbance signals will be considered.

2.1.1 ■ Quadratic stability

The linear system

$$\dot{x}(t) = Ax(t), \quad (2.1)$$

where $x(t) \in \mathbb{R}^n$ is the state variable, $\dot{x}(t)$ is the time derivative of $x(t)$, and A is the $n \times n$ state matrix, is said to be *asymptotically stable* if the state $x(t)$ converges to zero for any initial condition $x(0)$. A necessary and sufficient condition for this to happen is given by Lyapunov theory as described in the following. Consider a quadratic function $V(x) = x^T P x$ where matrix P is symmetric and positive definite, $P \succ 0$. This is an elliptic paraboloid, as illustrated in Figure 2.1. The time derivative of $V(x(t))$ is

$$\begin{aligned} \frac{d}{dt} V(x(t)) &= \dot{x}(t)^T P x(t) + x(t)^T P \dot{x}(t) \\ &= x(t)^T [A^T P + PA] x(t), \end{aligned}$$

so that if $A^T P + PA$ is negative definite ($A^T P + PA \prec 0$), then $\frac{d}{dt} V(x(t)) < 0$ whenever $\|x(t)\| \neq 0$, and this implies that $x(t) \rightarrow 0$, and the system is therefore asymptotically stable. It is a remarkable fact that the converse also holds true; that is, if the system is asymptotically stable, there certainly exists a symmetric matrix $P \succ 0$ such that $A^T P + PA \prec 0$.⁷ This gives the following Lyapunov stability result:

⁷The reader may be interested in showing that one such P is given by the expression $P = \int_0^\infty \exp(A^T \tau) \exp(A \tau) d\tau$.

System (2.1) is asymptotically stable if and only if there exists a symmetric matrix $P \succ 0$ such that $A^T P + PA \prec 0$. P is called a “Lyapunov matrix” for A .

Consider now the uncertain linear system

$$\dot{x}(t) = A(\delta)x(t). \quad (2.2)$$

System (2.2) is said to be *quadratically stable* if there exists a symmetric matrix P that is simultaneously Lyapunov for all matrices $A(\delta)$, that is, $P \succ 0$ and $A(\delta)^T P + PA(\delta) \prec 0, \forall \delta$. Note that quadratic stability is a stronger condition than *robust stability*, which is the requirement that $\dot{x} = A(\delta)x$ be asymptotically stable $\forall \delta$. In fact, for robust stability to hold, it is enough that each matrix $A(\delta)$ has associated a Lyapunov matrix $P(\delta)$, but different $P(\delta)$ can be Lyapunov for distinct values of δ , while quadratic stability requires that the same matrix P be Lyapunov for all δ ’s simultaneously. The reader is referred to [37, 86, 66] for a presentation of quadratic stability and its properties.

Suppose now that we are given an uncertain linear system of the form (2.2) and a description of the range Δ for δ , and we want to study the quadratic stability of (2.2). How do we address this problem? To ease the notation, start by noting that the two conditions $P \succ 0$ and $A(\delta)^T P + PA(\delta) \prec 0, \forall \delta$, can be put together in the following linear matrix inequality (LMI):⁸

$$\begin{bmatrix} -P & 0 \\ 0 & A^T(\delta)P + PA(\delta) \end{bmatrix} \prec 0, \quad \forall \delta \in \Delta. \quad (2.4)$$

How do we verify the validity of (2.4)? Since normally Δ contains infinitely many elements δ , verifying (2.4) numerically is in general intractable. An exception occurs when the set of matrices $A(\delta)$, $\delta \in \Delta$, forms a polytope; that is, the set of matrices is the convex combination of finitely many vertex matrices $A_1, A_2, \dots, A_p$, in which case it is enough to check condition (2.4) for all the vertices A_i only; see, e.g., [19], [17], or [86].

⁸Given a decision vector $\theta = [\theta_1, \theta_2, \dots, \theta_d]^T$, a linear matrix inequality is an expression of the form

$$M_0 + \theta_1 M_1 + \dots + \theta_d M_d \prec 0 \quad [\preceq 0], \quad (2.3)$$

where $M_0, M_1, \dots, M_d$ are given symmetric $n \times n$ matrices and $\prec 0$ [$\preceq 0$] denotes negative definiteness [semidefiniteness]. See [18]. It is easy to show that any expression $M(\theta) \prec 0$ [$\preceq 0$], where $M(\theta)$ is a square matrix whose entries are affine functions of θ , can be rewritten in the LMI form in (2.3) and vice versa.

An LMI is a convex constraint. This can be readily verified as follows:

$$\begin{aligned} \{\theta \in \mathbb{R}^d : M(\theta) \prec 0 [\preceq 0]\} &= \{\theta \in \mathbb{R}^d : x^T M(\theta)x < 0 [\leq 0], \forall x \in \mathbb{R}^d, x \neq 0\} \\ &= \bigcap_{x \in \mathbb{R}^d, x \neq 0} \{\theta \in \mathbb{R}^d : x^T M(\theta)x < 0, [\leq 0]\}, \end{aligned}$$

which is the intersection of (an uncountable number of) affine constraints in θ (keep in mind that M is affine in θ so that for any given x , $x^T M(\theta)x$ is an affine expression in θ). It is also worth noting that an LMI constraint can have “curvy” boundaries despite it being generated by affine constraints, and this is due to the fact that the latter are infinitely many.

LMIs have become very popular in recent years because

- they supply a very rich class of constraints that allows one to model diverse requirements; in particular, it turns out that many problems in systems and control can be expressed by means of LMIs [17, 4];
- there exist reliable optimization solvers, like those in the packages CVX [57, 56] and YALMIP [63], that deal efficiently with LMIs.

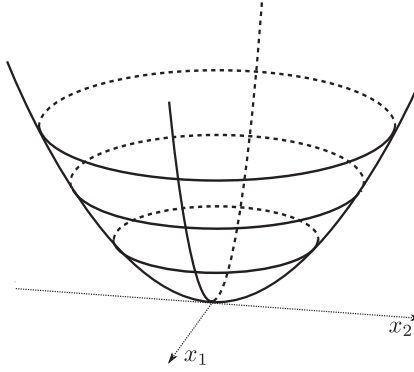


Figure 2.1. A quadratic function $V(x) = x^T P x$ with $P > 0$.

While checking condition (2.4) exactly is very hard in general, we want to show that a probabilistic relaxation of this problem can be addressed through the scenario approach.

Notice first that condition (2.4) is a feasibility condition, while the scenario approach introduced in Chapter 1 addresses optimization problems. To move towards an optimization reformulation, consider the problem

$$\begin{aligned} & \min_{\alpha, P} \alpha & (2.5) \\ & \text{subject to } -I \leq \begin{bmatrix} -P & 0 \\ 0 & A^T(\delta)P + PA(\delta) \end{bmatrix} \leq \alpha I, \quad \forall \delta \in \Delta. \end{aligned}$$

The connection with the feasibility problem (2.4) is as follows. In (2.5), one seeks a P so that the inner matrix in the LMI condition is “pushed down” as much as possible for all δ ’s in an attempt to make α negative. Note also that the lower bound $-I$ has been introduced since otherwise, when the inner matrix can be made negative definite, α escapes to minus infinity due to the linear dependence of the matrix on P . Suppose now that a probability measure $\mathbb{P}$ over Δ is assigned. $\mathbb{P}$ can describe our belief that certain situations are more likely to occur than others, or more simply it reflects our interest in various values of δ . If Δ has a finite volume, $\mathbb{P}$ is at times taken to be just the normalized volume [8, 86]. Then, one can build the scenario counterpart of (2.5) as follows:

$$\begin{aligned} & \min_{\alpha, P} \alpha & (2.6) \\ & \text{subject to } -I \leq \begin{bmatrix} -P & 0 \\ 0 & A^T(\delta_i)P + PA(\delta_i) \end{bmatrix} \leq \alpha I, \quad i = 1, \dots, N, \end{aligned}$$

where the scenarios δ_i are independently extracted according to $\mathbb{P}$.

If the optimal value α^* of (2.6) is ≥ 0 , clearly no quadratic stability holds. On the other hand, Theorem 1.3 says that for N , ϵ , and β linked to each other according to the theorem, with probability $1 - \beta$ the inner matrix in the LMI condition in (2.5) with $P = P^*$ is smaller than $\alpha^* I$ for all but an ϵ -fraction of δ ’s so that, with probability $1 - \beta$, finding $\alpha^* < 0$ implies that P^* is simultaneously Lyapunov for a probabilistic fraction of $1 - \epsilon$ of the plants.

2.1.2 ■ State-feedback quadratic stabilization

Consider now the uncertain linear system

$$\dot{x}(t) = A(\delta)x(t) + B(\delta)u(t), \quad (2.7)$$

where $u(t) \in \mathbb{R}^p$ is the control input. We want to quadratically stabilize (2.7) by a state-feedback control law $u(t) = Kx(t)$. The closed-loop state matrix is $A(\delta) + B(\delta)K$, so that according to (2.4) the closed-loop system is quadratically stable if there exists a symmetric matrix P such that

$$\begin{bmatrix} -P & 0 \\ 0 & A^T(\delta)P + K^T B(\delta)^T P + PA(\delta) + PB(\delta)K \end{bmatrix} < 0, \quad \forall \delta \in \Delta. \quad (2.8)$$

Thus, the quadratic stabilization problem can be formulated as finding a couple (K, P) such that (2.8) holds true.

Here, a new difficulty arises in which the matrix in this condition is not linear in (K, P) , so that (2.8) is not an LMI and does not assign a convex constraint. This difficulty can be circumvented as follows. Condition (2.8) requires that $P \succ 0$. A symmetric positive definite matrix is always invertible, and the inverse is also positive definite. Let $Q = P^{-1}$. Multiplying both sides of the LMI in (2.8) to the left and to the right by Q , one obtains

$$\begin{bmatrix} -Q & 0 \\ 0 & QA^T(\delta) + QK^T B(\delta)^T + A(\delta)Q + B(\delta)KQ \end{bmatrix} < 0, \quad \forall \delta \in \Delta, \quad (2.9)$$

and the existence of a Q such that (2.9) holds is equivalent to the existence of a P such that (2.8) holds. We can further define $Y = KQ$. Notice that Y is a $p \times n$ matrix that can be freely selected for any given $Q \succ 0$ by properly choosing K . Therefore, (Y, Q) is an equivalent reparameterization of the unknown (K, P) of the original problem. Hence, we conclude that the condition for the quadratic stabilization of system (2.7) is that there exist a Y and a symmetric matrix Q satisfying the condition

$$\begin{bmatrix} -Q & 0 \\ 0 & QA^T(\delta) + Y^T B(\delta)^T + A(\delta)Q + B(\delta)Y \end{bmatrix} < 0, \quad \forall \delta \in \Delta.$$

Moreover, K can be reconstructed from Q and Y by the relation $K = YQ^{-1}$.

Now, the problem of finding Y and Q can be dealt with through scenario optimization tantamount to the quadratic stability problem of Section 2.1.1. Here we give a numerical example of this approach.

Example 2.1. Given the uncertain system

$$\dot{x}(t) = \begin{bmatrix} 0.5\delta_2 & 1 + \delta_1 \\ -(1 + \delta_1)^2 & 2(0.1 + 0.5\delta_2)(1 + \delta_1) \end{bmatrix} x(t) + \begin{bmatrix} 10 \\ 15 \end{bmatrix} u(t),$$

with $|\delta_1| \leq 1$, $|\delta_2| \leq 1$, consider the scenario program

$$\begin{aligned} & \min_{\alpha, Y, Q} \alpha \\ & \text{subject to } -I \preceq \begin{bmatrix} -Q & 0 \\ 0 & QA^T(\delta_i) + Y^T B^T + A(\delta_i)Q + BY \end{bmatrix} \preceq \alpha I, \quad i = 1, \dots, N. \end{aligned}$$

With $\epsilon = 5\%$ and $\beta = 10^{-10}$, from Theorem 1.3 we obtain $N = 1162$. After sampling $N = 1162$ values of δ uniformly from the square $[-1, 1] \times [-1, 1]$, the scenario program was solved, and the solution turned out to be

$$\alpha^* = -0.00018 \quad Y^* = \begin{bmatrix} -0.0132 & -0.0199 \end{bmatrix}, \quad Q^* = \begin{bmatrix} 0.0026 & 0.0043 \\ 0.0043 & 0.1063 \end{bmatrix},$$

so that

$$K^* = Y^*(Q^*)^{-1} = [-5.1091 \quad 0.0195].$$

An *a posteriori* Monte Carlo analysis was then conducted to test the obtained result.

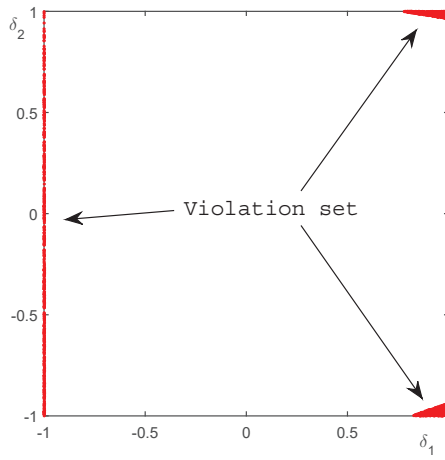


Figure 2.2. The violation set for the stabilization problem.

Figure 2.2 shows the portion of the square $[-1, 1] \times [-1, 1]$ where matrix

$$\begin{bmatrix} -Q^* & 0 \\ 0 & Q^*A^T(\delta) + (Y^*)^TB^T + A(\delta_i)Q^* + BY^* \end{bmatrix}$$

is not $\preceq \alpha^*I$ (violation set). The volume of the violation set was estimated to be 0.0096, below $\epsilon = 5\%$. ■

2.1.3 ■ Robust pole placement

We next consider a robust pole placement problem.

A continuous-time uncertain plant is described by the transfer function

$$G(s, \delta) = \frac{b(s, \delta)}{a(s, \delta)} = \frac{b_0(\delta) + b_1(\delta)s + \cdots + b_n(\delta)s^n}{a_0(\delta) + a_1(\delta)s + \cdots + a_n(\delta)s^n}.$$

Given the controller

$$C(s) = \frac{f(s)}{g(s)} = \frac{f_0 + f_1s + \cdots + f_{n-1}s^{n-1}}{g_0 + g_1s + \cdots + g_ns^n},$$

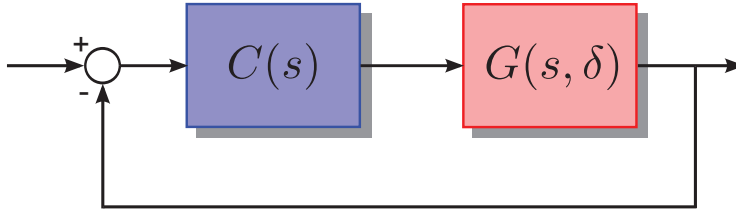


Figure 2.3. Feedback configuration for the pole placement problem.

in the standard feedback configuration of Figure 2.3, the goal is to select the parameters f_i and g_i of $C(s)$ so that the characteristic polynomial of the closed-loop system, whose zeros are the closed-loop poles, becomes similar to a given reference polynomial

$$r(s) = r_0 + r_1 s + \cdots + r_{2n} s^{2n}.$$

Now, notice that the characteristic polynomial of the closed-loop system is

$$p_{cl}(s, \delta) = p_{cl,0}(\delta) + p_{cl,1}(\delta)s + \cdots + p_{cl,2n}(\delta)s^{2n} = a(s, \delta)g(s) + b(s, \delta)f(s),$$

where the coefficients $p_{cl,j}(\delta)$ depend linearly on the controller parameters f_i and g_i . If the plant is not uncertain, i.e., its transfer function does not depend on δ , and the polynomials $b(s)$ and $a(s)$ are coprime, it follows that $p_{cl}(s)$ can be made exactly equal to $r(s)$.⁹ On the other hand, if the plant depends on δ , a perfect polynomial match becomes impossible, and one can set out to robustly minimize the distance between the coefficients of the two polynomials and write¹⁰

$$\min_{f_0, \dots, f_{n-1}, g_0, \dots, g_n} \left[\max_{\delta} \max_{j=0,1,\dots,2n} |p_{cl,j}(\delta) - r_j| \right]. \quad (2.10)$$

Since for any δ , $p_{cl,j}(\delta)$ depends linearly on the controller coefficients, the term $\max_{j=0,1,\dots,2n} |p_{cl,j}(\delta) - r_j|$ is the max of convex functions and is therefore convex; hence, the scenario approach can be readily applied to this problem.

2.1.4 ■ Other problems

In the previous sections, a few robust control problems that can be reformulated within the scenario setup have been presented. This is just a sample. Many other problems are amenable to a similar reformulation, including affine quadratic stability (AQS) [52] and the biquadratic stability condition of [89], robust H_2 synthesis (see, e.g., [5]), linear parameter-varying (LPV) design (see, e.g., [9]), the various robust design problems based on parameter-dependent linear matrix inequalities presented in [4], as well as problems in the robust model predictive control setup [72, 23, 79]. For discrete-time systems, the robust analysis and design criteria proposed in [40, 41] are also directly suitable for the scenario technique. In general, any robust control problem that is convex, or can be made convex, for a fixed plant (which is, for a fixed value of δ) lends itself to the scenario methodology. The reader is also referred to [20] for more discussion.

⁹This follows straightforwardly from the theory of resultants and in particular from the fact that the Sylvester matrix associated to polynomials $b(s)$ and $a(s)$ is full-rank when $b(s)$ and $a(s)$ are coprime. See [38].

¹⁰A better choice would be that of minimizing the distance between the zeros of the two polynomials; however, this approach leads to a nonconvex optimization problem.

2.2 ■ Disturbance compensation

We next consider an example where uncertainty is due to the presence of an external disturbance rather than to an uncertain dynamics. This example is taken from [30].

Consider a discrete-time linear system with scalar input u_t and scalar output y_t described by the equation

$$y_t = G(z)u_t + d_t,$$

where $G(z)$ is a known asymptotically stable, strictly proper transfer function and d_t is an additive stochastic disturbance. In this problem, d_t plays the role of the uncertain element, and according to the notation used in the scenario approach, it corresponds to δ . Our objective is to design a feedback controller

$$u_t = C(z)y_t$$

able to attenuate the effect of the disturbance, while also satisfying certain saturation constraints on the control action. Precisely, we consider the cost function

$$\sum_{t=1}^M y_t^2, \quad (2.11)$$

where y_t is the closed-loop output with zero initial conditions. The expression in (2.11) is the 2-norm of y_t over a finite, M -step horizon. Moreover, we enforce that u_t is within saturation limits:

$$|u_t| \leq u_{\text{sat}}, \quad t = 1, 2, \dots, M. \quad (2.12)$$

As for the controller $C(z)$, we refer to the following internal model control (IMC) structure (see [67]):

$$C(z) = \frac{Q(z)}{1 + G(z)Q(z)},$$

where $G(z)$ is the system transfer function, and $Q(z)$ is a free-to-choose stable transfer function. In Figure 2.4, one can easily see that the closed-loop transfer function

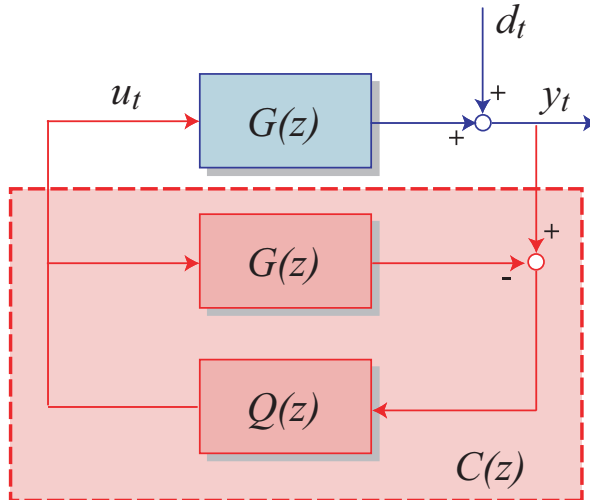


Figure 2.4. The IMC parameterization.

from d_t to u_t and that from d_t to y_t are, respectively, given by

$$Q(z), \quad 1 + G(z)Q(z). \quad (2.13)$$

Consequently, if $Q(z)$ is linearly parameterized, the cost (2.11) and the constraint (2.12) are convex in the parameters of $Q(z)$. For example, consider

$$Q(z) = q_0 + q_1 z^{-1} + q_2 z^{-2} + \cdots + q_k z^{-k}, \quad (2.14)$$

which is a finite impulse response (FIR) system, and let $q := [q_0 \ q_1 \ \dots \ q_k]^T \in \mathbb{R}^{k+1}$. Then, the input and the output of the controlled system can be written as

$$\begin{aligned} u_t &= q^T \phi_t \\ y_t &= q^T \psi_t + d_t, \end{aligned}$$

where ϕ_t and ψ_t are vectors containing delayed and filtered versions of the disturbance d_t :

$$\phi_t = \begin{bmatrix} d_t \\ d_{t-1} \\ \vdots \\ d_{t-k} \end{bmatrix} \quad \text{and} \quad \psi_t = \begin{bmatrix} G(z)d_t \\ G(z)d_{t-1} \\ \vdots \\ G(z)d_{t-k} \end{bmatrix}. \quad (2.15)$$

Hence, substituting in (2.11) yields $\sum_{t=1}^M y_t^2 = q^T A q + B q + C$, where

$$A = \sum_{t=1}^M \psi_t \psi_t^T, \quad B = 2 \sum_{t=1}^M d_t \psi_t^T, \quad C = \sum_{t=1}^M d_t^2, \quad (2.16)$$

which is a quadratic function of q , while (2.12) becomes $|\phi_t^T q| \leq u_{\text{sat}}$, $t = 1, 2, \dots, M$, which is also convex in q .

Suppose now that N disturbance realizations $d_{t,1}, \dots, d_{t,N}$ are available. Then, ϕ_t and ψ_t in (2.15) can be computed for each disturbance realization, and the scenario program in epigraphic form becomes

$$\begin{aligned} & \min_{q, h \in \mathbb{R}^{k+2}} h \\ & \text{subject to } q^T A_i q + B_i q + C_i \leq h, \quad i = 1, \dots, N, \\ & |\phi_{t,i}^T q| \leq u_{\text{sat}}, \quad t = 1, 2, \dots, M, \quad i = 1, \dots, N, \end{aligned} \quad (2.17)$$

where $\phi_{t,i}$ and A_i , B_i , and C_i are as in (2.15) and (2.16) with $d_{t,i}$ in place of d_t .

We next present some numerical results. Take $G(z) = \frac{1}{z-0.8}$, and suppose that the d_t is a sinusoidal signal with fixed frequency $\frac{\pi}{8}$ and random phase and amplitude

$$d_t = \alpha_1 \sin\left(\frac{\pi}{8} t\right) + \alpha_2 \cos\left(\frac{\pi}{8} t\right),$$

where α_1 and α_2 are independent random variables uniformly distributed in $[-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}]$. $Q(z)$ is just $Q(z) = q_0 + q_1 z^{-1}$, $M = 300$, and three different values of the saturation limit u_{sat} , i.e., 2, 0.8, and 0.2, are considered.

In the scenario approach, we set $\epsilon = 5\%$ and $\beta = 10^{-10}$ so that N given by Theorem 1.3 is $N = 1001$. Let (q^*, h^*) be the corresponding solution to (2.17) with $N = 1001$. Then, with probability no smaller than $1 - 10^{-10}$, the designed controller

with parameter q^* guarantees the upper bound h^* on the output 2-norm $\sum_{t=1}^{300} y_t^2$ over all disturbance realizations, except for a fraction of them whose probability is no bigger than 0.05. At the same time, the control input u_t is guaranteed not to exceed the saturation limit u_{sat} except for the same fraction of disturbance realizations. Figures 2.5, 2.6, and 2.7 represent the pole-zero maps and the Bode plots of the transfer function $1 + G(z)Q(z)$ between the disturbance d_t and the controlled output y_t (sensitivity function), for decreasing values of u_{sat} (2, 0.8, 0.2). As can be seen, when the saturation bound is large ($u_{\text{sat}} = 2$), the designed controller efficiently attenuates the sinusoidal disturbance at frequency $\frac{\pi}{8}$ by placing a pair of zeros approximately in $e^{\pm i\pi/8}$ in the sensitivity transfer function. As u_{sat} decreases, the control effort required to neutralize the sinusoidal disturbance exceeds the saturation limit, and a design with damped zeros is preferred. The optimal values h^* of the scenario program for $u_{\text{sat}} = 2, 0.8$, and 0.2 , is, respectively, 0.75, 3.61, and 90.94. As expected, h^* increases as u_{sat} decreases because the saturation constraint becomes more stringent. When the saturation bound is equal to 2, the scenario solution coincides with the solution that one would have naturally conceived without taking into account the saturation constraint; however, when the saturation constraint comes into play, the design becomes trickier, and the scenario approach is a viable method to find a solution.

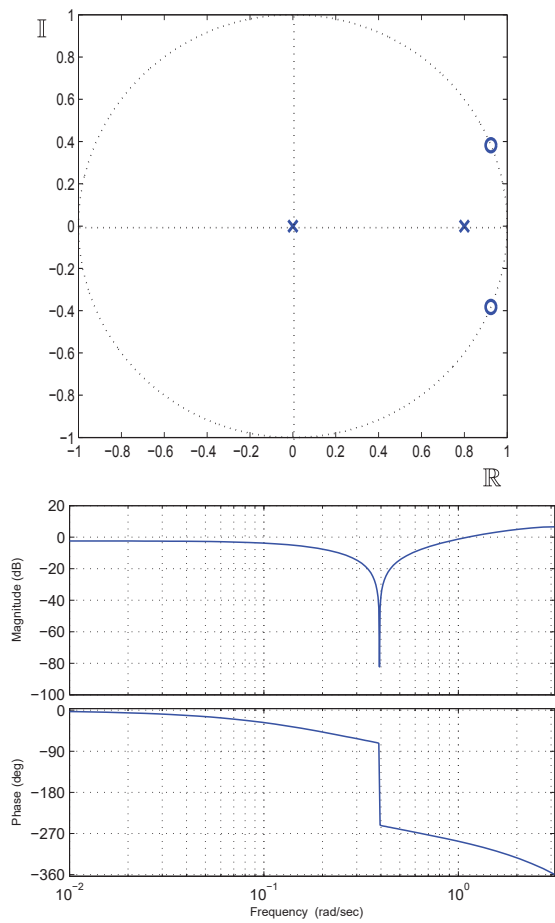


Figure 2.5. Pole-zero map and Bode plot of the sensitivity function for $u_{sat} = 2$.

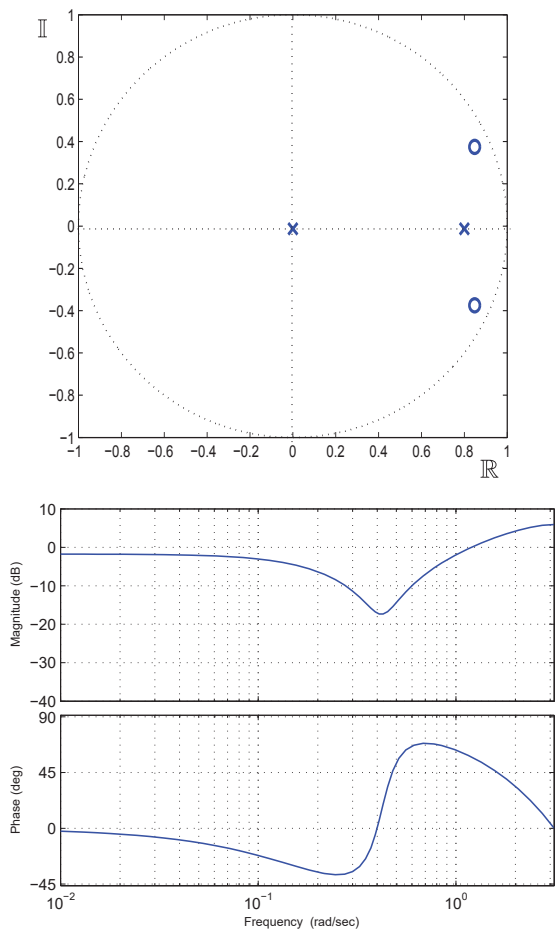


Figure 2.6. Pole-zero map and Bode plot of the sensitivity function for $u_{sat} = 0.8$.

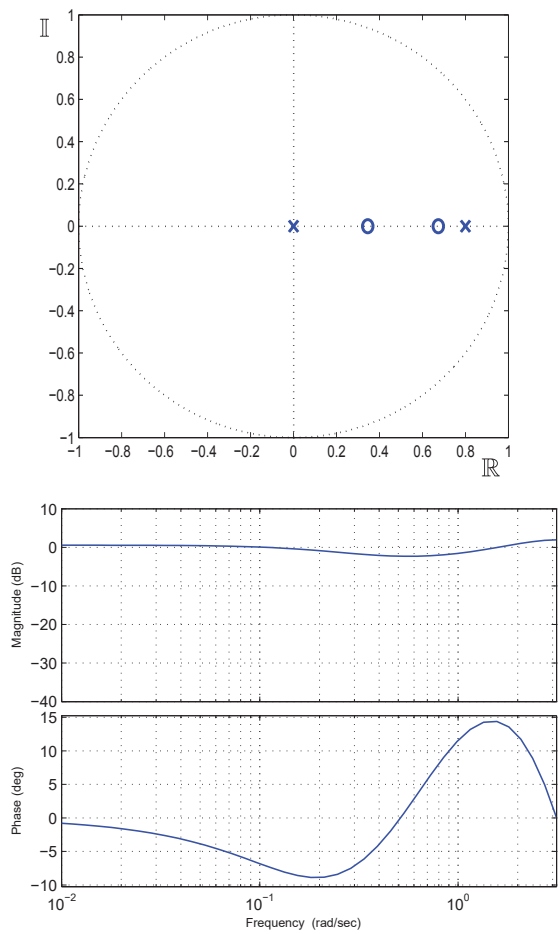


Figure 2.7. Pole-zero map and Bode plot of the sensitivity function for $u_{sat} = 0.2$.

Chapter 3

Theoretical results and their interpretation

In this chapter, some fundamental theorems of the scenario approach are presented. We start by introducing the mathematical setup of study and then state the main theorem that governs the generalization properties of the scenario approach, followed by some interpretations. The extension of the theorem to handle constraint discarding is dealt with in the second part of the chapter. The theorems are given without proofs, and the proofs can be found in Chapter 5. The results of Chapter 1 follow as corollaries of the theory presented in this chapter.

3.1 ■ Mathematical setup

We first remind the reader of the ingredients of the problem:

- (i) $c^T \theta$ = cost function;
- (ii) $\Theta \subseteq \mathbb{R}^d$ = admissible domain for θ ;
- (iii) $\{\Theta_\delta, \delta \in \Delta\}$ = family of constraints indexed by the uncertain parameter δ ;
- (iv) $\mathbb{P}$ = probability over Δ .

We advise the reader that measurability issues are glossed over throughout, and we implicitly assume that all involved events are measurable with respect to a suitable σ -algebra.

Definition 3.1 (violation set and violation probability). *The violation set of a given $\theta \in \Theta$ is the set $\{\delta \in \Delta : \theta \notin \Theta_\delta\}$. The violation probability (or just violation) of a given $\theta \in \Theta$ is the probability of the violation set of θ , that is, $V(\theta) := \mathbb{P}\{\delta \in \Delta : \theta \notin \Theta_\delta\}$.*

The violation set is a subset of the uncertainty domain Δ , and the violation is its probabilistic measure. Given a θ , the violation of θ quantifies the robustness level of θ against constraints $\Theta_\delta, \delta \in \Delta$. Note that the concept of violation does not involve optimization. The concept is illustrated in Examples 3.2 and 3.3.

Example 3.2. Suppose that $\theta \in \mathbb{R}^2$. The constraints are V-shaped as illustrated in Figure 3.1, and they are mathematically defined by the inequality $|\theta_1 - \delta| \leq \theta_2$, where

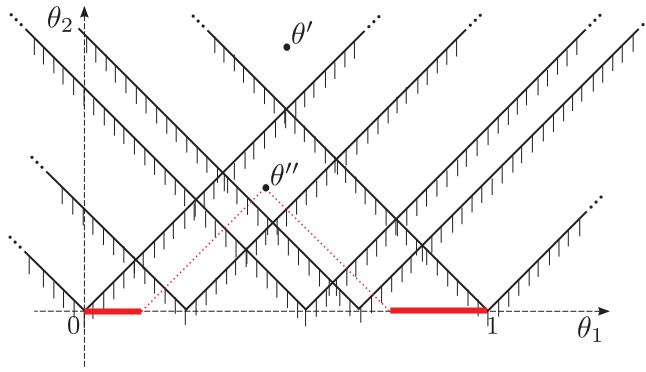


Figure 3.1. *V-shaped constraints. The red region is the set of vertices such that the corresponding V constraint is violated by θ'' ; the sum of the lengths of the two segments forming the red region is the violation probability $V(\theta'')$.*

δ is the vertex of the constraint set and takes value in $\Delta = [0, 1]$ with uniform probability. Consider the point θ' in the figure. θ' has zero violation because it is in the feasibility domain of all constraints. Instead, point θ'' violates the constraints whose vertices are in the red region. The violation of θ'' is the sum of the lengths of the two segments forming this region. ■

Example 3.3. Consider the feed-forward noise compensation example of Section 1.2.1. The optimization problem of minimizing $E[y_t^2]$ can be rewritten in epigraphic form as the minimization of ℓ with the constraint $\ell \geq E[y_t^2]$ (see Section 1.3). $E[y_t^2]$ is a function of $\nu = [\nu_1 \ \nu_2]^T$ and $\delta = [a \ b \ c \ d]^T$. Here, choosing a value for $\theta = [\nu^T \ell]^T$ corresponds to selecting a compensation unit and a value of the output variance, and $V(\theta)$ quantifies the probability of the set of ARMA systems such that the output variance with the selected compensation unit exceeds the selected value of ℓ . ■

The following convexity assumption on Θ and all Θ_δ , $\delta \in \Delta$, is in effect throughout this chapter.

Assumption 3.4 (convexity). Θ and Θ_δ , $\delta \in \Delta$, are convex and closed sets.

Sets Θ_δ can, e.g., be assigned via linear or quadratic inequalities or by linear matrix inequalities.

Given a sample $(\delta_1, \delta_2, \dots, \delta_N)$ of independent random elements from $(\Delta, \mathbb{P})$, a scenario optimization program is written as

$$\begin{aligned} \text{SP}_N : \min_{\theta \in \Theta} c^T \theta \\ \text{subject to } \theta \in \bigcap_{i=1, \dots, N} \Theta_{\delta_i}. \end{aligned} \quad (3.1)$$

Letting θ^* be the solution of (3.1), the fundamental problem we wish to address is the quantification of $V(\theta^*)$. Before stating the main theorem, an example helps to better clarify the nature of the problem we are about to study.

Example 3.5 (smallest interval that contains a sample of points). N points p_i are independently drawn from $[0, 1]$ according to a uniform probability, and the smallest interval that contains all the points is considered; see Figure 3.2.



Figure 3.2. Points in $[0, 1]$. The smallest interval that contains all the points is marked in blue.

To formalize the problem as an optimization program, notice that an interval can be parameterized by its center θ_1 and its semiwidth θ_2 , the distance from the center to the extremes. Then, the smallest interval is obtained by solving the program

$$\begin{aligned} \min_{\theta_1, \theta_2 \in \mathbb{R}^2} \quad & \theta_2 \\ \text{subject to} \quad & |\theta_1 - p_i| \leq \theta_2, \quad i = 1, \dots, N, \end{aligned} \quad (3.2)$$

where $|\theta_1 - p_i| \leq \theta_2$ says that the distance from the center of the interval to the i th point is no more than the semiwidth of the interval, expressing the fact that the point is inside the interval. This is a scenario program of the type (3.1) with $\delta_i = p_i$. Notice also that the constraints of this problem are the V-shaped constraints of Example 3.2.

We want to quantify the probability that one more point uniformly sampled from $[0, 1]$ falls outside the interval, and this probability is the length of the complement of the blue interval in Figure 3.2. In view of (3.2), this probability corresponds to $V(\theta^*)$, the violation of the scenario solution. Since the interval depends on the sample of N points, so does the length of its complement. Thus, what we want to describe is a random variable, and we aim to study the probability distribution of this random variable.

For $N = 2$, computing the distribution of $V(\theta^*)$ is an instructive exercise that can be solved by elementary computations.

Consider the square $[0, 1]^2$, where the first coordinate is p_1 and the second is p_2 . A point in this square corresponds to a sample of $N = 2$ points from $[0, 1]$. Take a point with coordinates $p_1 \leq p_2$ like the blue point in Figure 3.3. The corresponding solution to (3.2) is $\theta_1^* = (p_2 + p_1)/2$, $\theta_2^* = (p_2 - p_1)/2$, and the violation is the sum of the lengths of the two segments $[0, p_1]$ and $[p_2, 1]$, that is, $V(\theta^*) = 1 - (p_2 - p_1)$. The points in $[0, 1]^2$ that attain the same violation as the blue point are those on the green 45-degree line segments in the figure. In fact, the points on the green segment containing the blue point correspond to translating p_1 and p_2 while keeping their relative distance unchanged; the second green line segment accounts instead for the case $p_1 \geq p_2$. The points in the shaded region are instead those whose violation is less than or equal to that of the blue point. Thus, letting ϵ be the violation of the blue point (so that the two shaded triangles have short sides of length ϵ), we have that $V(\theta^*) \leq \epsilon$ with probability $2 \cdot (\epsilon^2/2) = \epsilon^2$. Now viewing ϵ as a variable in $[0, 1]$, we have reached the conclusion that $V(\theta^*)$ is a random variable with cumulative distribution ϵ^2 (and hence density 2ϵ) as depicted in Figure 3.4. ■

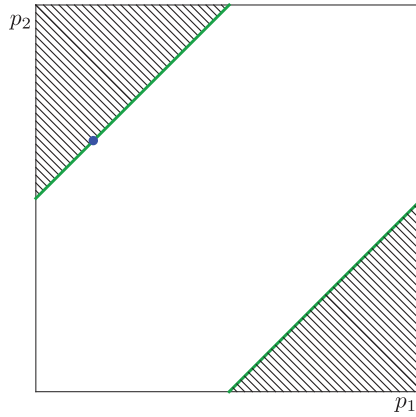


Figure 3.3. A point in $[0, 1]^2$ corresponds to a sample of $N = 2$ points from $[0, 1]$. The shaded area corresponds to points that generate a solution with violation less than or equal to that of the solution generated by the blue point.

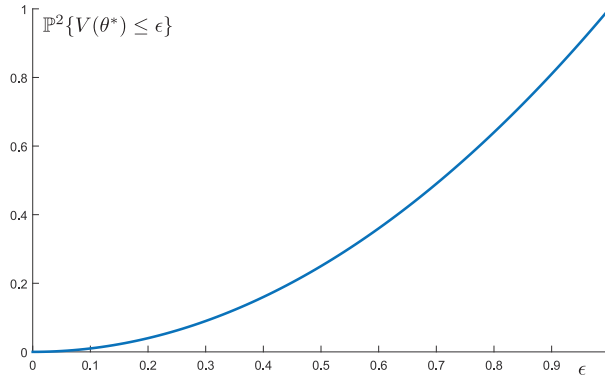


Figure 3.4. The cumulative distribution of the violation for the smallest interval construction of Example 3.5.

The previous example has pointed out a fundamental fact: θ^* depends on $\delta_1, \delta_2, \dots, \delta_N$ and it is therefore a random vector defined over $(\Delta^N, \mathbb{P}^N)$, where the probability $\mathbb{P}^N$ is a product probability since the δ_i 's are independent. When θ^* is substituted for θ in $V(\theta)$, one obtains $V(\theta^*)$. $V(\theta^*)$ is therefore the composition of a random vector and a deterministic function and is itself a random variable defined over $(\Delta^N, \mathbb{P}^N)$. The random variable $V(\theta^*)$ can be suitably described by means of its cumulative distribution.

One natural question to ask is: What do we expect the cumulative distribution of $V(\theta^*)$ to depend on? Referring back to the example of the smallest interval, if the number N of points is increased, on average the violation will be reduced. So, it appears that in a given problem, the cumulative distribution of $V(\theta^*)$ depends on N . How does this distribution depend on the problem? To develop an intuition, let us consider a problem similar to that of the smallest interval where the points are sampled from $\mathbb{R}^2$. In Figure 3.5, on the left, the disk of smallest volume that contains all points is visualized, followed by the smallest ellipsoid and by a third construction that allows for star-shaped regions (these are regions that contain one

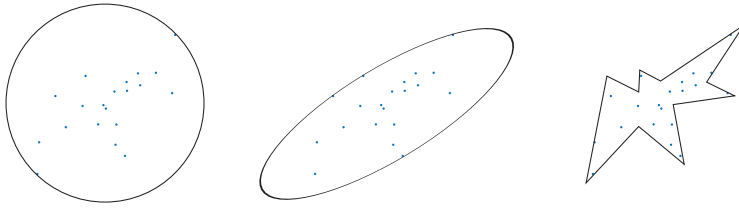


Figure 3.5. Points in $\mathbb{R}^2$ can be contained in domains of various shapes. The figure illustrates constructions at increasing levels of complexity.

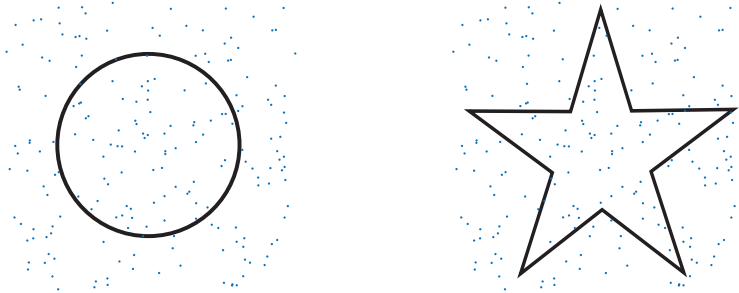


Figure 3.6. Two regions in $\mathbb{R}^2$. Suppose that both regions contain a probabilistic mass of 0.5. A Monte Carlo method for evaluating the probabilistic mass has the same probabilistic properties in the two cases, and the shape of the region is irrelevant.

point that can be connected by a segment to any other point in the region without crossing the region's boundary). Since a disk is an ellipsoid, and an ellipsoid is a star-shaped region, the three constructions consider classes of sets of increasing complexity. What do we expect? As we consider classes of increasing complexity we allow for more degrees of freedom. With more degrees of freedom, the region can be wrapped up around the points more tightly, and the probabilistic mass that is left outside the region is expected to increase. In short, when a solution taken from a complex class well adapts to the seen points, it is not expected to generalize as well.

One fact that is key in the above discussion is that an optimization procedure involves a selection from a class (in the example, the class of disks, the class of ellipsoids, and the class of star-shaped regions). Complexity refers to the class, not to a single region in the class. To better understand this point, consider for example the two regions in Figure 3.6. They have different shapes. Suppose nevertheless that they contain the same probabilistic mass, say 0.5. N points are independently sampled, and the probabilistic mass is estimated as the ratio between the number of points in the region and N (Monte Carlo estimation). In this problem, no optimization is performed, and the difference between the empirical estimates and the real value 0.5 follows the same probabilistic law in the two cases. The shape has no role.

Before stating the main theorem, an additional assumption of existence and uniqueness of the solution is introduced. We need a premise.

In the implementation of the scenario program SP_N , N is the actual number of scenarios that are used. It turns out that studying the properties of the solution

θ^* obtained from the program SP_N that has N scenarios calls for the consideration of programs with the same structure as SP_N but with a set of constraints whose size ranges over all integers. Accordingly, we introduce a running integer m that takes value $0, 1, \dots$, and consider programs similar to (3.1) which, however, have m constraints as follows:

$$\begin{aligned} \min_{\theta \in \Theta} c^T \theta \\ \text{subject to } \theta \in \bigcap_{i=1, \dots, m} \Theta_{\delta_i}. \end{aligned} \quad (3.3)$$

Similarly to (3.1), in (3.3) $(\delta_1, \delta_2, \dots, \delta_m)$, is a sample of i.i.d. (independent and identically distributed) elements from $(\Delta, \mathbb{P})$.

Assumption 3.6 (existence and uniqueness). *For every m and for every sample $(\delta_1, \delta_2, \dots, \delta_m)$, the solution of program (3.1) exists and is unique.*

Assumption 3.6 requires that (3.3) be feasible. Despite feasibility, (3.3) may miss the opportunity to obtain a solution when the cost value keeps improving as θ drifts away in some direction (see Figure 3.7) and the assumption rules out this possibility. One way to secure that the drifting to infinity does not occur, and to thereby make the existence part of the assumption verified, is to ensure that optimization is confined to a compact set Θ . As for the uniqueness, when the initial formulation of the problem does not automatically guarantee that the solution is unique, uniqueness can still be obtained by introducing suitable tie-break rules as discussed in Section 2.1 of [27], and the results of this chapter maintain their validity.

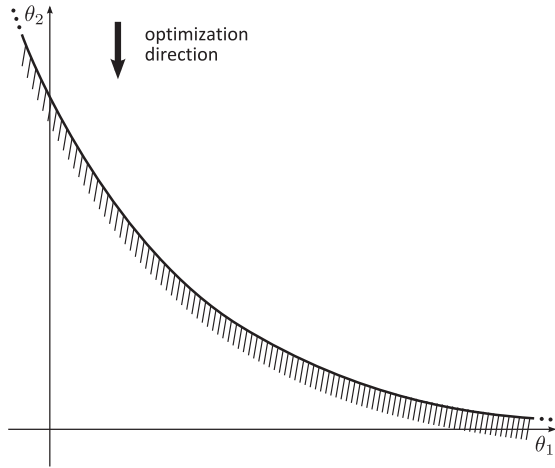


Figure 3.7. A situation where the cost value keeps improving as θ drifts away to the right.

3.2 ■ The generalization theorem of the scenario approach

The main theorem is stated first, followed by various comments. The proof of the theorem is given in Section 5.2.

Theorem 3.7 (see [27]). *Let $N \geq d$. Under Assumptions 3.4 and 3.6, it holds that*

$$\mathbb{P}^N\{V(\theta^*) > \epsilon\} \leq \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}. \quad (3.4)$$

The following comments are in order.

- (i) The right-hand side of (3.4) depends on N , as was expected. Instead, the problem at hand only features through d , the number of optimization variables.
- (ii) In view of Theorem 3.7, the cumulative distribution of $V(\theta^*)$ can be bounded as follows:

$$F_V(\epsilon) := \mathbb{P}^N\{V(\theta^*) \leq \epsilon\} \geq 1 - \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}. \quad (3.5)$$

The right-hand side of (3.5) is the cumulative distribution function of a so-called beta distribution with d and $N-d+1$ degrees of freedom, $B(d, N-d+1)$. Its density is

$$f_B(\epsilon) = d \binom{N}{d} \epsilon^{d-1} (1-\epsilon)^{N-d}, \quad (3.6)$$

as it can be straightforwardly verified by differentiating the right-hand side of (3.5). Figure 3.8 plots $f_B(\epsilon)$ for $d = 2$ and for various values of N . For smaller values of N , the density spreads to the right, while it concentrates towards zero as $N \rightarrow \infty$.

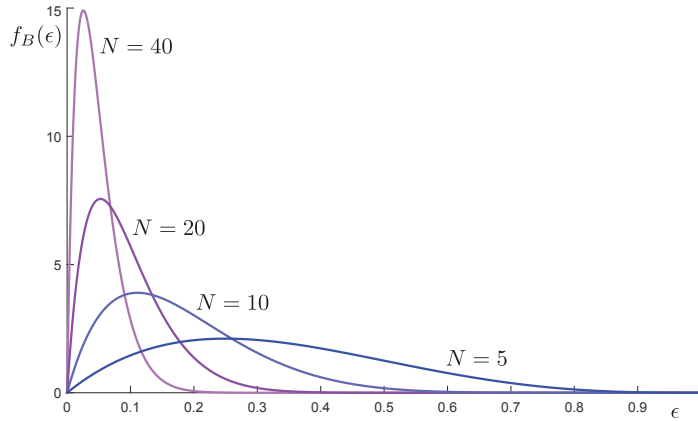


Figure 3.8. Density $f_B(\epsilon)$ for $d = 2$ and various values of N . As N is increased, the density concentrates towards zero.

- (iii) For $N = 40$, Figure 3.9 further shows how the density changes with d . As d is increased, the density spreads and moves towards the right, signifying that the violation tends to be larger. This is in line with the discussion in Section 3.1.

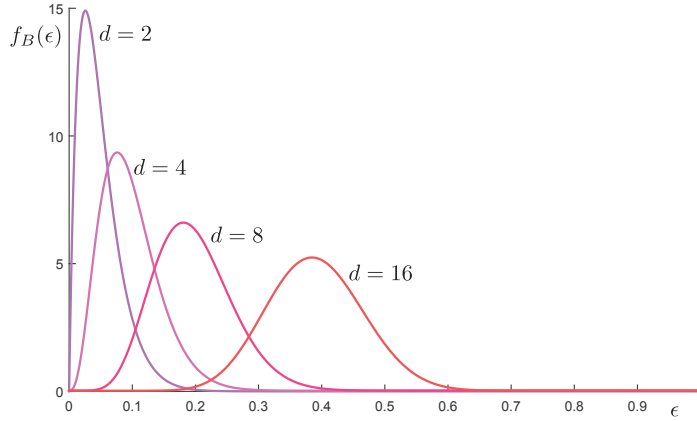


Figure 3.9. Density $f_B(\epsilon)$ for $N = 40$ and various values of d . As d is increased, the density spreads and moves towards the right.

- (iv) Take $d = 2$. If $N = 2$, (3.6) gives $f_B(\epsilon) = 2\epsilon$. This is the density that we have obtained in Example 3.5 with two points, which shows that for $d = 2$ and $N = 2$, the result is tight and cannot be improved. Provably, for any N , Figure 3.8 plots the exact density of the violation for the problem of the smallest interval in Example 3.5, and the result is therefore tight for $d = 2$ and any N . It is a fact that, for any d , problems exist where the violation density coincides with the density in (3.6). One such problem is the construction of Chebyshev layers, discussed in Chapter 6, whenever the points are sampled from a distribution that admits density. Thus, the result in (3.6) is not improvable for any d and N .¹¹
- (v) The fact that the result in (3.6) is not improvable does not mean that (3.6) holds with equality for any problem. Instead it means that equality holds for some problems. Hence, one can still hope to obtain better results for a given problem by adapting the evaluation of the violation to quantities that are observed at the time a specific scenario problem is solved. This interesting approach is discussed in Section 8.4.

¹¹One aspect that is key in obtaining results as tight as those in Theorem 3.7 is that the scenario theory aims to directly evaluate the violation of the solution θ^* , without any diversion towards uniform results. Elaborating on this concept, one might consider using the Vapnik–Chervonenkis theory [93, 95, 94], to obtain a result similar to that of Theorem 3.7 as follows. For any θ , one considers the set $\{\delta \in \Delta : \theta \notin \Theta_\delta\}$. As θ is varied, one obtains a collection $\mathcal{C}$ of subsets of Δ . $\mathcal{C}$ has a Vapnik–Chervonenkis dimension $VC(\mathcal{C})$ (the value of $VC(\mathcal{C})$ can be difficult to compute). If $VC(\mathcal{C}) < \infty$, then it holds that $\mathbb{P}\{\delta \in \Delta : \theta \notin \Theta_\delta\}$ can be evaluated simultaneously for any θ from the measurements $\delta_1, \delta_2, \dots, \delta_N$. Precisely, given an $\epsilon > 0$, relation $(\mathbf{1}_A$ is the indicator function of set A , i.e., $\mathbf{1}_A = 1$ over A and $\mathbf{1}_A = 0$ otherwise)

$$\sup_{\theta} \left| \mathbb{P}\{\delta \in \Delta : \theta \notin \Theta_\delta\} - \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\delta_i : \theta \notin \Theta_{\delta_i}) \right| > \epsilon \quad (3.7)$$

holds with probability no larger than β , where β is a number that depends on $VC(\mathcal{C})$ and N and that tends to zero as $N \rightarrow \infty$ [94]. Since $\theta^* \in \Theta_{\delta_i}$ for any i , the term $\frac{1}{N} \sum_{i=1}^N \mathbf{1}(\delta_i : \theta^* \notin \Theta_{\delta_i})$ turns out to be zero, and from (3.7) one obtains that $\mathbb{P}\{\delta \in \Delta : \theta^* \notin \Theta_\delta\} \leq \epsilon$ with probability $\geq 1 - \beta$, which is a result of the same type as that in Theorem 3.7. A fundamental drawback with this approach is that, owing to the uniformity of the result in (3.7), the evaluation of the sample size N needed to secure given ϵ and β turns out to be very loose. Moreover, there are cases where $VC(\mathcal{C})$ is even ∞ , which gives $\beta = 1$ for any finite N .

- (vi) It is a very important fact for applications that the result in Theorem 3.7 does not depend on $\mathbb{P}$. In many real problems, $\mathbb{P}$ is not known. When the scenario approach is implemented, all one needs are samples of the uncertainty variable, and these are often collected from the real world as observations. To the scenario solution one attaches a certificate of quality that is obtained from Theorem 3.7. This does not require knowledge of the underlying scheme according to which observations are generated. Thus, the scenario approach is fully observation-based, both at an algorithmic level and at the level of probabilistic certificates. Still, we notice that a fundamental assumption exists: observations are independent of one another.
- (vii) The fact that the result does not depend on $\mathbb{P}$ is expressed by saying that it is *universal*. Despite its universality, the result provides practical evaluations in many applications.

3.2.1 ■ ϵ - β results: Rapprochement with the results in Chapter 1

Theorem 1.3 in Chapter 1 claims that if $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, then, with probability $\geq 1 - \beta$, it holds that $\mathbb{P}\{\delta \in \Delta : \theta^* \notin \Theta_\delta\} \leq \epsilon$. This result is now obtained as a corollary of Theorem 3.7.

In view of the definition of violation (Definition 3.1), relation $\mathbb{P}\{\delta \in \Delta : \theta^* \notin \Theta_\delta\} \leq \epsilon$ can be rewritten as $V(\theta^*) \leq \epsilon$. Theorem 3.7 states that $\mathbb{P}^N\{V(\theta^*) > \epsilon\} \leq \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}$, which is equivalent to $\mathbb{P}^N\{V(\theta^*) \leq \epsilon\} \geq 1 - \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}$. Thus, what we have to show is that $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$ suffices to have that $1 - \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i} \geq 1 - \beta$ or, equivalently, that

$$\sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i} \leq \beta. \quad (3.8)$$

Start by elaborating the left-hand side of (3.8) as follows:¹²

$$\begin{aligned} \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i} &= 2^{d-1} \sum_{i=0}^{d-1} \binom{N}{i} \frac{\epsilon^i}{2^{d-1}} (1-\epsilon)^{N-i} \\ &\leq 2^{d-1} \sum_{i=0}^{d-1} \binom{N}{i} \left(\frac{\epsilon}{2}\right)^i (1-\epsilon)^{N-i} \\ &\leq 2^{d-1} \sum_{i=0}^N \binom{N}{i} \left(\frac{\epsilon}{2}\right)^i (1-\epsilon)^{N-i} \\ &= 2^{d-1} \left(\frac{\epsilon}{2} + (1-\epsilon)\right)^N \\ &= 2^{d-1} \left(1 - \frac{\epsilon}{2}\right)^N \\ &\leq 2^{d-1} \exp\left(-\frac{\epsilon}{2}N\right), \end{aligned} \quad (3.9)$$

where the last inequality holds because $1 - \frac{\epsilon}{2} \leq \exp(-\frac{\epsilon}{2})$ since $\exp(-x)$ is a convex function whose graph is above its tangent in $x = 0$, which is $1 - x$. In view of (3.9),

¹²The result can be obtained in various ways; the derivation below has been first suggested in [2].

(3.8) holds true provided that

$$2^{d-1} \exp\left(-\frac{\epsilon}{2}N\right) \leq \beta.$$

Applying natural logarithm to both sides, we obtain

$$(d-1)\ln 2 - \frac{\epsilon}{2}N \leq \ln \beta, \quad (3.10)$$

which is the same as

$$N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + (d-1)\ln 2 \right).$$

Thus, $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + (d-1)\ln 2 \right)$ suffices for (3.8) to hold. Since $\ln 2 < 1$, $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$ also suffices, and this establishes the validity of Theorem 1.3.

In closing this section, we note that while Theorem 1.3 gives a handy and explicit formula for N , directly solving (3.8) for N leads to values of N that typically improve by a factor of 2 or so over the explicit evaluation $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$. Bisection can be used for the purpose of solving (3.8). A MATLAB code to compute N is given below; the code can be cut and pasted into the MATLAB workspace for easy implementation.

```
function N = findN(eps,bet,d)

N1 = d;
N2 = ceil(2/eps*(d-1+log(1/bet)));

while N2-N1>1
    N = floor((N1+N2)/2);
    if betainc(1-eps,N-d+1,d) > bet
        N1=N;
    else
        N2=N;
    end
end

N = N2;
end
```

3.2.2 ■ The expected value $\mathbb{E}[V(\theta^*)]$

Theorem 3.7 asserts that the cumulative distribution of $V(\theta^*)$ is dominated by a beta distribution $B(d, N - d + 1)$. It is easy to see that this dominance translates into a dominance of the expected value so that the expected value of $V(\theta^*)$ is no more than the expected value of the $B(d, N - d + 1)$ distribution. The latter is found in any textbook of statistics, and it is equal to $d/(N + 1)$. We have thus obtained the following result.

Theorem 3.8. *Under Assumptions 3.4 and 3.6, it holds that*

$$\mathbb{E}[V(\theta^*)] \leq \frac{d}{N+1}. \quad (3.11)$$

The interpretation of $\mathbb{E}[V(\theta^*)]$ can be clarified by writing this expectation more explicitly. This can be done in two different ways.

The first way consists of using the density $f_V(\epsilon)$ of the random variable $V(\theta^*)$ (assuming it exists) and writing

$$\mathbb{E}[V(\theta^*)] = \int_0^1 \epsilon f_V(\epsilon) d\epsilon. \quad (3.12)$$

This expression admits an easy, though not quite rigorous, interpretation. $V(\theta^*)$ is the probability that θ^* violates a new instance of δ . Under the integral sign, ϵ is a variable that runs from 0 to 1, i.e., it runs over all the values that $V(\theta^*)$ can possibly assume. ϵ is multiplied by $f_V(\epsilon)$, which quantifies the probability that $V(\theta^*) = \epsilon$. Thus, $\epsilon f_V(\epsilon)$ is interpreted as the probability that θ^* violates one more instance of δ conditional to $V(\theta^*) = \epsilon$ multiplied by the probability that $V(\theta^*) = \epsilon$. Integrating over all possible values for ϵ , one obtains the total probability that a sample $(\delta_1, \delta_2, \dots, \delta_N)$ is seen, and then the corresponding solution θ^* violates another instance of δ .

A second, more rigorous, way to recover the same interpretation for $\mathbb{E}[V(\theta^*)]$ is obtained by recalling the general fact in probability theory that the expectation of a random variable ξ is simply the integral of ξ over the space of probabilistic outcomes Ω with respect to the probability measure $\mathbb{P}$, that is, $\mathbb{E}[\xi] = \int_{\Omega} \xi(\omega) \mathbb{P}(d\omega)$. In fact, the more commonly used expression $\mathbb{E}[\xi] = \int_{\mathbb{R}} x f(x) dx$, where $f(x)$ is the probability density of ξ , is obtained by a change of variables from the domain Ω to the domain $\mathbb{R}$ where ξ takes value. In our case, the expectation $\mathbb{E}[V(\theta^*)]$ can be written as

$$\begin{aligned} \mathbb{E}[V(\theta^*)] &= \int_{\Delta^N} V(\theta^*) \mathbb{P}^N(d\delta_1, \dots, d\delta_N) \\ &= \int_{\Delta^N} \mathbb{P}\{\delta \in \Delta : \theta^* \notin \Theta_{\delta}\} \mathbb{P}^N(d\delta_1, \dots, d\delta_N) \\ &= [\mathbf{1} = \text{indicator function}] \\ &= \int_{\Delta^N} \left[\int_{\Delta} \mathbf{1}\{\delta \in \Delta : \theta^* \notin \Theta_{\delta}\} \mathbb{P}(d\delta) \right] \mathbb{P}^N(d\delta_1, \dots, d\delta_N) \\ &= \int_{\Delta^N \times \Delta} \mathbf{1}\{\delta \in \Delta : \theta^* \notin \Theta_{\delta}\} \mathbb{P}^{N+1}(d\delta, d\delta_1, \dots, d\delta_{N+1}) \\ &= \mathbb{P}^{N+1}\{\delta \in \Delta : \theta^*(\delta_1, \dots, \delta_N) \notin \Theta_{\delta}\}. \end{aligned}$$

This latter expression says that $\mathbb{E}[V(\theta^*)]$ is the total probability that the θ^* obtained from a sample $(\delta_1, \delta_2, \dots, \delta_N)$ violates another instance of δ .

The result in Theorem 3.8 does have practical importance, particularly in applications involving sequential data acquisition. Suppose that data are collected in succession in time. A design is made with the scenario approach by using the last N data points, and the design is updated any time one more data point comes in by adding this data point to the scenario program, while also discarding the oldest data point. In other words, the design is based on a window of N data points that shifts along the time axis. After each design, one waits for the next observation δ and adds one to a counter if the corresponding constraint Θ_{δ} is violated by the current design θ^* . Then, $\mathbb{E}[V(\theta^*)]$ is the limit of the empirical frequency with which the constraint

posed by the new observation is violated by the current design,¹³ and knowing a bound for $\mathbb{E}[V(\theta^*)]$ beforehand provides information on this limit. Applications of this setup are found in sequential investments, prediction, etc.

We close this section by noting that, similarly to the expected value, higher moments $\mathbb{E}[V(\theta^*)^2]$, $\mathbb{E}[V(\theta^*)^3]$, $\dots$, can be interpreted as the probability that the scenario solution θ^* is constructed from $\delta_1, \delta_2, \dots, \delta_N$ and then two new independent instances of the uncertain parameter δ are violated, or three new independent instances of the uncertain parameter δ are violated, and so on. The knowledge of the cumulative distribution function of $V(\theta^*)$ offered by Theorem 3.7 allows one to also set bounds on these probabilities.

3.3 ■ Scenario optimization with constraint removal

After sampling N constraints, one can consider removing some of them to improve the solution, i.e., to obtain a solution with lower cost value. This can be done in various ways. Constraints can be removed optimally, that is, one removes those constraints whose elimination produces the largest possible improvement as compared to the improvement obtained by the elimination of any other group of constraints of the same size. This approach corresponds to solving a chance-constrained problem with a discrete (concentrated over the available constraints) probability distribution, and it has been intensively studied in the literature; see, e.g., [43, 44, 77, 13, 65]. Nonetheless, optimal constraint removal can be computationally very demanding and becomes rapidly prohibitive as the number of constraints to remove grows. Hence, one normally opts for some heuristics to remove constraints. These range from greedy procedures where constraints are removed one after the other by selecting each time that constraint whose elimination gives the largest one-step improvement, to purely random methods where a constraint is removed at each step by random selection from the currently active constraints. Figure 3.10 shows an example where two constraints are eliminated according to the optimal strategy and the greedy one. Importantly, the generalization Theorem 3.9 below applies irrespective of the strategy according to which scenarios are removed. Thus, choosing a better strategy to remove constraints leads to a lower value of the solution, while the violation guarantee remains the same.

In what follows, θ_k^* is any solution that violates k constraints. Saying that k constraints are violated is not the same as saying that k constraints have been removed. For example, in a random procedure, a constraint that has been removed at some step may be satisfied again at a later stage of the procedure; see Figure 3.11 for an example. The same may happen for a greedy procedure. Since Theorem 3.9 is applicable to solutions θ_k^* that violate k constraints, to apply the theorem, if some constraints that have been removed are not violated after the elimination of k constraints, these nonviolated constraints are reinstated and one proceeds to

¹³To see that this is the case, consider a different scheme in which one uses N data points to make the design and adds one to a counter if the constraint posed by the new observation is violated, and then throws away all the $N + 1$ δ 's, and repeats the same procedure with the next $N + 1$ observations, with no overlap of data points. Each set of $N + 1$ observations is independent of any other set, so that an application of the law of large numbers leads to the fact that $\mathbb{E}[V(\theta^*)] = \mathbb{P}^{N+1}\{\delta \in \Delta : \theta^*(\delta_1, \dots, \delta_N) \notin \Theta_\delta\}$ is the limit of the empirical frequency. Our case with a shifting window can be transformed into $N + 1$ sequences without overlap: (1st sequence) $[1, N + 1], [N + 2, 2N + 2], [2N + 3, 3N + 3], \dots$; (2nd sequence) $[2, N + 2], [N + 3, 2N + 3], [2N + 4, 3N + 4], \dots$; and so on. For each sequence the empirical frequency converges to $\mathbb{E}[V(\theta^*)]$, so that the overall empirical frequency, which is the average of the empirical averages, also tends to $\mathbb{E}[V(\theta^*)]$.

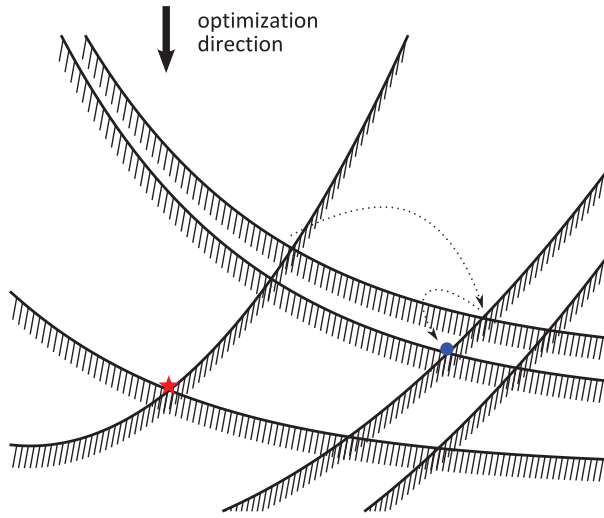


Figure 3.10. The scenario solution can be improved by constraint removal. The red solution (star) is the optimal solution after removing two constraints. If a greedy procedure is adopted, one first removes the constraint on the right, as its removal gives the largest one-step improvement. Then the constraint on the left is eliminated, leading to the blue solution (dot).

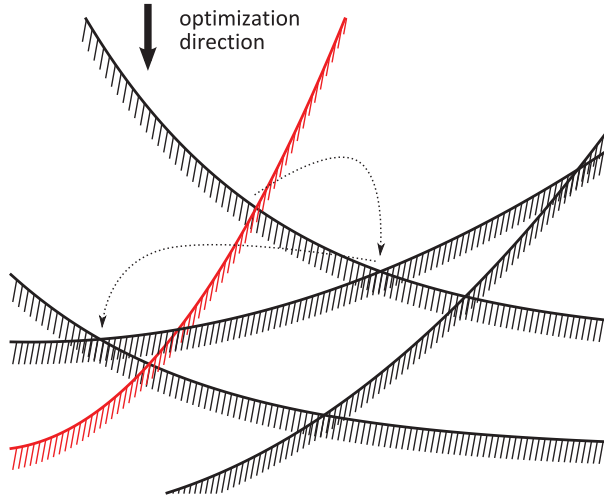


Figure 3.11. A previously eliminated constraint can be satisfied later during constraint removal. The constraint in red that is eliminated at the first step becomes satisfied at the second step when active constraints are randomly discarded in succession.

remove more constraints so that, at the end, exactly k constraints are violated. The so-obtained solution is the θ_k^* to which Theorem 3.9 is applied.

Theorem 3.9 (see [28]). Let $N \geq d$. Under Assumptions 3.4 and 3.6, it holds that

$$\mathbb{P}^N \{V(\theta_k^*) > \epsilon\} \leq \binom{k+d-1}{k} \sum_{i=0}^{k+d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}. \quad (3.13)$$

Theorem 3.9 extends Theorem 3.7. When $k = 0$, Theorem 3.9 coincides with Theorem 3.7.

A typical use of Theorem 3.9 refers to the situation when the number N of scenarios is assigned, for example, N is the number of available observations. After fixing a small value for β , e.g., $\beta = 10^{-7}$, and a value for ϵ , one computes the largest k such that the right-hand side of (3.13) is smaller than β . This is the largest number of constraints that one is allowed to violate while maintaining the desired guarantees.

In some problems, it may be desirable to inspect the performance against the violation for various values of k before deciding how many constraints one wants to violate. This approach is compatible with the theory of this section. To this end, one fixes an upper bound $\bar{k}$ to k and lets $k \in [1, \bar{k}]$. $\bar{k}$ can possibly be as large as all constraints, that is, $\bar{k} = N$. After selecting a value for β , one computes the values of ϵ such that the right-hand side of (3.13) equals $\beta/\bar{k}$ for $k = 1, \dots, \bar{k}$. Let these values be ϵ_k , $k = 1, \dots, \bar{k}$. It follows that each solution θ_k^* , $k = 1, \dots, \bar{k}$, does not violate more than ϵ_k with probability $\beta/\bar{k}$, so that all θ_k^* 's violate no more than ϵ_k simultaneously for all $k = 1, \dots, \bar{k}$ with probability $\bar{k} \cdot \beta/\bar{k} = \beta$. Hence, one can inspect the performance achieved for various k 's vs. the corresponding violation level ϵ_k and select a suitable trade-off. The result is guaranteed with confidence $1 - \beta$. Note that this is the approach that was used in the feed-forward noise compensation example in Section 1.2.5.

In closing this section, we feel it is advisable to add a comment relating to an issue that the reader may have noticed: In which sense is sampling N and then discarding k better than using directly fewer scenarios so that the same ϵ - β result holds without discarding any constraints? For one thing, requiring fewer scenarios as in the second approach comes with an advantage when scenarios are a costly resource. The answer to the question above is somehow subtle. Before reading the next example that provides an answer, the reader may want to spend some time reflecting upon this issue.

Example 3.10 (Example 3.5 continued). In the construction of the smallest interval of Example 3.5, consider first the case where the solution is computed from $N = 19$ scenarios with no constraint removal, and then the case where $N = 770$ scenarios are sampled and $k = 200$ of them are discarded according to a random procedure that sequentially removes active constraints. In Figure 3.12, the densities of the violation of the solution for the two cases are displayed. The solid blue curve is the $f_B(\epsilon)$ of (3.6), which, as already noted, is exact in the situation at hand. The dashed red curve, instead, is the density of the cumulative distribution function

$$1 - \sum_{i=0}^{k+d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}, \quad (3.14)$$

which can be proven to give the exact probability distribution of the solution with k removed constraints in this example. The tail of the two curves beyond value $\epsilon = 0.3$ has the same mass equal to 10^{-2} , that is, the probability that the violation is bigger than 0.3 is the same in the two cases.¹⁴ However, the two densities are very different: The red density is much more concentrated around the value 0.3. We interpret this fact to mean that the level of intrinsic randomness present in the problem is reduced by allowing for constraint removal. Hence, by constraint removal

¹⁴We have taken large values for β and ϵ to obtain graphs that do not concentrate near zero and are therefore visually inspectable.

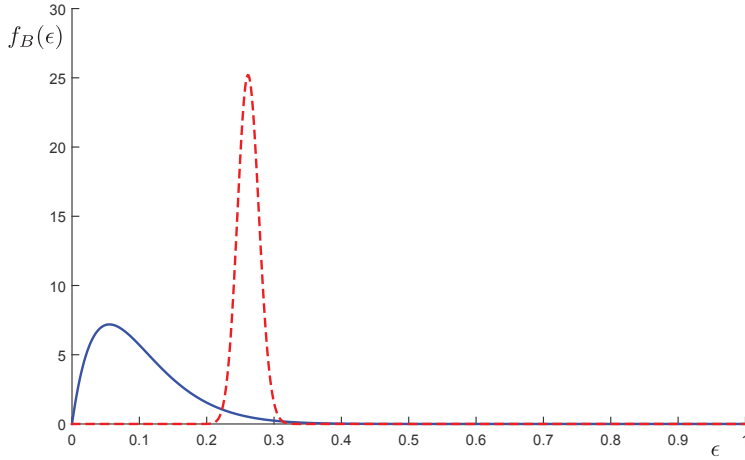


Figure 3.12. Density of violation for the scenario program with (dashed red curve) and without (solid blue curve) constraint removal.

the violation has a tendency to be close to the selected value 0.3, and thereby one obtains a better performance (i.e., a smaller interval) than without constraint removal. This fact has general validity, and similar observations can be applied to any optimization problem. ■

3.3.1 ■ Derivation of Theorem 1.2 as a Corollary of Theorem 3.7

We show that the ϵ_k given by formula (1.9) implies that the right-hand side of (3.13) with ϵ_k in place of ϵ is $\leq \beta$, so that Theorem 1.2 is obtained from Theorem 3.9.

Let $a = 1 + \frac{1}{\sqrt{k}}$. We have

$$\begin{aligned}
 \sum_{i=0}^{k+d-1} \binom{N}{i} \epsilon_k^i (1-\epsilon_k)^{N-i} &= a^{k+d-1} \sum_{i=0}^{k+d-1} \binom{N}{i} \frac{\epsilon_k^i}{a^{k+d-1}} (1-\epsilon_k)^{N-i} \\
 &\leq a^{k+d-1} \sum_{i=0}^{k+d-1} \binom{N}{i} \left(\frac{\epsilon_k}{a}\right)^i (1-\epsilon_k)^{N-i} \\
 &\leq a^{k+d-1} \sum_{i=0}^N \binom{N}{i} \left(\frac{\epsilon_k}{a}\right)^i (1-\epsilon_k)^{N-i} \\
 &= a^{k+d-1} \left(\frac{\epsilon_k}{a} + (1-\epsilon_k)\right)^N \\
 &= a^{k+d-1} \left(1 - \left(1 - \frac{1}{a}\right) \epsilon_k\right)^N. \tag{3.15}
 \end{aligned}$$

Since function $\exp(-x)$ is convex in x , it holds that $1-x \leq \exp(-x)$, where $1-x$ is the line tangent to the exponential function at $x=0$. Hence, with $x = \left(1 - \frac{1}{a}\right) \epsilon_k$, we obtain $\left(1 - \left(1 - \frac{1}{a}\right) \epsilon_k\right)^N \leq \exp\left(-\left(1 - \frac{1}{a}\right) \epsilon_k N\right)$ and, with $x = (1-a)$, we instead obtain $a^{k+d-1} = (1-(1-a))^{k+d-1} \leq \exp(-(1-a)(k+d-1))$, which, used in (3.15), gives

$$\sum_{i=0}^{k+d-1} \binom{N}{i} \epsilon_k^i (1-\epsilon_k)^{N-i} \leq \exp(-(1-a)(k+d-1)) \cdot \exp\left(-\left(1 - \frac{1}{a}\right) \epsilon_k N\right). \tag{3.16}$$

On the other hand, it holds that

$$\binom{k+d-1}{k} = \frac{(k+d-1)!}{k!(d-1)!} \leq (k+d-1)^{d-1}. \quad (3.17)$$

In view of (3.16) and (3.17), the right-hand side of (3.13) is $\leq \beta$ provided that

$$(k+d-1)^{d-1} \cdot \exp(-(1-a)(k+d-1)) \cdot \exp\left(-\left(1-\frac{1}{a}\right)\epsilon_k N\right) \leq \beta.$$

Applying the natural logarithm to both sides, and recalling that $a = 1 + \frac{1}{\sqrt{k}}$, we obtain

$$(d-1)\ln(k+d-1) + \frac{1}{\sqrt{k}}(k+d-1) - \frac{1}{\sqrt{k}+1}\epsilon_k N \leq \ln \beta,$$

which is the same as

$$\begin{aligned} \epsilon_k &\geq \frac{\sqrt{k}+1}{N} \left[(d-1)\ln(k+d-1) + \frac{1}{\sqrt{k}}(k+d-1) + \ln \frac{1}{\beta} \right] \\ &= \frac{k}{N} + \left[\frac{\sqrt{k}}{N} + \frac{\sqrt{k}+1}{N} \left((d-1)\ln(k+d-1) + \frac{d-1}{\sqrt{k}} + \ln \frac{1}{\beta} \right) \right], \end{aligned} \quad (3.18)$$

where the last expression is the right-hand side of (1.9). This concludes the derivation.

Chapter 4

Probabilistic vs. deterministic robustness

The result given in Theorem 3.7 in Chapter 3 is that $V(\theta^*) \leq \epsilon$ holds with high confidence $1 - \beta$. This result is of probabilistic nature. β is a probability with respect to $\mathbb{P}^N$. Moreover, $V(\theta^*)$ is itself a probability, $V(\theta^*) = \mathbb{P}\{\delta : \theta^* \notin \Theta_\delta\}$. Are we happy with dealing with uncertainty from a probabilistic perspective? This chapter deals with this question and attempts to compare probabilistic with deterministic robustness.

4.1 ■ Sure statements vs. statements with a probabilistic validity

Robustness in control often refers to properties on which one is not ready to accept a probabilistic compromise. For example, binding requirements relating to the stability of an aircraft or the operation of a nuclear plant are often regarded as hard constraints. In contrast, outside control, probabilistic arguments are largely accepted, and they are in fact mainstream in many fields such as quantitative finance and machine learning. In an investment, for example, “no-risk, no-gain” is commonplace. In between, there is a continuous scale of grey hues; see Figure 4.1. In telecommunications, for instance, one would prefer that the line never drops. However, lines in mobile communication do indeed drop, and this generally causes no drama.

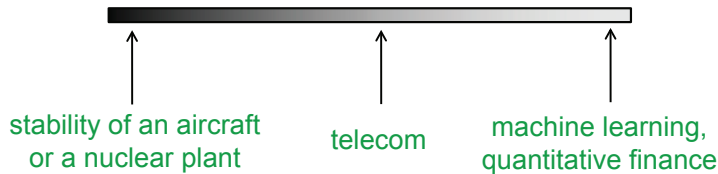


Figure 4.1. *The grey scale of risk attitude. The notion of probabilistic guarantee finds varying acceptance depending on the field.*

An issue that comes up naturally in relation to deterministic robustness is what a “sure” or “deterministic” statement means. Mathematically, the answer is clear: It is a statement that holds true for any outcome in the uncertainty set Δ . But then

the question arises as to whether Δ itself is a sure description of all the potential uncertainty outcomes. Here, the focus of attention shifts from the mathematical level to the level of the mismatch between model and reality. To quote from Mark Twain:

What gets us into trouble is not what we don't know. It's what we know for sure that just ain't so.

Certainly, the way out of the difficulty that Δ can underestimate the uncertainty set that is offered by considering a vast Δ is impractical because a robust design over a vast Δ normally suffers from severe conservatism.

Thus, in a given application, one of the following two situations may occur.

(i) *The Δ that is used to describe uncertainty is sure enough, and a robust design still performs satisfactorily.*

If this is so, a deterministically robust approach is perfectly sound and viable; or

(ii) *for a sure enough Δ , the performance is too poor to be acceptable.*

In this second case, the designer may be apt to fall into the subtle pitfall of resizing Δ so that the performance becomes satisfactory. However, this approach has two significant drawbacks: an obvious one and a not-so-obvious one. They are as follows.

(a) *First, since the result is robustly guaranteed for any $\delta \in \Delta$, one is led to believe that the result is sure, while it is not, because the resizing has made Δ itself uncertain. We are in the potentially most harmful situation, as Mark Twain's adage above teaches us.*

(b) *Second and more subtly, arbitrary resizing Δ usually has little pay-off. The reason is that resizing reduces uncertainty blindly, with no attention towards performance.*

Point (b) deserves to be better investigated, and we prefer to do so by means of an example.

Example 4.1 (lateral motion of an aircraft). This example was introduced in [92], and then revisited in [3, 71, 86]. The numerical values considered here are taken from [92].

The lateral motion of the aircraft can be described by the equation

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & L_p & L_\beta & L_r \\ 0.086 & 0 & -0.11 & -1 \\ 0.086N_\beta & N_p & N_\beta + 0.11N_\beta & N_r \end{bmatrix} x(t) + \begin{bmatrix} 0 & 0 \\ 0 & -3.91 \\ 0.035 & 0 \\ -2.53 & 0.31 \end{bmatrix} u(t), \quad (4.1)$$

where the four state variables are the bank angle, the derivative of the bank angle, the sideslip angle, and the yaw rate, and the two inputs are the rudder deflection and the aileron deflection. The various parameters that appear in the state matrix

are uncertain and take value in the following intervals:

$$\begin{aligned} L_p &\in [-2.93, -1] \\ L_\beta &\in [-73.14, -4.75] \\ L_r &\in [0.78, 3.18] \\ N_{\dot{\beta}} &\in [0, 0.1] \\ N_p &\in [-0.042, 0.086] \\ N_\beta &\in [2.59, 8.94] \\ N_r &\in [-0.39, -0.29]. \end{aligned}$$

In the following, we let $\delta := [L_p \ L_\beta \ L_r \ N_{\dot{\beta}} \ N_p \ N_\beta \ N_r]^T$, and Δ is the Cartesian product of the above intervals, which is a hyper-rectangle in a 7-dimensional space. For short, the state and input matrices in (4.1) will be written as $A(\delta)$ and B .

Given two symmetric positive definite matrices S and R and an initial condition x_0 , let $u(t) = Kx(t)$ (state feedback), and consider the cost function

$$\ell(\nu, \delta) = \int_0^\infty [x(t)^T S x(t) + u(t)^T R u(t)] dt,$$

where ν contains the entries of K . As an additional requirement, we also impose that the closed loop be quadratically stable. Following [70], the controller is designed as

$$K^* = X^*(Q^*)^{-1},$$

where X^* and Q^* are the solutions to the following convex program with LMIs:

$$\begin{aligned} &\min_{X, Q > 0} x_0^T Q^{-1} x_0 \\ &\text{subject to} \\ &\left[\begin{array}{cc|cc} -(QA(\delta)^T + A(\delta)Q + X^T B^T + BX) & X^T & Q \\ X & R^{-1} & 0 \\ Q & 0 & S^{-1} \end{array} \right] \succeq 0, \quad \forall \delta \in \Delta, \end{aligned} \quad (4.2)$$

$x_0 = x(0)$ being the initial state of (4.1).

The motivation for this choice is that for any Q and X satisfying the LMI in (4.2), the associated controller $K = XQ^{-1}$ quadratically stabilizes (4.1) and, moreover, $x_0^T Q^{-1} x_0$ is guaranteed to be an upper bound of $\ell(\nu, \delta)$ for all δ . The optimal value $\ell^* = x_0^T (Q^*)^{-1} x_0$ of (4.2) is thus a robustly guaranteed performance level for all $\delta \in \Delta$, a value that has been optimized in (4.2).

In problem (4.2), $A(\delta)$ depends on δ linearly. Since Δ is a hyper-rectangle, it is a known fact (see, e.g., [7, 17, 86]) that enforcing the $2^7 = 128$ LMIs corresponding to the vertices of Δ implies the satisfaction of the LMIs corresponding to all δ 's in Δ . Hence, problem (4.2) reduces to a convex problem with a finite number of constraints, which can be solved by standard solvers for semidefinite programming.

In our simulation, we set $S = 0.01I$, $R = I$, and $x_0 = [1 \ 1 \ 1 \ 1]^T$ and implemented (4.2) via the modeling language CVX [57, 56], with solver SDPT3 [88, 91]. The solution was

$$Q^* = \begin{bmatrix} 7.5501 & -7.7849 & 0.0316 & 0.4934 \\ -7.7849 & 20.0229 & 0.2768 & -1.8251 \\ 0.0316 & 0.2768 & 0.0833 & 0.1829 \\ 0.4934 & -1.8251 & 0.1829 & 1.3175 \end{bmatrix},$$

$$K^* = \begin{bmatrix} 0.6920 & 0.9276 & -14.7276 & 4.9913 \\ 0.4823 & 0.5023 & -3.5502 & 0.7729 \end{bmatrix},$$

and the optimal cost was

$$\ell^* = 16.8.$$

For this problem, however, this cost value is too large to be acceptable. Hence, one can be tempted to resize the Δ domain, and to solve (4.2) with Δ' in place of Δ , where Δ' is a “shrunk” version of Δ , where each side of the uncertainty hyper-rectangle is contracted so that the initial proportions are maintained. Letting

$$\text{vol}(\Delta')/\text{vol}(\Delta) = 1 - \epsilon,$$

the resizing procedure was repeated for various values of ϵ ranging from 0.005 to 0.025, and the dashed red line in Figure 4.2 depicts the obtained cost ℓ_{rs}^* (rs = resizing) against ϵ . One can see that resizing gives little benefit. The reason is that the resizing procedure is blind; no attention is paid to the final goal of reducing the cost.

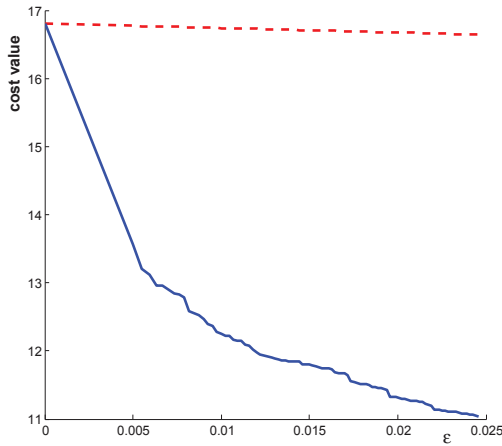


Figure 4.2. Cost vs. volume reduction. Dashed line: resizing. Solid line: scenario approach.

Figure 4.2 also shows in solid blue a curve that shows how much more improvement can be obtained by removing a suitable ϵ -portion of the uncertainty domain. How can such a curve be obtained? The curve was obtained by the scenario approach with constraint removal from Section 3.3 with a greedy removal scheme where $\mathbb{P}$ was uniform on Δ and β was set to the value 10^{-7} . As constraints are removed in the scenario program, the solution finds its way to improving the performance level. In the initial problem with uncertainty ranging in a hyper-rectangle, this corresponds to removing portions of the uncertainty domain that are relevant to reducing the cost, while the volume of the violated region is kept under control by the scenario theory.

In closing, we would like to note that for uncertainty taking place in a Euclidean space having more than just three or four dimensions, it is quite common that the portion of the uncertainty domain that must be removed to improve the solution is not the outer shell of the domain. Rather, it is an elongated region with little volume, a structure that has been named “icicle geometry” in [61]; see Figure 4.3. The reason

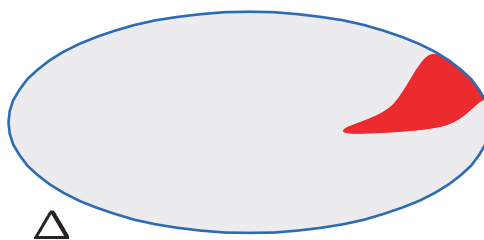


Figure 4.3. *The icicle geometry. Often, the region in the optimization domain that has to be removed to improve the cost function has an elongated shape with little volume, which resembles the form of an icicle.*

for this is that in high dimensions, icicle-shaped regions are the norm rather than the exception. For example, in a hyper-cube $[0, 1]^{10}$, the cube of half-length $[0, 0.5]^{10}$ occupies about 0.001 of the volume, but it stretches all the way from one corner to the center of the hyper-cube. ■

4.2 ■ Beyond the use of models

In deterministic robustness, uncertainty is modeled as a set with no additional structure. In contrast, the information that is included in a probabilistic model of uncertainty is way more complex and articulated because a probabilistic value has to be attached to each single event. A legitimate question to ask is where all this information comes from in a given application.

Interestingly, in the scenario approach, all one uses are scenarios. At times, these scenarios are drawn from a probabilistic model, a situation that gives rise to the subtle issue of controlling the mismatch between model and reality. On the other hand, when scenarios are observations, no knowledge of $\mathbb{P}$ is required for the scenario approach to be applied. In other words, a probability is assumed to exist, but no knowledge of this probability is needed, and the scenario approach directly connects knowledge that comes from observations to decisions. To quote from C. S. Lewis:

What I like about experience is that it is such an honest thing. . . . You may have deceived yourself, but experience is not trying to deceive you.

The scenario approach strives for generality in order to provide guarantees that depend as little as possible on extra knowledge, besides the knowledge that comes from data. In this sense, it tries to make the most of the “honest thing.” Still, one aspect that must be kept in mind is that in scenario optimization it is assumed that the same probability distribution is shared by all of the observations. This fact may set limitations when the scenario approach is applied to long data records.

Chapter 5

Proofs

This chapter contains a proof of the main theorem, Theorem 3.7, for $d = 2$. The general case $d \neq 2$ follows *mutatis mutandis* with no conceptual modifications. In the proof, we do not cover the so-called degenerate case, which is rather technical while carrying little extra insight, and refer the reader to the relevant literature for that instead. In the last part of the chapter, we also sketch a proof of how Theorem 3.9 can be obtained by using Theorem 3.7.

5.1 ■ Support constraints and fully supported problems

Consider the scenario program in Figure 5.1. How many constraints, if removed, improve the solution? This happens for two constraints, those active at the solution. The same happens in Figure 5.2. Are there cases where the solution improves correspondingly with the removal of only one constraint? Yes; an example can be

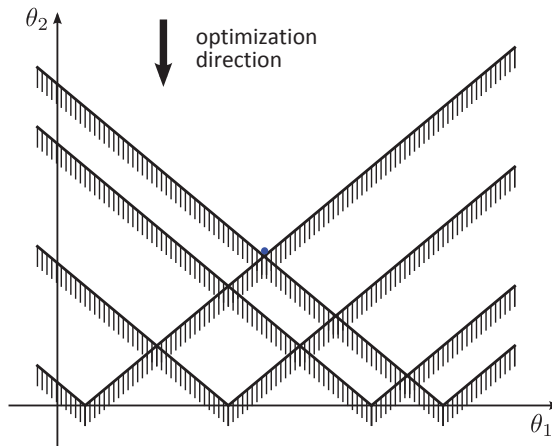


Figure 5.1. A scenario program with V-shaped constraints. If one of the two active constraints is removed, the solution improves.

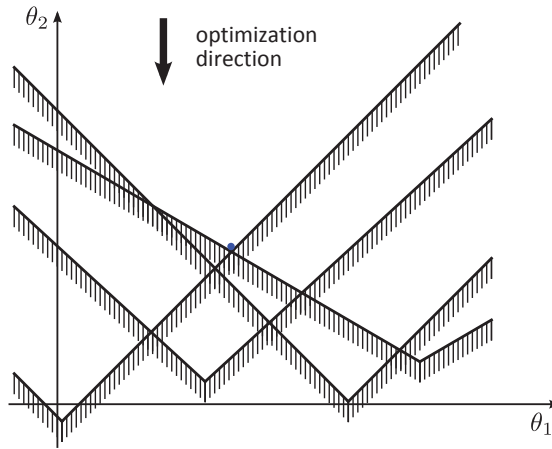


Figure 5.2. Another scenario program with V-shaped constraints. If one of the two active constraints is removed, the solution improves.

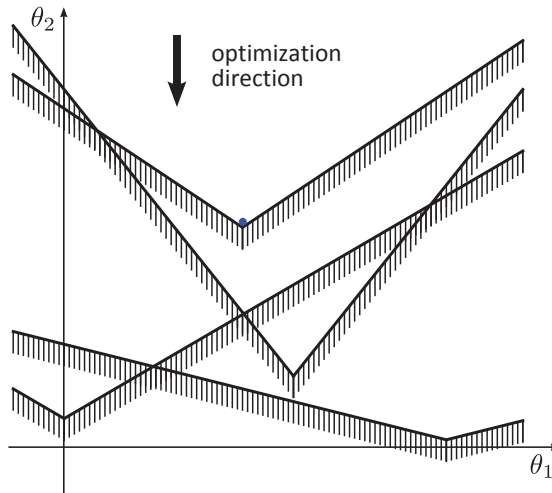


Figure 5.3. A scenario program where, for only one constraint, the solution improves if the constraint is removed.

seen in Figure 5.3. Can we instead create a situation with $d = 2$ where the solution improves with the removal of any constraint from a set of three constraints? Intuitively, this does not seem possible.

The above examples point to a fact that is central in the generalization theory of convex scenario problems: The number of scenarios supporting the solution cannot be arbitrarily large, and in fact it is bounded by the number of optimization variables. To formalize this concept, we start by giving the definition of support constraint.

Consider scenario programs similar to (3.1) where, however, there are m

constraints instead of N , m being any integer no smaller than $d = 2$:

$$\begin{aligned} \min_{\theta \in \Theta} c^T \theta \\ \text{subject to } \theta \in \bigcap_{i=1, \dots, m} \Theta_{\delta_i}, \end{aligned} \quad (5.1)$$

where $(\delta_1, \delta_2, \dots, \delta_m)$ is a sample of i.i.d. (independent and identically distributed) elements from $(\Delta, \mathbb{P})$.

Definition 5.1 (support constraint). A constraint $\theta \in \Theta_{\delta_i}$ of the scenario program (5.1) is a support constraint if its removal improves the solution of the program.

Theorem 5.2. For every m , the number of support constraints is less than or equal to 2.¹⁵

Proof of Theorem 5.2. The proof follows from an application of the following Helly's lemma, perhaps the most important result in convex set theory.

Lemma 5.3 (Helly's [59]). Given a collection of finitely many convex sets in $\mathbb{R}^2$, if the intersection of any group of three sets is nonempty, then the intersection of all sets is nonempty.

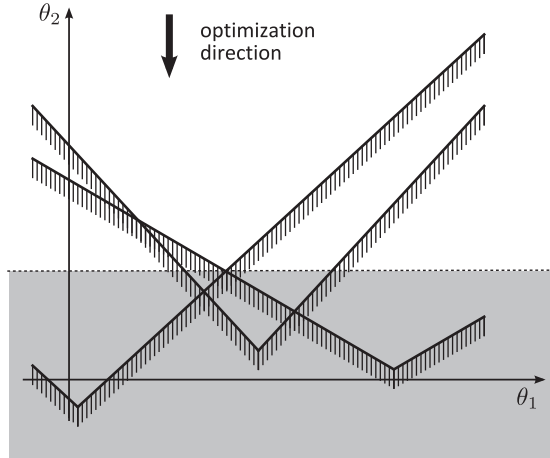


Figure 5.4. Set A of super-optimal θ 's (grey region) and sets Θ_{δ_i} in a scenario program.

To apply Helly's lemma in our context, consider the collection of convex sets given by the constraints $\theta \in \Theta_{\delta_i}$, $i = 1, 2, \dots, m$, and, in addition, one more set A that consists of the “super-optimal” θ 's, that is, those θ 's for which the cost $c^T \theta$ is strictly smaller than $c^T \theta^*$, where θ^* is the solution to (5.1). See Figure 5.4 for a graphical illustration of A and the sets Θ_{δ_i} . If any three sets of the type Θ_{δ_i} are considered, their intersection contains θ^* and is therefore nonempty. By contradiction, suppose now that the number of support constraints is more than two. If so, if we select any two sets of the type Θ_{δ_i} and A , then at least one support constraint is missing so that by definition of support constraint, the solution with the

¹⁵In dimension d , the number of support constraints is less than or equal to d .

two selected constraints in place is smaller than $c^T \theta^*$, and therefore this solution is also in A . This shows that the intersection of the two Θ_{δ_i} 's and A is again nonempty. Thus, the assumption in Helly's lemma that the intersection of any group of three sets from $\{\Theta_{\delta_i}, i = 1, \dots, m; A\}$ is nonempty is satisfied, and from Helly's lemma we have that the intersection of all Θ_{δ_i} 's and A is nonempty. But a point in this intersection is feasible for the scenario program (5.1) and super-optimal, contradicting the fact that θ^* is the optimal solution of (5.1). Hence, this leads to a contradiction, and therefore the number of support constraints cannot be larger than two. $\square$

We are now ready to introduce the notion of *fully supported* scenario problem.

Definition 5.4 (fully supported problem). *A scenario optimization problem, as characterized by the cost function $c^T \theta$, the domain Θ , the family of constraint sets $\{\Theta_{\delta}, \delta \in \Delta\}$, and the probability $\mathbb{P}$, is said to be fully supported if for every $m \geq d$ the number of support constraints of the program (5.1) is equal to d with probability 1 (probability 1 refers to the choices of the sample $(\delta_1, \delta_2, \dots, \delta_m)$).*

In the next section, we prove that the result stated in Theorem 3.7 holds with equality for fully supported problems, and that fully supported problems are in a sense worst case, so that the result that holds for fully supported problems provides a suitable bound in all other cases.

5.2 ■ Proof of the generalization Theorem 3.7

We consider first fully supported scenario optimization problems. Referring to the scenario program (3.1), which is (5.1) with $m = N$, we want to prove the validity of relation

$$\mathbb{P}^N \{V(\theta^*) > \epsilon\} = \sum_{i=0}^1 \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}, \quad (5.2)$$

which shows that the result stated in Theorem 3.7 holds with equality in this case.

Suppose first that $N = 2$. We prove that

$$\mathbb{P}^2 \{V(\theta^*) > \epsilon\} = \sum_{i=0}^1 \binom{2}{i} \epsilon^i (1-\epsilon)^{2-i} = 1 - \epsilon^2, \quad (5.3)$$

which is (5.2) for $N = 2$. This result forms the main step to prove the result for a generic $N \geq 2$.

For any $m \geq 2$, pick a realization of $(\delta_1, \delta_2, \dots, \delta_m)$ and consider program (5.1). Since we assumed that the problem is fully supported, with probability 1 there are two support constraints, say those with indexes $\bar{i}$ and $\bar{j}$, $\bar{i} < \bar{j}$. Consider $\binom{m}{2}$ baskets labeled with i, j , say $\mathcal{S}_{i,j}$, where $i < j$ are indexes from $\{1, 2, \dots, m\}$, and place the realization at hand in the basket with label $\bar{i}, \bar{j}$. Repeating this operation for all realizations gives a partition, up to a zero probability set, of Δ^m ; see Figure 5.5.

Let

$$F_V(\epsilon) = \mathbb{P}^2 \{V(\theta^*) \leq \epsilon\} \quad (5.4)$$

be the probability distribution of $V(\theta^*)$ when $N = 2$.¹⁶ We compute the probability of one set in the partition, say the first set $\mathcal{S}_{1,2}$, in two different ways, and then by

¹⁶We want to prove that (5.3) holds. Until (5.3) is proven, $F_V(\epsilon)$ is simply regarded as the distribution of $V(\theta^*)$, whose expression is not known.

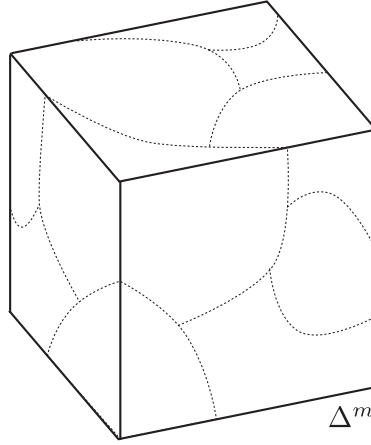


Figure 5.5. Partition of Δ^m . Each set in the partition contains realizations of the samples whose support constraints have the same indexes.

equalizing the two expressions that have been obtained, we draw interesting conclusions on $F_V(\epsilon)$.

First, we have

$$\mathbb{P}^m\{\mathcal{S}_{1,2}\} = \int_{[0,1]} (1-\alpha)^{m-2} F_V(d\alpha). \quad (5.5)$$

To establish (5.5), let $\widetilde{\mathcal{S}}_{1,2}$ be the set where $\delta_3, \dots, \delta_m$ are not violated by the solution obtained with only the constraints $\theta \in \Theta_{\delta_1}$ and $\theta \in \Theta_{\delta_2}$ in place. It is an intuitive fact that $\mathcal{S}_{1,2}$ and $\widetilde{\mathcal{S}}_{1,2}$ are the same up to a probability 0 set, and to streamline the presentation, we postpone the proof of this fact until the end of this first part of the proof devoted to fully supported problems. Hence, we compute $\mathbb{P}^m\{\widetilde{\mathcal{S}}_{1,2}\}$, which is the same as $\mathbb{P}^m\{\mathcal{S}_{1,2}\}$, and show that it is given by (5.5).

Select fixed values $\bar{\delta}_1, \bar{\delta}_2$ for δ_1, δ_2 and let $V(\theta_{1,2}^*)$ be the violation of the solution $\theta_{1,2}^*$ with only the two constraints $\theta \in \Theta_{\bar{\delta}_1}$ and $\theta \in \Theta_{\bar{\delta}_2}$ in place. Then, the probability that the scenarios $\delta_3, \dots, \delta_m$ fall in the nonviolated set, that is, the probability that $(\bar{\delta}_1, \bar{\delta}_2, \delta_3, \dots, \delta_m) \in \widetilde{\mathcal{S}}_{1,2}$, is $(1 - V(\theta_{1,2}^*))^{m-2}$. Integrating over the domain Δ^2 of (δ_1, δ_2) , we then have

$$\begin{aligned} \mathbb{P}^m\{\widetilde{\mathcal{S}}_{1,2}\} &= \int_{\Delta^2} (1 - V(\theta_{1,2}^*))^{m-2} \mathbb{P}^2(d\bar{\delta}_1, d\bar{\delta}_2) \\ &= \int_{[0,1]} (1-\alpha)^{m-2} F_V(d\alpha), \end{aligned}$$

where the last equality is a change of variables from the domain $(\bar{\delta}_1, \bar{\delta}_2)$ to that of the violation of the corresponding solution. Since $\mathbb{P}^m\{\mathcal{S}_{1,2}\} = \mathbb{P}^m\{\widetilde{\mathcal{S}}_{1,2}\}$, this gives (5.5).

The second way of writing $\mathbb{P}^m\{\mathcal{S}_{1,2}\}$ is simply obtained by noting that all sets $\mathcal{S}_{i,j}$ have the same probability since all (δ_i, δ_j) are interchangeable. Since sets $\mathcal{S}_{i,j}$

form a partition of Δ^m up to a probability 0 set, we then have

$$\sum_{i,j} \mathbb{P}^m \{\mathcal{S}_{1,2}\} = 1 \Rightarrow \binom{m}{2} \mathbb{P}^m \{\mathcal{S}_{1,2}\} = 1 \Rightarrow \mathbb{P}^m \{\mathcal{S}_{1,2}\} = \frac{1}{\binom{m}{2}},$$

that is, the probability of each set is 1 over the number of sets in the partition. Equalizing this last expression with the right-hand side of (5.5) yields

$$\frac{1}{\binom{m}{2}} = \int_{[0,1]} (1-\alpha)^{m-2} F_V(d\alpha). \quad (5.6)$$

Since m is any integer ≥ 2 , (5.6) gives all moments of the distribution $F_V(\epsilon)$ (*moment problem*). It is easy to see by integration by parts that $F_V(\epsilon) = \epsilon^2$ (compare with (5.3)) satisfies (5.6) for all $m \geq 2$. On the other hand, no other solution exists since the solution to a moment problem for a distribution supported over the compact set $[0, 1]$ is unique (see, e.g., Corollary 1, §12.9, Chapter II of [83]). Thus, using the definition of $F_V(\epsilon)$ in (5.4), (5.3) is proven.

Turn now to consider program (3.1). In light of the assumption of fully support-ness, program (3.1) also has two support constraints with probability 1. Partition the event $\{(\delta_1, \dots, \delta_N) \in \Delta^N : V(\theta^*) > \epsilon\}$ by intersecting it with the $\binom{N}{2}$ sets $\mathcal{S}_{i,j}$ obtained by grouping together the realizations of $(\delta_1, \dots, \delta_N)$ whose support constraints have indexes i, j . We then have

$$\begin{aligned} & \mathbb{P}^N \{V(\theta^*) > \epsilon\} \\ &= \mathbb{P}^N \left\{ \cup_{i,j} \{V(\theta^*) > \epsilon \text{ and } \theta^* \text{ is supported by the constraints with indexes } i, j\} \right\} \\ &= \sum_{i,j} \mathbb{P}^N \{V(\theta^*) > \epsilon \text{ and } \theta^* \text{ is supported by the constraints with indexes } i, j\} \\ &= [\mathbf{1}_A \text{ is the indicator function of set } A, \text{ i.e., } \mathbf{1}_A = 1 \text{ over } A \text{ and } \mathbf{1}_A = 0 \text{ otherwise}] \\ &= \binom{N}{2} \int_{\Delta^2} (1 - V(\theta_{1,2}^*))^{N-2} \mathbf{1}_{\{V(\theta_{1,2}^*) > \epsilon\}} \mathbb{P}^2(d\bar{\delta}_1, d\bar{\delta}_2) \\ &= \binom{N}{2} \int_{(\epsilon, 1]} (1-\alpha)^{N-2} F_V(d\alpha) \\ &= [\text{since } F(d\alpha) = 2\alpha \, d\alpha] \\ &= \binom{N}{2} \int_{(\epsilon, 1]} [(1-\alpha)^{N-2} 2\alpha] \, d\alpha \\ &= [\text{integrating by parts}] \\ &= \sum_{i=0}^1 \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}, \end{aligned}$$

which shows that the result stated in Theorem 3.7 holds with equality for fully supported problems.

Proof of the fact that $\mathcal{S}_{1,2} = \widetilde{\mathcal{S}}_{1,2}$ up to a probability zero set

$\mathcal{S}_{1,2} \subseteq \widetilde{\mathcal{S}}_{1,2}$: Take a $(\delta_1, \dots, \delta_m) \in \mathcal{S}_{1,2}$ and eliminate a constraint among $\delta_3, \dots, \delta_m$. Since this constraint is not of support, the solution remains unchanged; moreover, it is easy to see that the first two constraints are still the support constraints for the problem with $m-1$ constraints. If we now remove another constraint among

those that are not of support, the conclusion is similarly drawn that the solution remains unchanged, and the first two constraints are still the support constraints for the problem with $m - 2$ constraints. Proceeding this way until all constraints but the first two are removed, we obtain that the solution with only the two support constraints $\theta \in \Theta_{\delta_1}$, $\theta \in \Theta_{\delta_2}$ in place is the same as the solution with all m constraints. Since no constraint among $\delta_3, \dots, \delta_m$ can be violated by the solution with all m constraints and such a solution is the same as the solution with only the first two constraints, it follows that $(\delta_1, \dots, \delta_m) \in \widetilde{\mathcal{S}}_{1,2}$.

$\widetilde{\mathcal{S}}_{1,2} \subseteq \mathcal{S}_{1,2}$ up to a probability 0 set: Suppose now that $\delta_3, \dots, \delta_m$ are not violated by the solution generated by δ_1, δ_2 , i.e., $(\delta_1, \dots, \delta_m) \in \widetilde{\mathcal{S}}_{1,2}$. A simple reasoning reveals that $(\delta_1, \dots, \delta_m)$ does not belong to any other set $\mathcal{S}_{i,j} \neq \mathcal{S}_{1,2}$. In fact, adding non-violated constraints to δ_1, δ_2 does not change the solution, and each of the added constraints can be removed without altering the solution. Therefore, none of the constraints $\delta_3, \dots, \delta_m$ can be of support, and hence the sample is not in any $\mathcal{S}_{i,j} \neq \mathcal{S}_{1,2}$. It follows that $\widetilde{\mathcal{S}}_{1,2}$ is a subset of the complement of $\cup_{(i,j) \neq (1,2)} \mathcal{S}_{i,j}$, which is $\mathcal{S}_{1,2}$ up to a probability 0 set.

We next consider non-fully supported problems and show the validity of (3.4) in general.

For a non-fully supported problem, scenario program (5.1) has less than two support constraints with nonzero probability. A support constraint has to be an active constraint, and the typical reason for a lack of support constraints is that at the optimum the active constraints are less than two; see Figure 5.3. To carry on a proof along lines akin to those for the fully supported case, we are well-advised to generalize the notion of solution to that of ball-solution; a ball-solution always has at least two active constraints. For simplicity, we henceforth assume that constraints are not trivial, i.e., $\Theta_{\delta} \neq \mathbb{R}^2$, $\forall \delta \in \Delta$.

Roughly speaking, given a program whose solution is θ^* , its *ball-solution* is a ball centered in θ^* whose radius has been enlarged until the ball touches two constraints. See Figure 5.6 for an example of ball-solution. The mathematical definition of ball-solution is as follows.

Definition 5.5 (ball-solution). *The ball-solution $\mathcal{B}(\theta^*, r^*)$ is the largest closed ball centered in θ^* fully contained in the feasibility domain of all constraints with the exception of at most $d - 1$ of them, i.e., $\Theta_{\delta_i} \cap \mathcal{B}(\theta^*, r^*) = \mathcal{B}(\theta^*, r^*)$ for all i 's, except at most one of them.*

Note also that when there are two or more active constraints, $r^* = 0$ and $\mathcal{B}(\theta^*, r^*)$ reduces to the standard solution θ^* .

The notion of active constraint can be generalized to balls by saying that a constraint is active for a ball if the ball touches the boundary of the constraint. If in addition the ball is fully contained in the constraint, then the constraint is said to be strictly active. See Figure 5.7 for a graphical illustration of active and strictly active constraints for a ball, while the precise definition is as follows.

Definition 5.6 (active constraint for a ball). *A constraint $\theta \in \Theta_{\delta}$ is active for a ball $\mathcal{B}(\theta, r)$ if $\Theta_{\delta} \cap \mathcal{B}(x, r) \neq \emptyset$ and $\Theta_{\delta} \cap \mathcal{B}(x, r + h) \neq \mathcal{B}(x, r + h)$, $\forall h > 0$. If in addition $\Theta_{\delta} \cap \mathcal{B}(x, r) = \mathcal{B}(x, r)$, $\theta \in \Theta_{\delta}$ is said to be strictly active.*

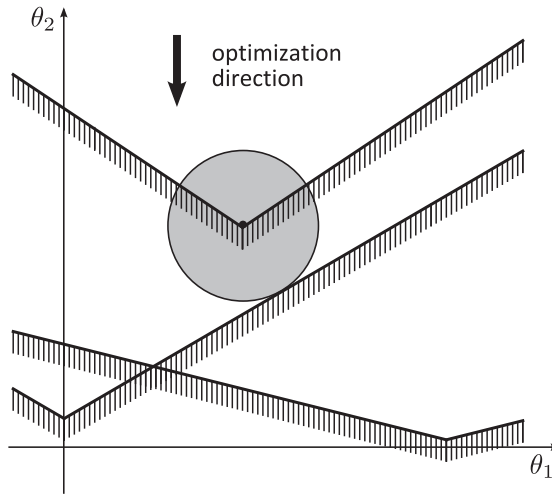


Figure 5.6. Ball-solution.

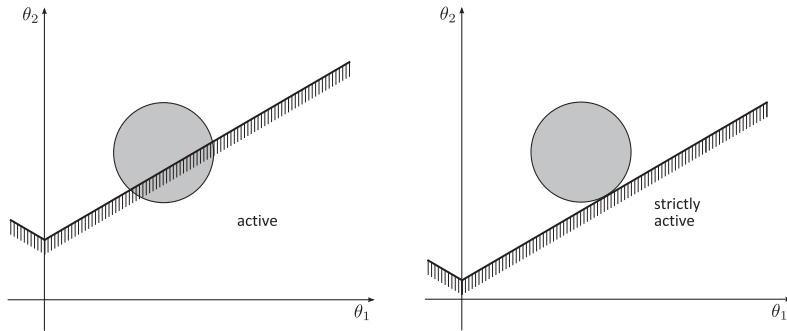


Figure 5.7. Active and strictly active constraint for a ball.

If the ball is a single point, active and strictly active is the same and reduces to the standard notion of active.

By construction, a ball-solution has at least two active constraints. To go back to the track of the proof for the case of fully supported problems, however, we need to have exactly two support constraints. The following definition naturally extends the notion of support constraint to the case of ball-solutions.

Definition 5.7 (ball-support constraint). A constraint $\theta \in \Theta_{\delta_i}$ of the scenario program (5.1) is a ball-support constraint if its removal changes the ball-solution.

An active constraint need not be a ball-support constraint, nor does a scenario program always have two ball-support constraints (see Figure 5.8, where δ_2 and δ_3 are not of support). It is clear that the number of ball-support constraints is less than or equal to 2. The case with less than two ball-support constraints is regarded as *degenerate*, since it requires that constraints not be generically distributed. As degeneracy is not dealt with in this book, the reader is referred to [27] for a full

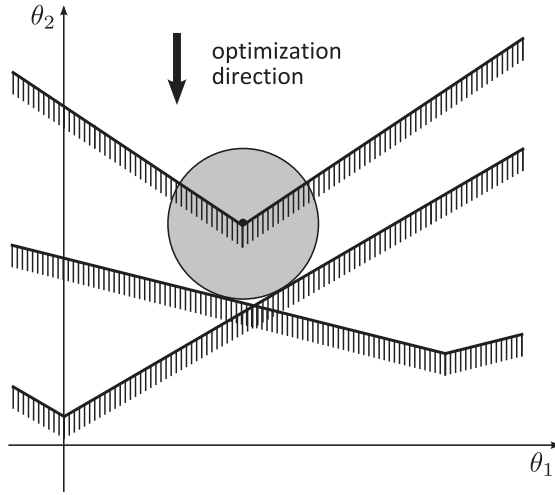


Figure 5.8. Only δ_1 is a ball-support constraint.

presentation of this case. Hence, we proceed by assuming that the scenario optimization problem we are dealing with is fully ball-supported according to the following definition.

Definition 5.8 (fully ball-supported problem). A scenario optimization problem, as characterized by the cost function $c^T \theta$, the domain Θ , the family of constraint sets $\{\Theta_\delta, \delta \in \Delta\}$, and the probability $\mathbb{P}$, is said to be fully ball-supported if for every $m \geq d$ the number of ball-support constraints of the program (5.1) is equal to d with probability 1 (probability 1 refers to the choices of the sample $(\delta_1, \delta_2, \dots, \delta_m)$).

To proceed, we need to introduce the notion of constraint violated by a ball: A constraint $\theta \in \Theta_\delta$ is violated by $\mathcal{B}(\theta, r)$ if $\Theta_\delta \cap \mathcal{B}(\theta, r) \neq \mathcal{B}(\theta, r)$. The definition of violation probability then generalizes naturally to the ball case.

Definition 5.9 (violation probability of a ball). The violation probability (or just violation) of a given ball $\mathcal{B}(\theta, r)$, $\theta \in \Theta$, $r \geq 0$, is defined as $V_{\mathcal{B}}(\theta, r) = \mathbb{P}\{\delta \in \Delta : \Theta_\delta \cap \mathcal{B}(\theta, r) \neq \mathcal{B}(\theta, r)\}$.

Clearly, $V_{\mathcal{B}}(\theta, r) \geq V(\theta)$. Hence, if θ^* and $\mathcal{B}(\theta^*, r^*)$ are the solution and the ball-solution of the scenario program (3.1), we have that

$$\mathbb{P}^N\{V(\theta^*) > \epsilon\} \leq \mathbb{P}^N\{V_{\mathcal{B}}(\theta^*, r^*) > \epsilon\}. \quad (5.7)$$

Below, we show that a result similar to (5.2) holds for fully ball-supported problems, namely,

$$\mathbb{P}^N\{V_{\mathcal{B}}(\theta^*, r^*) > \epsilon\} = \sum_{i=0}^1 \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}, \quad (5.8)$$

and this result, together with (5.7), leads to the thesis that

$$\mathbb{P}^N \{V(\theta^*) > \epsilon\} \leq \sum_{i=0}^1 \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}.$$

The proof of (5.8) repeats verbatim the proof for standard solutions of fully supported problems given before provided that one replaces

- (i) *solution* with *ball-solution*,
- (ii) *support constraint* with *ball-support constraint*,
- (iii) *violation probability* V with *violation probability of a ball* $V_{\mathcal{B}}$,

with only one exception: The part where we proved that $\mathcal{S}_{1,2} \subseteq \widetilde{\mathcal{S}}_{1,2}$ has to be modified in a way that we spell out in the following.

The first rationale to conclude that “the solution with only the two support constraints $\theta \in \Theta_{\delta_1}$, $\theta \in \Theta_{\delta_2}$ in place is the same as the solution with all m constraints” is still valid and leads in our present context to the fact that the *ball-solution* with only the two *ball-support* constraints $\theta \in \Theta_{\delta_1}$, $\theta \in \Theta_{\delta_2}$ in place is the same as the *ball-solution* with all m constraints. Instead, the last argument with which we concluded that $\mathcal{S}_{1,2} \subseteq \widetilde{\mathcal{S}}_{1,2}$ is no longer valid since ball-solutions can violate constraints. To amend it, suppose for the purpose of contradiction that a constraint among $\theta \in \Theta_{\delta_3}, \dots, \theta \in \Theta_{\delta_m}$, say $\theta \in \Theta_{\delta_3}$, is violated by the ball-solution with two constraints. Two cases can occur: (i) The ball-solution has only one strictly active constraint among $\theta \in \Theta_{\delta_1}, \theta \in \Theta_{\delta_2}$; or (ii) it has more than one. In case (i), one constraint among $\theta \in \Theta_{\delta_1}, \theta \in \Theta_{\delta_2}$ is violated by the ball-solution, so that with the extra $\theta \in \Theta_{\delta_3}$ violated constraint, the number of violated constraints of the ball-solution with m constraints would add up to at least 2; this contradicts the definition of ball-solution. If instead (ii) is true, a simple reasoning reveals that with one more constraint $\theta \in \Theta_{\delta_3}$ violated by the ball-solution, the strictly active constraints (which, in this case, are more than 1) cannot be of ball-support for the problem with m constraints; this contradicts the fact that $(\delta_1, \dots, \delta_m) \in \mathcal{S}_{1,2}$.

5.3 ■ Proof of Theorem 3.9

Given a subset $I = \{i_1, \dots, i_k\}$ of k indexes from $\{1, \dots, N\}$, let θ_I^* be the solution of the scenario program without the constraints with index in I , i.e.,

$$\begin{aligned} & \min_{\theta \in \Theta} c^T \theta \\ & \text{subject to } \theta \in \bigcap_{i \in \{1, \dots, N\} - I} \Theta_{\delta_i}. \end{aligned} \quad (5.9)$$

Moreover, let

$$\Delta_I^N = \{(\delta_1, \dots, \delta_N) \in \Delta^N : \theta_I^* \text{ violates the constraints } \theta \in \Theta_{\delta_{i_1}}, \dots, \theta \in \Theta_{\delta_{i_k}}\}. \quad (5.10)$$

Thus, Δ_I^N contains the sample realizations for which removing the constraints with indexes in I leads to a solution θ_I^* that violates all the removed constraints.

Since θ_k^* violates k constraints with probability 1, it must be that $\theta_k^* = \theta_I^*$ for some I such that $(\delta_1, \dots, \delta_N) \in \Delta_I^N$. Thus,

$$\{(\delta_1, \dots, \delta_N) \in \Delta^N : V(\theta_k^*) > \epsilon\} \subseteq \bigcup_{I \in \mathcal{I}} \{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} \quad (5.11)$$

up to a zero probability set, where $\mathcal{I}$ is the collection of all possible choices of k indexes from $\{1, \dots, N\}$.

A bound for $\mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta^N : V(\theta_k^*) > \epsilon\}$ is now obtained by first bounding $\mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\}$ for one I , and then summing over $I \in \mathcal{I}$.

Fix an $I = \{i_1, \dots, i_k\}$, and write

$$\begin{aligned} & \mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} \\ &= \int_{(\epsilon, 1]} \mathbb{P}^N\{\Delta_I^N | V(\theta_I^*) = \alpha\} F_V(d\alpha) \\ &= \int_{(\epsilon, 1]} \mathbb{P}^N\{\theta_I^* \text{ violates the constraints } \Theta_{\delta_{i_1}}, \dots, \Theta_{\delta_{i_k}} | V(\theta_I^*) = \alpha\} F_V(d\alpha), \end{aligned} \quad (5.12)$$

where F_V is the cumulative distribution function of the random variable $V(\theta_I^*)$, and $\mathbb{P}^N\{\Delta_I^N | V(\theta_I^*) = \alpha\}$ is the conditional probability of the event Δ_I^N under the condition that $V(\theta_I^*) = \alpha$ (for an explanation of this conditioning operation, see, e.g., equation (17), § 7, Chapter II of [83]).

To evaluate the integrand in (5.12), recall that $V(\theta_I^*) = \alpha$ means that θ_I^* violates constraints with probability α ; then, owing to the fact that the δ_i 's are independent, the integrand equals α^k . Substituting in (5.12) yields

$$\mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} = \int_{(\epsilon, 1]} \alpha^k F_V(d\alpha). \quad (5.13)$$

To proceed, note that $F_V(\alpha)$ is the cumulative distribution function of the violation of a solution obtained with $N-k$ constraints. Hence, an application of Theorem 3.7 gives $F_V(\alpha) \geq 1 - \sum_{i=0}^1 \binom{N-k}{i} \alpha^i (1-\alpha)^{N-k-i} =: F_B(\alpha)$. Now, the integrand α^k in (5.13) is an increasing function of α , so that $F_V(\alpha) \geq F_B(\alpha)$ implies that $\int_{(\epsilon, 1]} \alpha^k F_V(d\alpha) \leq \int_{(\epsilon, 1]} \alpha^k F_B(d\alpha)$, as the following calculation shows:

$$\begin{aligned} & \int_{(\epsilon, 1]} \alpha^k F_V(d\alpha) = [\text{Theorem 11, §6, Chapter II of [83]}] \\ &= 1 - \epsilon^k F_V(\epsilon) - \int_{(\epsilon, 1]} F_V(\alpha) k \alpha^{k-1} d\alpha \\ &\leq 1 - \epsilon^k F_B(\epsilon) - \int_{(\epsilon, 1]} F_B(\alpha) k \alpha^{k-1} d\alpha \\ &= \int_{(\epsilon, 1]} \alpha^k F_B(d\alpha). \end{aligned}$$

Hence, $\mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\}$ can finally be bounded as follows:

$$\begin{aligned}
 \mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} &\leq \int_{(\epsilon, 1]} \alpha^k F_B(d\alpha) \\
 &= \left[\text{the density of } F_B \text{ is } 2 \binom{N-k}{2} \alpha(1-\alpha)^{N-k-2} \right] \\
 &= \int_{(\epsilon, 1]} \alpha^k \cdot 2 \binom{N-k}{2} \alpha(1-\alpha)^{N-k-2} d\alpha \\
 &= [\text{integration by parts}] \\
 &= \frac{2 \binom{N-k}{2}}{(k+2) \binom{N}{k+2}} \sum_{i=0}^{k+1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}. \tag{5.14}
 \end{aligned}$$

To conclude the proof, go back to (5.11) and note that $\mathcal{I}$ contains $\binom{N}{k}$ choices and that (5.14) holds irrespective of the chosen I . Thus,

$$\begin{aligned}
 \mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta^N : V(\theta_k^*) > \epsilon\} &\leq \sum_{I \in \mathcal{I}} \mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} \\
 &= \binom{N}{k} \mathbb{P}^N\{(\delta_1, \dots, \delta_N) \in \Delta_I^N : V(\theta_I^*) > \epsilon\} \\
 &\leq [\text{use (5.14)}] \\
 &\leq \binom{N}{k} \frac{2 \binom{N-k}{2}}{(k+2) \binom{N}{k+2}} \sum_{i=0}^{k+1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i} \\
 &= \binom{k+1}{k} \sum_{i=0}^{k+1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i},
 \end{aligned}$$

which is (3.13) for $d = 2$.

Chapter 6

Region estimation models

Estimation problems are problems where the value of a hidden and difficult-to-access variable is estimated from other, more accessible, quantities. They represent a fundamental part of many endeavors in the practice of science and engineering. In this chapter, we consider region estimation models where the hidden variable is estimated to belong to a region in the domain where the variable takes value, and show that region estimation models with guaranteed probability of successful estimation can be constructed using the scenario theory. This chapter is based on the results in [25].

6.1 ■ Observations and estimation models

Figure 6.1 displays a set of N observations collected in the past. Say that tomorrow we are given the value u_{N+1} of yet another point taken from the same population as the first N observations and we are asked to estimate the value y_{N+1} . The reason for estimating y_{N+1} from u_{N+1} is that y is a difficult-to-access variable, while u can be measured more easily. To give concreteness to this problem, think of a medical application where u is the outcome of a clinical test and y is the degree of a certain disease. How do we approach this problem?

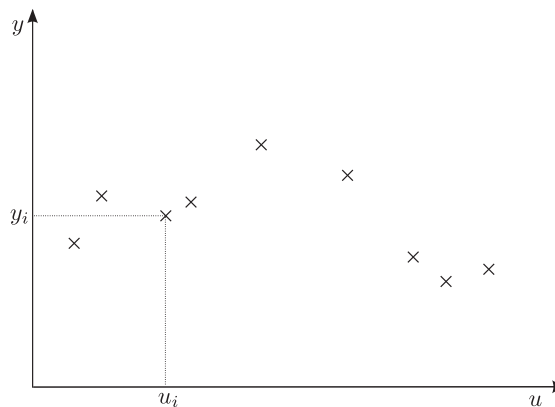


Figure 6.1. A set of observations in the domain (u, y) .

6.1.1 ■ Region estimation models

The value y_{N+1} is probably determined by many variables, so knowing u_{N+1} is not enough information to uniquely establish y_{N+1} . This situation is illustrated in Figure 6.2(a), where the value of y is determined by two variables: cause 1 and cause 2. Suppose that only cause 2 is measured and that $u = \text{cause 2}$. This corresponds to looking at the picture along the axis of cause 1, so that the function that determines y is projected onto the (u, y) domain and becomes a multivalued map $u \rightarrow y$; see Figure 6.2(b). This suggests that a sensible approach to the prediction of y_{N+1} consists of delivering a whole region $R(u_{N+1})$ to which y_{N+1} is expected to belong (Figure 6.3(a)). As u is varied, the region $R(u)$ forms what we call a region estimation model (REM), as illustrated in Figure 6.3(b).

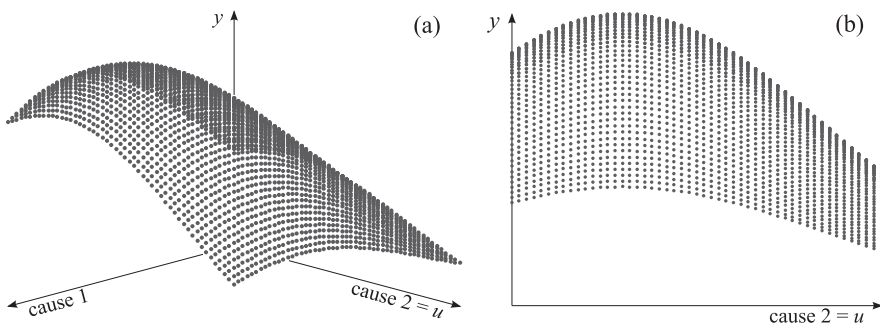


Figure 6.2. The cause-effect map. (a) The value of y is determined by cause 1 and cause 2; (b) function y projected onto the (u, y) domain.

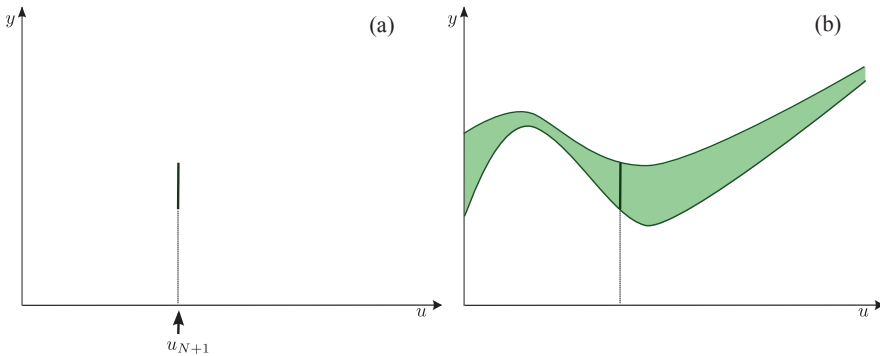


Figure 6.3. A region estimation model (REM); to a u , a REM associates a region $R(u)$. (a) The region $R(u_{N+1})$ associated to one value u_{N+1} of u . (b) $R(u)$ as a function of u .

6.1.2 ■ Reliability and size of a REM

How good is a REM? Figure 6.4 illustrates the two attributes of a REM that are relevant to determine its quality. Suppose that a map $u \rightarrow y$ has a support as shown in Figure 6.4(a). *Reliability* is the characteristic that a REM provides correct estimates, that is, $y \in R(u)$ most of the time. This happens for the REM in Figures 6.4(c) and 6.4(d) since the support of the map $u \rightarrow y$ is included in the REM domain,

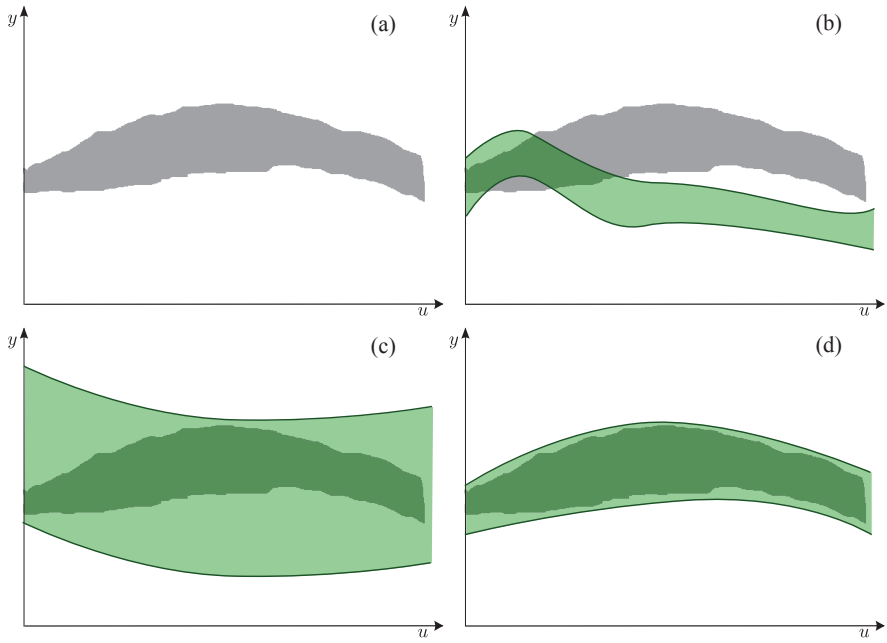


Figure 6.4. *The two attributes of a REM. (a) The grey region is the support for the multivalued map $u \rightarrow y$. (b) The REM returns a reasonably narrow estimation interval, but this interval is not reliable. (c) The REM is reliable but not tight. (d) The REM is reliable and tight.*

while this condition is not satisfied by the REM of Figure 6.4(b). Reliability, however, is not the only important characteristic. Considering Figure 6.4(c), the REM in this figure is reliable but not as tight as the REM in Figure 6.4(d), which means that the average size of the latter is smaller than that of the former. As a consequence, when the REM in Figure 6.4(c) is used, it returns unduly large intervals that may not be very useful in practice.

It is a simple, and yet important, remark to note that it makes no sense to say that “a model is reliable” without any further specification since reliability refers to the ability of performing a correct estimate in relation to a given problem. Hence, the same model can be reliable for one estimation problem and not for another. In contrast, the model size refers to the expanse of the model, and this is a characteristic of the model only. In short,

- (a) reliability is a property with two arguments: model and reality; while
- (b) the model size depends only on one argument: the model.

The above observation is quite relevant to the practice of identifying a REM from data. After the REM has been identified, its size can be evaluated for satisfaction, and the REM can possibly be rejected if its size is judged to be excessive to make the REM practically useful. This introduces a feedback by which the identification procedure is adjusted to meet adequate standards for the model size. For example, the variable u that has been used can be complemented with some other variables to make the estimation more accurate. On the other hand, the model reliability is not directly measurable even after that the identification process has been terminated

since it also depends on reality. For this reason, a theory is needed that provides us with an evaluation of the reliability, while the model size can be directly judged. This state of things also has implications on the use of *prior information*. When designing a scheme for identifying a REM from data, one normally incorporates the prior information that is available. While such prior information carries useful knowledge that is used to steer the identification process towards models that are more likely to describe the phenomenon under consideration, it is a good thing that the theory by which the reliability of the model is judged does not depend on the prior information. Indeed, if the prior information is incorrect, the judgement of reliability, which is what we cannot verify even *a posteriori*, remains intact. We shall see that the scenario theory indicates a route to obtain this result.

6.1.3 ■ Identification of a REM

Given a set of observations $(u_1, y_1), (u_2, y_2), \dots, (u_N, y_N)$, our goal is to identify a REM that meets suitable standards in terms of size and reliability. Following the example given in Section 1.2.2, here we present a simple construction that allows us to rapidly focus on significant aspects of the theory. Extensions will follow in the next section.

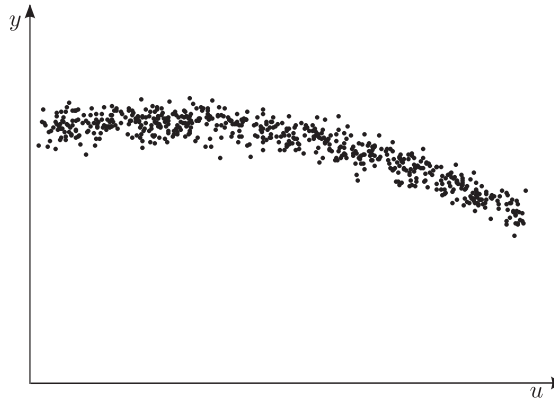


Figure 6.5. The data points.

Consider the data points in Figure 6.5. We want to identify a REM that is structured as a layer with constant size centered around a quadratic polynomial.¹⁷ Such a layer is sometimes called a Chebyshev layer; see Figure 6.6. Since we desire to secure good standards of reliability, it makes sense to require that the layer agrees with the sample of data points.¹⁸ Moreover, to make the model useful in estimation, among models that satisfy this constraint we want to favor models that have smaller size. This naturally leads us to consider the optimization program

$$\min_{v_1, v_2, v_3} \left[\max_{i=1, \dots, N} |y_i - [v_1 + v_2 u_i + v_3 u_i^2]| \right], \quad (6.1)$$

¹⁷It appears that we have many data points to estimate the layer. This is so because u is monodimensional, which was considered to make the figures easily inspectable. In general, u can have any dimension, and if the same number of data points were given in a domain where u contains, say, six variables, the data points would be sparse.

¹⁸In some cases, one may prefer to allow the REM to incorrectly predict data points that have “odd” behavior (outliers) so that a smaller model is obtained. This approach is briefly discussed in Section 6.2.

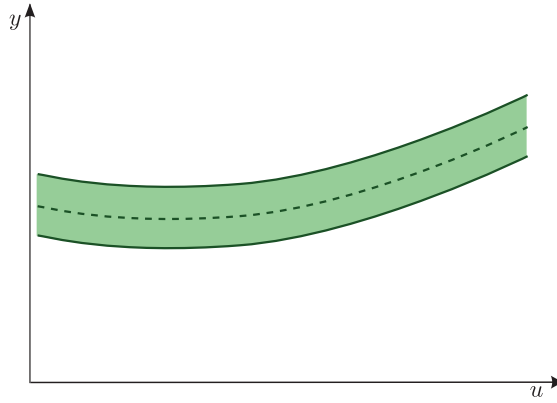


Figure 6.6. A Chebyshev layer.

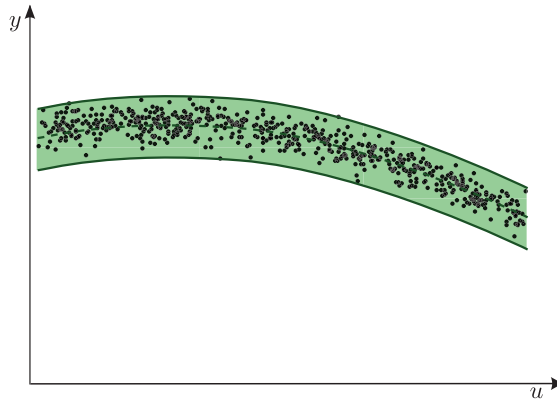


Figure 6.7. The identified Chebyshev layer.

which we already saw in Chapter 1, as equation (1.7). Applying this approach to the data set at hand, we obtained the result in Figure 6.7.

6.1.4 ■ Theoretical evaluation of reliability

How far can we trust the model that has been identified from (6.1)? Let us assume that the values (u, y) occur with an unknown probability $\mathbb{P}$. Figure 6.8 shows the support of $\mathbb{P}$ in grey and the prediction layer in green. If the next observation happens to be in the portion of the grey region that is not covered by the layer, the layer fails to correctly predict y_{N+1} . The probability for this to happen is $\mathbb{P}\{y_{N+1} \notin R(u_{N+1})\}$. If we now rewrite (6.1) in epigraphic form as we did in Section 1.3, so obtaining

$$\begin{aligned} \min_{\ell, \nu_1, \nu_2, \nu_3} \quad & \ell \\ \text{subject to} \quad & |y_i - [\nu_1 + \nu_2 u_i + \nu_3 u_i^2]| \leq \ell, \end{aligned} \quad (6.2)$$

we see that $\mathbb{P}\{y_{N+1} \notin R(u_{N+1})\}$ can be rewritten as

$$\mathbb{P}\{|y_{N+1} - [\nu_1^* + \nu_2^* u_{N+1} + \nu_3^* u_{N+1}^2]| > \ell^*\}.$$

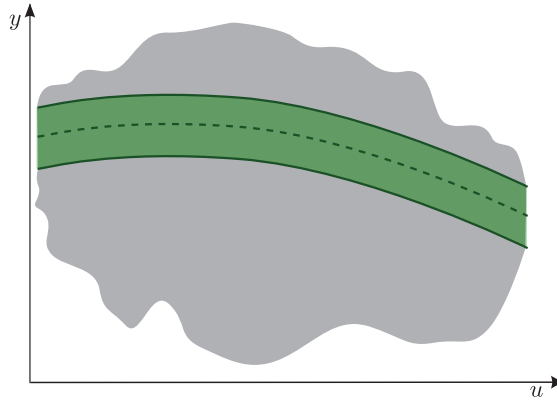


Figure 6.8. *Reliability of a Chebyshev layer. The next observation is incorrectly estimated if it falls in the grey region that is not covered by the layer.*

According to Definition 3.1, this is the violation $V(\gamma^*, \ell^*)$, and hence Theorem 3.7, and all the ensuing scenario results, can be applied to this context. For instance, if we assume that the observations $(u_1, y_1), (u_2, y_2), \dots, (u_N, y_N)$ are independent, Theorem 1.1 in the present context becomes the following.

Theorem 6.1. *For any $\epsilon \in (0, 1)$ (risk parameter) and $\beta \in (0, 1)$ (confidence parameter), if the number of observations N satisfies $N \geq \frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$, then, with probability $\geq 1 - \beta$, the Chebyshev layer fails to correctly predict (y_{N+1}, u_{N+1}) with probability no more than ϵ .*

More generally, Theorem 3.7 can be resorted to draw conclusions on the model reliability. Applying Theorem 3.7 to the example in Section 6.1.3, where we had $N = 620$ and $d = 4$, with $\beta = 10^{-10}$, we obtain $\epsilon = 5\%$.

Interestingly, all statements in the scenario theory are distribution-free, which, in the present context, means that they hold true irrespective of the domain of u and y and of the probability distribution by which u and y take value. As a consequence, the evaluations of the reliability of the identified REM returned by Theorem 6.1 (or, more generally, those obtained by resorting to Theorem 3.7) are rigorously correct without any need for prior information on the mechanism generating the pair (u, y) . As anticipated, this does not mean in any way that prior information is of no use in the identification of a REM. Prior information plays a fundamental role in selecting which variables are to be incorporated in u and also in deciding which class of REM is best used to obtain a small size of the layer.

The fact that all statements in the scenario theory are distribution-free also implies that the number of variables present in u is immaterial in the evaluation of the reliability results. This can be seen from the statement of Theorem 6.1, where the reliability result remains the same should u contain 1, 2, or 10 variables. Observing that, e.g., 620 data points as in Figure 6.5 would look very sparse in a 20-dimensional space, this invariance result may appear strange. On the other hand, it is worth remarking that the number of variables contained in u may have an indirect role in the evaluation of reliability. In fact, in a high-dimensional space a model class requires more degrees of freedom than in few dimensions to secure enough flexibility of description. This increase in the degrees of freedom is secured with more

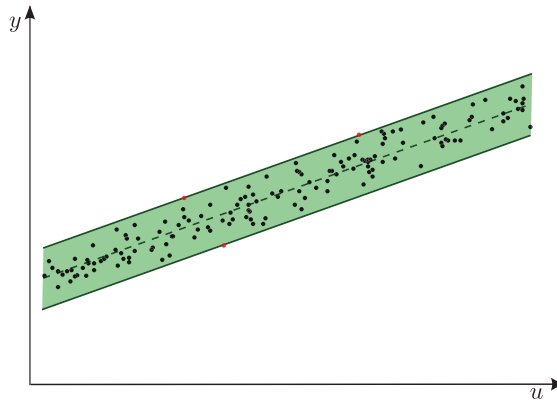


Figure 6.9. *An optimal Chebyshev layer. The boundary touches three data points.*

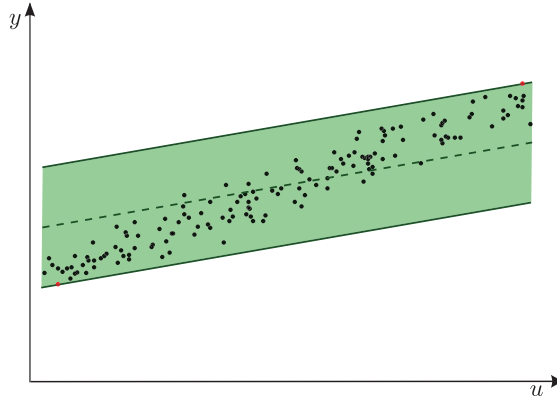


Figure 6.10. *A not optimal layer whose boundary touches two data points.*

parameters, a fact that impacts on the model reliability through the dependence of the reliability results on d .

Another interesting observation is that Chebyshev layers offer a nice example of fully supported problems. Consider the layer in Figure 6.9 centered around a straight line. This layer is defined by three parameters, ν_1 , ν_2 , and ℓ , and its boundary touches three data points. This layer has been optimized, that is, its size cannot be further reduced while maintaining consistency with the data set. Instead, the layer in Figure 6.10 only touches two data points. Clearly, this layer is not optimized. The fact that a layer can be further optimized if its boundary does not touch as many data points as there are parameters is quite general, and it holds true for center lines that are polynomials of any degree, $y = \sum_{i=1}^{d-1} \nu_i u^{i-1}$, a result proven in [53]. A data point that touches the boundary corresponds to an active constraint. It can be further shown that such a constraint is of support with probability 1 whenever $\mathbb{P}$ admits a density, a result also proven in [53] (if instead $\mathbb{P}$ has concentrated mass, the same point can be sampled twice so that it is not of support). Hence, the identification of Chebyshev layers is an example of fully supported scenario problems whenever $\mathbb{P}$ admits a density.

6.1.5 ■ Model class adjustment

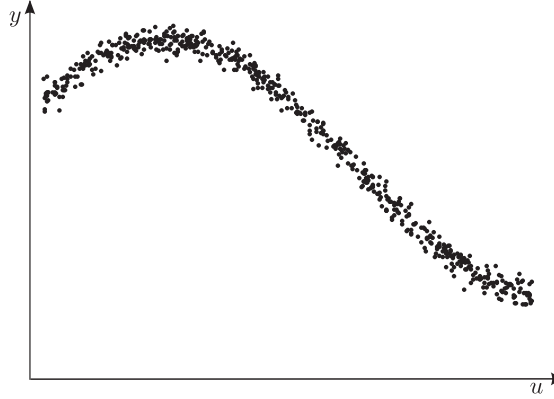


Figure 6.11. *A new set of the data points.*

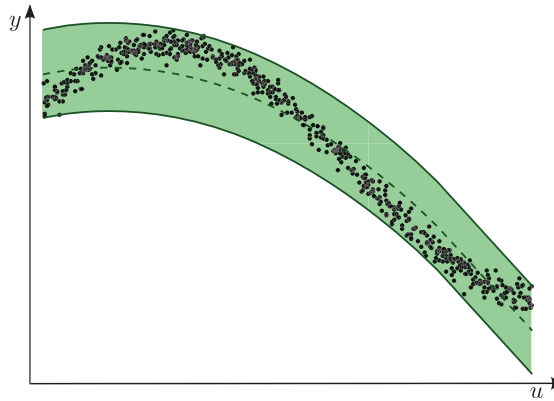


Figure 6.12. *The identified Chebyshev layer for the new data set.*

The same optimization program (6.1) as before was applied to the new set of 620 data points of Figure 6.11, which resulted in the REM of Figure 6.12. Since N and d are the same as before, the reliability result that $\epsilon = 5\%$ is guaranteed with confidence $\beta = 10^{-10}$ also holds in this case. On the other hand, the layer obtained here has a bigger size than the one we found in Section 6.1.3. Moreover, a visual inspection of Figure 6.12 shows that the layer presents wide portions with no data points, which suggests that the layer conforms poorly to the underlying data generation mechanism.¹⁹ To improve the descriptive capabilities of the layer, one can consider increasing the degrees of freedom in the model. Figure 6.13 shows the result obtained by considering a layer centered around a cubic polynomial instead of

¹⁹In problems where u is high dimensional, this visual inspection is not possible; however, an evaluation can be obtained by comparing the width of the layer with the average distance of the points from the layer boundary.

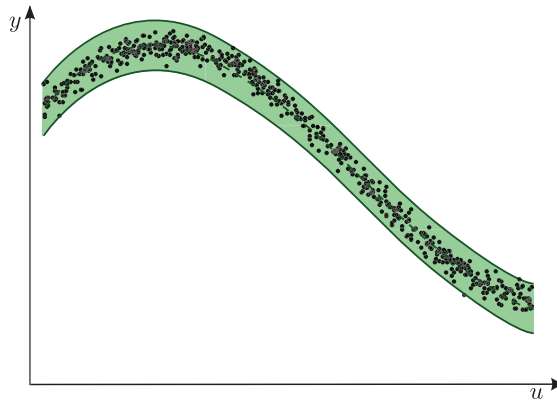


Figure 6.13. *The identified Chebyshev layer centered around a cubic polynomial.*

a quadratic one, which is attained by solving the program

$$\min_{v_1, v_2, v_3, v_4} \left[\max_{i=1, \dots, N} |y_i - [v_1 + v_2 u_i + v_3 u_i^2 + v_4 u_i^3]| \right]. \quad (6.3)$$

The layer size is clearly better here. By Theorem 3.7, with $\beta = 10^{-10}$ as before, the value for ϵ becomes $\epsilon = 5.37\%$ with an increase of 0.37% as compared to the previous construction.

The reader may ask at this point: How many model classes can I try out before I decide which model I prefer? Is it legitimate to select a model from various classes and make the same ϵ, β statement as though the selection were based on just one class? Something looks strange here. To understand this issue, suppose we consider many model classes of Chebyshev layers with the same number of parameters having, however, center lines of different shape; that is, instead of considering the center line $v_1 + v_2 u_i + v_3 u_i^2$, one considers $v_1 + v_2 f_j(u_i) + v_3 g_j(u_i)$, where the functions f_j and g_j are not just monomials—they can be trigonometric functions or whatever—and j runs over a huge number of distinct choices. In the scenario theory, nothing prevents us from considering arbitrary functions, and the theoretical result remains the same for any j . Can we select one layer that has been obtained for a specific j and claim our ϵ, β result? What sounds strange is that if we continue long enough, we shall sooner or later certainly hit a model that fits the data set very well, that is, it has a very thin size. However, this is so because the model is tailored to the data set, not to the data generation mechanism.

Now, a moment's reflection probably reveals to the reader the right answers to all these questions. If it is true that the violation is below ϵ for a model class with probability $1 - \beta$ and the same holds for a second model class, then it is also true that the violation is below ϵ simultaneously for both model classes with probability $1 - 2\beta$. Hence, if we select our preferred model from one of the two classes, the ϵ bound on the violation is not guaranteed as for one class; here β moves up to 2β . Similarly, with p model classes we get $p\beta$, and, if p is very large, we lose our confidence in the result. Still, in the scenario theory, confidence is very “cheap,” which means that a very small β can be selected without increasing N by much (refer, e.g., to Theorem 6.1, where N scales logarithmical in $1/\beta$), and hence very many model classes can be considered before this effect significantly affects the theoretical result.

6.2 ■ Beyond Chebyshev layers

The REM of Section 6.1.3 was obtained by linearly regressing y against a vector containing various powers of u , which gave a center line of a Chebyshev layer that was designed to contain all the points in the data set. Later, in Section 6.1.5 we noticed that more general choices of functions can be considered for constructing the center line. This can all be generalized in various directions:

- (i) The regression vector, say Φ , can be more general than a vector of functions of u . For instance, it can contain more than one variable, e.g., $\Phi = [u_1 \ u_2 \ u_3]^T$, or various powers of these variables, e.g., $\Phi = [u_1 \ u_1^2 \ u_2 \ u_3 \ u_3^2 \ u_3^3]^T$. Generally, Φ can contain any measurable quantity that is correlated with y and is therefore useful to determine its value.
- (ii) A REM can have a structure more complex than that of a Chebyshev layer.
- (iii) Not all of the data points must be contained in the identified REM; not including some data points helps to obtain a REM that has a narrower size.

In this section, we briefly describe these extensions, while the interested reader is referred to [25] for a full account.

Consider the REM

$$R_{c,r,\ell} = \{y : y = \lambda^T \Phi + e, \quad \lambda \in \mathcal{B}_{c,r}, |e| \leq \ell\}, \quad (6.4)$$

where $\mathcal{B}_{c,r}$ is the closed ball with center c and radius r in the Euclidean space $\mathbb{R}^{\dim(\Phi)}$. In (6.4), the parameters that define the REM are c , r , and ℓ , while λ and e are instruments by which the variability of y is described. Once c , r , and ℓ are given, the REM is completely assigned. If $c = \nu$ and $r = 0$, we obtain

$$R_{\nu,0,\ell} = \{y : y = \nu^T \Phi + e, \quad |e| \leq \ell\},$$

where the right-hand side can be rewritten as

$$\{y : |y - \nu^T \Phi| \leq \ell\},$$

in which we recognize a Chebyshev layer. The extra degree of freedom given by r allows one to obtain a REM with variable amplitude. A simple example where $\Phi = u$ is given in Figure 6.14, while Figure 6.15 represents a case where $\Phi = [u \ u^2 \ u^3]^T$.

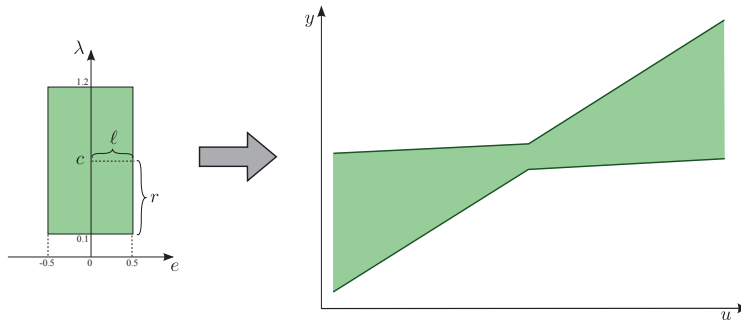


Figure 6.14. The REM $R_{c,r,\ell} = \{y : y = \lambda u + e, \quad \lambda \in \mathcal{B}_{c,r}, |e| \leq \ell\}$, with the domain for λ and e given on the left.

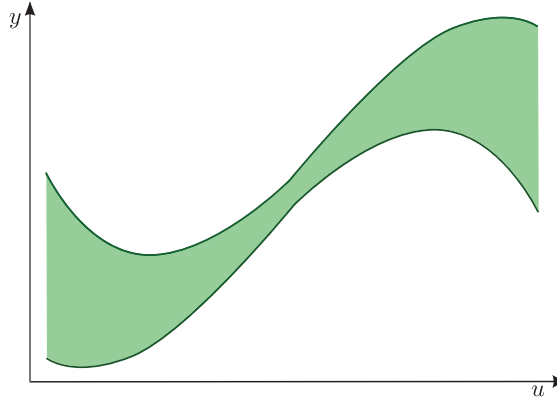


Figure 6.15. The REM $R_{c,r,\ell} = \{y : y = [\lambda_1 u + \lambda_2 u^2 + \lambda_3 u^3] + e, \lambda \in \mathcal{B}_{c,r}, |e| \leq \ell\}$ for $c = (1, 0, -0.7)$, $r = 0.2$, and $\ell = 0.05$.

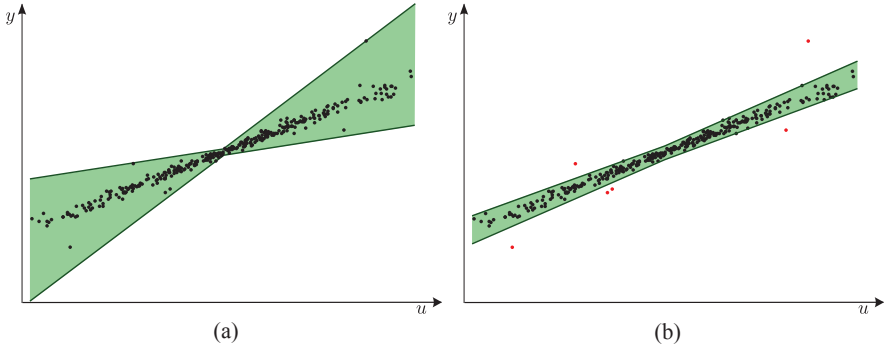


Figure 6.16. Some of the data points may not be included in a REM to improve the REM size. (a) All data points are in the REM. (b) The REM obtained after some data points are left out.

When a REM of the form (6.4) is identified from a data set, one has first to decide which quantity is minimized. This is not obvious, as the width of the model is not of constant size as it is in a Chebyshev layer. If, e.g., one has some prior knowledge about $\mathbb{E}[\|\Phi\|]$, say $\alpha \approx \mathbb{E}[\|\Phi\|]$, a sensible choice can be that of minimizing $\alpha r + \ell$. Indeed, for a given Φ and c , the quantity $\lambda^T \Phi + e$, where $\lambda \in \mathcal{B}_{c,r}$ and $|e| \leq \ell$, is maximized by $\lambda = c + r\Phi/\|\Phi\|$ and $e = \ell$, which gives $\lambda^T \Phi + e = c^T \Phi + r\|\Phi\| + \ell$; the minimum value $\lambda^T \Phi + e = c^T \Phi - r\|\Phi\| - \ell$ is instead achieved for $\lambda = c - r\Phi/\|\Phi\|$ and $e = -\ell$. Hence, as Φ is varied, the average width is $\mathbb{E}[2r\|\Phi\| + 2\ell] = 2(r\mathbb{E}[\|\Phi\|] + \ell)$, which is estimated by $2(r\alpha + \ell)$. By minimizing this quantity under the constraint that the points in the data set are in the REM, one finally arrives at the following linear program (compare with (6.2)):

$$\begin{aligned} & \min_{\ell, c, r} r\alpha + \ell \\ & \text{subject to } \ell \geq 0, r \geq 0, \\ & |y_i - c^T \Phi_i| \leq \ell + r\|\Phi_i\|. \end{aligned}$$

Reference [25] discusses how this setup can be generalized to the case when an ellipsoid is considered in place of the ball $\mathcal{B}_{c,r}$, which leads to considering a semidefinite program.

Figure 6.16 further illustrates a situation where one decides that some of the data points are better left out to improve the model size. Since data points correspond to constraints, the theory of Section 3.3 on “scenario optimization with constraints removal” can be applied in this context. The reader is referred to [25] for a discussion on this approach.

Chapter 7

Application to statistical learning

This chapter focuses on classification problems where the unknown label of an object (like “faulty” or “working,” “sick” or “healthy,” “up” or “down,” etc.) is estimated from accessible variables. It is shown that classifiers carrying a known probability of misclassification can be constructed using methods that come from the scenario approach. After providing a general presentation of the statistical learning approach to classification, we introduce the so-called *guaranteed error machine*, or GEM, which is a prototypical example of a scenario-based algorithm to construct classifiers, and apply it to a case study in the classification of breast tumors.

7.1 ■ An introduction to statistical learning and classification

Classification studies the problem of estimating the *label* y of an *object*. The label is a discrete quantity belonging to a finite set, and an *instance* $x \in \mathbb{R}^p$ is a vector of measured attributes of the object from which y is estimated. In what follows, we consider binary classification, i.e., $y \in \{0, 1\}$. 0 and 1 represent two different classes, whose meaning varies depending on the application at hand and can, e.g., be “sick” or “healthy,” “right” or “wrong,” “male” or “female.” We assume that the value of y is determined by x , that is, there exists a function $y = y(x)$. However, we have no access to the map $y = y(x)$, so, given an x , we have to guess its class label y by means of some *classifier* $\hat{y} = \hat{y}(x)$. A classifier errs on x if $y(x) \neq \hat{y}(x)$, and the goal is to construct classifiers that err as rarely as possible.²⁰

In the *statistical learning* approach to classification, it is assumed that x values occur according to a probability μ , and the reliability of a classifier is measured by $\mu\{x : y(x) \neq \hat{y}(x)\}$, the probability of the set of x values where $y(x)$ and $\hat{y}(x)$ do not agree. $\mu\{x : y(x) \neq \hat{y}(x)\}$ is called the *probability of error* and is indicated by $PE(\hat{y})$.

Given $\hat{y}(\cdot)$, $PE(\hat{y})$ cannot be computed, for its computation would require knowledge of μ and $y(\cdot)$. Hence, selecting a good classifier cannot rely on direct minimization of $PE(\hat{y})$. Instead, we assume that one has access to a database of

²⁰One can consider cases where the value of y is not fully determined by x and, at a given x , y can take value 0 or 1 with given probabilities. However, this situation is only seemingly more general than that where y is determined by x . In fact, it can be traced back to when $y(\cdot)$ is a deterministic function by augmenting x with a new variable x' and letting $y = y(x, x')$, where x' accounts for the other sources of variability in y besides x . In this case, a classifier cannot use x' to classify x , that is, $\hat{y}(x, x')$ is a constant function in the x' direction.

past examples $\mathcal{E}_N = (x_1, y_1), \dots, (x_N, y_N)$, where the x_i 's are independently drawn according to μ and $y_i = y(x_i)$, and is asked to select a classifier $\hat{y}_N(\cdot)$ based on $\mathcal{E}_N$. A classification *algorithm* is a rule that makes this selection. Examples of classification problems arise in diverse disciplines including medicine, hand-written digit recognition, and fault detection, to cite but a few. An example in medicine is that of classifying breast tumors as “malignant” or “benign” based on the analysis of a small quantity of tissue taken from the tumor. This example is presented in Section 7.3. In hand-written digit recognition, the box in which a digit is written is subdivided into small cells, and each cell is scanned to check if the ink has or has not touched it. This generates a 0/1 string whose length is equal to the number of cells. This string is the x variable by which the digit is recognized, and the database from which the classifier is built is a record of examples where a digit is written more times and scanned. See also [93, 45, 94, 39, 81] for general presentations of classification.

7.1.1 ■ The stochastic nature of $PE(\hat{y}_N)$

Given a data generation mechanism, i.e., a pair $(\mu, y(\cdot))$, $PE(\hat{y})$ is a deterministic function that associates a number in $[0, 1]$ to any given classifier $\hat{y}(\cdot)$. If we further replace $\hat{y}(\cdot)$ with $\hat{y}_N(\cdot)$, the classifier obtained from an algorithm, $PE(\hat{y}_N)$ becomes a random variable formed by the composition of $\hat{y}_N(\cdot)$, which depends on the random examples $\mathcal{E}_N$, with the deterministic function $PE(\cdot)$. The dependence of $PE(\hat{y}_N)$ on $\mathcal{E}_N$ expresses the fact that different sets of examples can be more or less effective to form an estimate of $y(\cdot)$ and, when an algorithm is fed with $\mathcal{E}_N$, it returns a classifier $\hat{y}_N(\cdot)$ that more or less closely matches $y(\cdot)$ depending on the seen $\mathcal{E}_N$.

One fact that is important to make explicit is that $PE(\hat{y}_N)$ is a random variable whose probabilistic characteristics, its distribution in particular, depend on the unknown data generation mechanism $(\mu, y(\cdot))$. Thus, the same algorithm can generate classifiers that are likely to be more or less reliable depending on the context in which the algorithm is applied.

7.1.2 ■ Ternary-valued classifiers

Sometimes, ternary-valued classifiers that are allowed to return the label unknown in doubtful cases, so expressing abstention from classifying, are considered. For $\{0, 1, \text{unknown}\}$ -valued classifiers, the definition of probability of error is modified as follows: $PE(\hat{y}_N) = \mu\{x : \hat{y}_N(x) = 0 \text{ or } 1, \text{ and } y(x) \neq \hat{y}_N(x)\}$, that is, $PE(\hat{y}_N)$ is the probability that an answer is issued and this answer is incorrect. Clearly, obtaining a small probability of error cannot be regarded as the sole goal for ternary-valued classifiers, as this is easily achieved by issuing the unknown label most of the time, and hence one would also like that the label unknown is used as sparingly as possible.

An important remark is that no algorithm for $\{0, 1\}$ -valued classifier exists that guarantees high probability of correct classification for all data generation mechanisms $(\mu, y(\cdot))$. While this fact can be formalized and rigorously proven, an intuitive reasoning suffices to clarify this concept: To any given algorithm, a very complex data generation mechanism can be presented where the $y(\cdot)$ function keeps jumping wildly from 0 to 1 in nearby x locations, so that in no way can the algorithm reliably reconstruct the function from N observations. The GEM algorithm introduced in the next section generates $\{0, 1, \text{unknown}\}$ -valued classifiers.

GEM comes accompanied by a reliability result that is grounded in the scenario theory and whose existence is made possible by the option of issuing the unknown label. The reader may notice here an analogy with the REMs studied in Chapter 6 where the reliability guarantees were possible because the predictor had an interval as output: An interval is a multivalued output pretty much like `unknown` can be seen as a multivalued output in classification problems where y can only take value 0 or 1 and both are included if the `unknown` label is issued.

7.2 ■ The algorithm GEM

GEM is an algorithm that has a tunable parameter d that is used to tune the extension of the region where the classifier does provide a classification. A large value of d generates classifiers that are less likely to return the label `unknown`, but these classifiers also misclassify more frequently, whereas smaller values of d correspond to more risk-averse classifiers where the probability of misclassification is reduced at the expense of returning an `unknown` with higher probability. In this section, we first introduce GEM in a very simple example, followed by a general presentation.

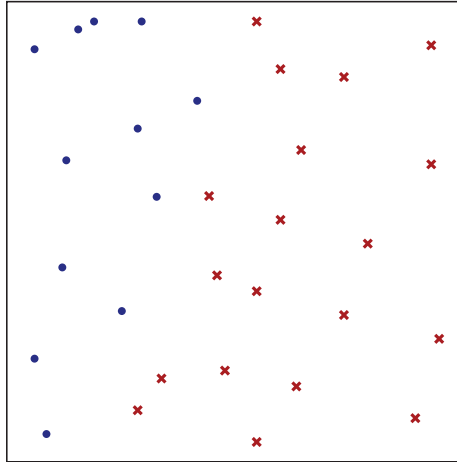


Figure 7.1. *Data set. Red crosses correspond to the label 0 and blue dots to the label 1.*

Suppose that the instances x are bivariate and take value in a square. We are given the data set in Figure 7.1. Take $d = 3$. The construction starts with forming the largest disk whose center coincides with the center of the square and that contains only red points. For the data set at hand, this disk is displayed in Figure 7.2(a). We let $\hat{y}_N(x) = 0$ (red) for all x in this disk, and the examples in the data set that are in the disk are discarded and no longer considered in the rest of the construction. One blue point is found at the boundary of the disk, which we call “base instance.” This is the center of the next disk, which is the largest possible containing only blue points; see Figure 7.2(b). Similarly to the first disk, in this second disk we let $\hat{y}_N(x) = 1$ (blue), and the examples in the disk are discarded. Since $d = 3$, we are allowed a third step in the construction. The new red base instance is determined and a red disk is built. In the end, the classifier in Figure 7.2(c) is obtained, where the region that is not covered by any disk is classified as `unknown`.

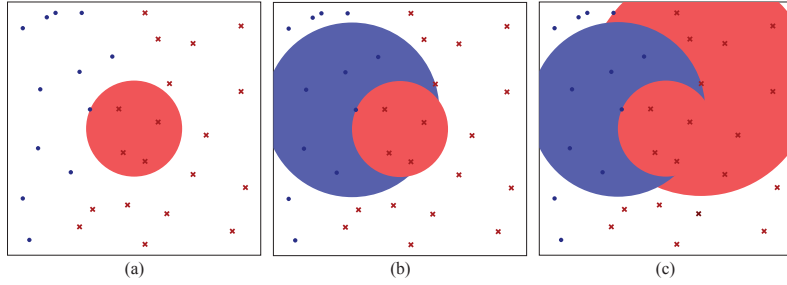


Figure 7.2. *The GEM algorithm. The three figures (a), (b), and (c) show the progress of the algorithm in constructing the classifier.*

Altogether, this construction is not based on a single scenario optimization procedure but rather on three optimizations in cascade, where at each step the radius of a disk is maximized under the constraint that the disk does not include points of the wrong color. Nevertheless, simple reasoning reveals that the argument used in Chapter 5 to prove Theorem 3.7 applies in the present context to establish Theorem 7.1 below with no conceptual modifications, which also shows the versatility of the argument in the proof. While a complete proof of Theorem 7.1 can be found in [24], here we limit ourselves to offering the following hints:

- (i) The probability of error in the present context is the same as the violation probability in Chapter 3, while $\hat{y}_N(\cdot)$ plays the same role of the scenario solution θ^* ;
- (ii) the role of support constraints in Theorem 3.7 is played here by base instances;
- (iii) in Theorem 3.7, given the support constraints the solution does not change if other satisfied constraints are added; here, given the base instances, $\hat{y}_N(\cdot)$ does not change if correctly classified points are added;
- (iv) in Theorem 3.7, adding violated constraints changes the solution; here, adding misclassified points changes $\hat{y}_N(\cdot)$;
- (v) similarly to Theorem 3.7, where the number of support constraints can never be larger than d while it can be less than d , here the number of base instances can never be larger than d by construction while it can be less than d if the whole x domain is filled up by $\hat{y}_N(\cdot)$ before the barrier of the maximum number of d base instances is reached.

The above construction with disks can be generalized to more complex regions leading to the following algorithm (where $x \in \mathbb{R}^p$); see [24] for more details.

THE GEM ALGORITHM

0. Let x_B = a predetermined point in the domain for x and $y(x_B)$ = a predetermined value 0 or 1, $j = 1$, $P = \{1, 2, \dots, N\}$, and $Q = \emptyset$ (empty set).
1. **If** $|Q| \leq d - (p(p+1)/2 + p)$ ($|\cdot|$ means cardinality), go to point 2a;
elseif $d - (p(p+1)/2 + p) < |Q| \leq d - (p+1)$, go to point 2b;
else, go to point 2c.

2a. Solve the following convex optimization problem:

$$\begin{aligned} \min_{A=A^T \in \mathbb{R}^{p \times p}, b \in \mathbb{R}^p} \quad & \text{Trace}(A) \\ \text{subject to} \quad & (x_i - x_B)^T A (x_i - x_B) + b^T (x_i - x_B) \geq 1, \\ & \forall i \in P \text{ such that } y_i \neq y(x_B) \\ & \text{and } A \succeq 0 \text{ (} A \text{ positive semidefinite).} \end{aligned}$$

If more than one pair (A, b) solves the problem, take the pair (A, b) with smallest norm of b , $\|b\| = \sqrt{\sum_{r=1}^p b_r^2}$. If a tie still occurs, break it according to a lexicographic rule on the elements of A and b . Let (A^*, b^*) be the optimal solution.

Go to point 3.

2b. Solve the following convex optimization problem:

$$\begin{aligned} \min_{a \geq 0, b \in \mathbb{R}^p} \quad & a \\ \text{subject to} \quad & a \cdot \|x_i - x_B\|^2 + b^T (x_i - x_B) \geq 1, \\ & \forall i \in P \text{ such that } y_i \neq y(x_B). \end{aligned}$$

If more than one pair (a, b) solves the problem, take the pair (a, b) with smallest norm of b . Let (a^*, b^*) be the optimal solution, and define $A^* = a^* I$ (I = identity matrix).

Go to point 3.

2c. Solve the following convex optimization problem:

$$\begin{aligned} \min_{a \geq 0} \quad & a \\ \text{subject to} \quad & a \cdot \|x_i - x_B\|^2 \geq 1, \quad \forall i \in P \text{ such that } y_i \neq y(x_B). \end{aligned}$$

Let a^* be the optimal solution, and define $A^* = a^* I$ and $b^* = 0$.

Go to point 3.

3. Form the region $\mathcal{R}_j = \{x : (x - x_B)^T A^* (x - x_B) + (b^*)^T (x - x_B) < 1\}$, and let $\ell_j = y(x_B)$.

Update P by removing the indexes of the instances in $\mathcal{R}_j$. If P is empty, go to point 4;

update $Q = Q \cup \{\text{indexes of the active instances}\}$, where the “active” instances are those that fall on the boundary of $\mathcal{R}_j$;

if $|Q| < d$, search for the “active” instance furthest away from x_B (if there is more than one instance at the furthest distance from x_B , take any one of them) and rename it as x_B ; let $j = j + 1$, and go to point 1;

else, go to point 4.

4. Define the classifier

$$\hat{y}_N(x) = \begin{cases} \text{unknown} & \text{if } x \notin \mathcal{R}_r, 1 \leq r \leq j, \\ \ell_q & \text{otherwise, with } q = \min r \text{ such that } x \in \mathcal{R}_r, 1 \leq r \leq j. \end{cases}$$

Points 2a, 2b, and 2c construct regions containing examples all having the same label as the label $y(x_B)$. Point 2a constructs regions more complex than 2b, which are in turn more complex than those in 2c: 2a constructs ellipsoids containing x_B , those in 2b are spheres containing x_B , while those in 2c are spheres having x_B at their center. If P does not become empty, the procedure is halted when the total number of the active instances $|Q|$ reaches the selected bound d , and redirecting the algorithm to simpler constructions when $|Q|$ gets close to d —this is done in point 1—serves the purpose of never exceeding d upon termination of the algorithm.

The following theorem gives the generalization result for GEM; a full proof can be found in [24].

Theorem 7.1. *Suppose that the probability μ has density.²¹ Then, the probability distribution of $PE(\hat{y}_N)$ for the GEM algorithm satisfies the relation*

$$F_{PE}(\epsilon) := \mu^N \{PE(\hat{y}_N) \leq \epsilon\} \geq 1 - \sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}. \quad (7.1)$$

A few remarks on the interpretation and practical use of Theorem 7.1 are in order.

- (i) Like the result for the distribution of the violation of the solution to a scenario optimization program discussed in Chapter 3, the bound for $F_{PE}(\epsilon)$ in the theorem is $B(d, N - d + 1)$, the beta distribution with d and $N - d + 1$ degrees of freedom; this bound does not depend on the data generation mechanism $(\mu, y(\cdot))$ and applies distribution-free.
- (ii) A Monte Carlo test was performed for the $y(\cdot)$ function of Figure 7.3. $N = 200$ examples were extracted $M = 1000$ times. For each multi-extraction of 200

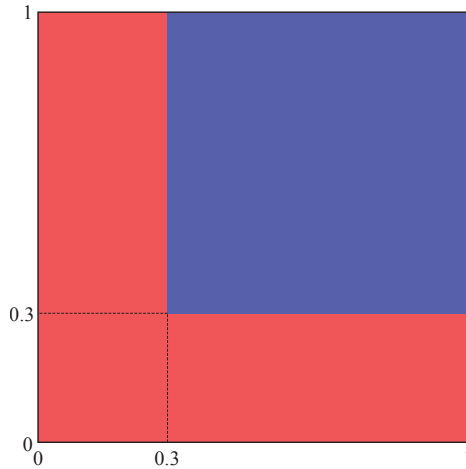


Figure 7.3. Function $y(\cdot)$. The upper square has label 1; μ is uniform over the big square.

²¹This assumption rules out with probability 1 the possibility that the number of active instances at each construction 2a, 2b, and 2c of the GEM algorithm exceeds the number of optimization variables in the corresponding optimization program.

examples, the classifier $\hat{y}_{200}(\cdot)$ was constructed with the GEM algorithm with $d = 5$ and the corresponding $PE(\hat{y}_{200})$ was then computed. Note that this computation is possible here due to the artificial nature of the problem so that $(\mu, y(\cdot))$ is known. The histogram obtained from the $M = 1000$ trials is shown in Figure 7.4 against the density of the distribution $B(5, 196)$ given in the right-hand side of (7.1) (in this example, $d = 5$ and $N - d + 1 = 196$).

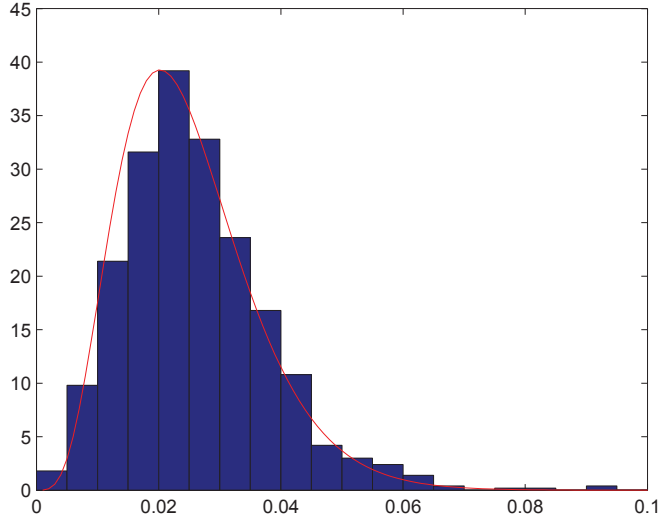


Figure 7.4. Histogram of $PE(\hat{y}_{200})$.

- (iii) In [24], GEM is compared with other popular classification methods like NNC (nearest-neighbor classifier), SVM (support vector machine), and SCM (set covering machine), showing that GEM has a performance comparable with these methods. On the other hand, GEM comes with two important advantages: (a) Due to the presence of the *unknown* label, a chance of abstention can be traded in favor of reliability, and the parameter d is the tuning knob in this process; (b) perhaps more importantly, the prediction error is in GEM *a priori* kept under control by a rigorous theory.
- (iv) A typical use of Theorem 7.1 is that one selects a risk level ϵ and chooses the largest d such that $PE(\hat{y}_N) \leq \epsilon$ holds with high confidence $1 - \beta$ by letting $\sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1 - \epsilon)^{N-i} \leq \beta$. The fact that very small values for β can be selected (since the beta distribution is thin-tailed) means that $PE(\hat{y}_N) \leq \epsilon$ can be enforced with such a high confidence that the complement event that $PE(\hat{y}_N) > \epsilon$ loses any practical relevance. Finding the largest value of d can be challenging. However, one can proceed according to a bisection procedure: Tentative values of d are inserted in the expression $\sum_{i=0}^{d-1} \binom{N}{i} \epsilon^i (1 - \epsilon)^{N-i}$ and compared with β to decide which portion of d values has to be further explored. For a quick evaluation (which, however, underestimates the largest value of d compatible with the reliability constraint), one can also resort to the corollary below which provides an explicit expression for d that is easily obtained from equation (3.10) in Section 3.2.1.

Corollary 7.2. *Under the assumptions in Theorem 7.1, given $\epsilon, \beta \in (0, 1)$ we have that $\mu^N \{PE(\hat{y}_N) \leq \epsilon\}$ holds with probability $1 - \beta$ provided that*

$$d \leq 1 + \frac{N\epsilon}{2\ln 2} + \frac{\ln \beta}{\ln 2}. \quad (7.2)$$

For, e.g., $N = 1000$, $\epsilon = 5\%$, and $\beta = 10^{-6}$, (7.2) gives $d = 17$, while the actual largest d that is obtained by bisection is 21.

A second use of the beta distribution $B(d, N - d + 1)$ is to tune d so that the total probability of seeing N examples and that the next $(N + 1)$ -th example is misclassified is below a fixed threshold. As we know from Section 3.2.2 of Chapter 3, this probability is given by the expected value of $PE(\hat{y}_N)$, which is in turn bounded by the mean of the beta distribution, that is,

$$\mathbb{E}[PE(\hat{y}_N)] \leq \frac{d}{N + 1}. \quad (7.3)$$

Thus, with, e.g., $N = 999$ examples one can be 5% confident that the 1000th example will not be misclassified if $d = 50$.

7.3 ■ Application to breast tumor diagnosis

Traditionally, the diagnosis of breast tumors has been performed by a full biopsy, an invasive surgical procedure. To alleviate the disruption involved in the biopsy, in the past 30 years or so a technique called fine-needle aspiration (FNA) has been introduced where a small amount of tissue is aspirated from the tumor, analyzed at the microscope and digitized to finally extract various features of the tumoral cells like nuclear size, shape, and texture. These features are then used as input to a model that returns a label “malignant” or “benign.” Reportedly, however, diagnoses based on FNA are not certain, and FNA is only used as a support to diagnosis, while in doubtful cases a full biopsy remains a necessity [51, 54].

To construct the model, various machine learning techniques have been considered; see, e.g., [85]. To this purpose, a sample of women are given both an FNA and a full biopsy, so that each woman is described by a set of tumoral features and a label, benign or malignant, obtained from the biopsy. This sample is used to train the learning machine from which the model is obtained. In a future clinical case, a woman is given a FNA, and the model output corresponding to the woman’s FNA tumoral features is used to evaluate the nature of the tumor.

Here we apply the GEM algorithm to a data set of 683 cases (239 malignant and 444 benign) taken from the *UCI Machine Learning Repository* [49], with nine tumoral features, namely, *clump thickness*, *uniformity of cell size*, *uniformity of cell shape*, *marginal adhesion*, *single epithelial cell size*, *bare nuclei*, *bland chromatin*, *normal nucleoli*, and *mitoses*, that form the x vector.

Table 7.1 gives the empirical results. In the table, $\#(\text{errors})$ is the number of cases that are incorrectly classified in a 10-fold cross validation. Precisely, a window containing a number of examples equal to 68, the integer part of the total number of examples divided by 10, is left out, and the examples in the window are used as tests; the window is then shifted so that an adjacent window is used as a test window and so on until all examples but the last three have played the role of tests. Each time, the number of errors in the test set are counted, and $\#(\text{errors})$ is the sum of all of these

Table 7.1. Results in the classification of breast tumors.

d	5	10	15	20	25	30	35
#(errors)	5 (0.74%)	12 (1.76%)	20 (2.94%)	25 (3.68%)	26 (3.82%)	29 (4.26%)	31 (4.56%)
#(unknowns)	378	347	148	106	49	16	0
#(correct)	297	321	512	549	605	635	649
$d/(N+1)$	0.81%	1.63%	2.44%	3.25%	4.07%	4.88%	5.69%

errors in the 10 test sets. In the same row, in parentheses, the value $\frac{\#(\text{errors})}{\#(\text{test examples})}$ ($\#(\text{test examples}) = 680$) is shown; $\#(\text{unknowns})$ and $\#(\text{correct})$ are obtained similarly to $\#(\text{errors})$ summing the number of unknowns and of correctly classified examples in the 10 test sets; the last row displays the bound $\frac{d}{N+1}$ for $\mathbb{E}[PE(\hat{y}_N)]$ given in equation (7.3), where $N = 615$ is the number of training examples (total number of examples, 683, minus the examples in the test set, 68). This theoretical value should be compared with the number in parentheses in the second row. The value in the last row is theoretically guaranteed. When the empirical ratio exceeds the theoretical value, this is because the empirical ratio is an imprecise, empirical estimate of the real value of $\mathbb{E}[PE(\hat{y}_N)]$. Importantly, the theoretical value in the last row is obtained without resorting to any *a priori* knowledge on the distribution of the population.

Chapter 8

Miscellanea

This chapter contains a miscellanea of additional results. Some have been investigated in depth and are by now well developed. Others are the subject of current research, and the facts known at the time this book is being compiled are only partial. All are only sketched, and the reader is referred to the literature for a deeper presentation of these topics. By including this chapter in this book, our intention was to offer a broader view on scenario optimization and to stimulate research done by others in open directions.

8.1 ■ Probability box

Consider the two instances of scenario programs of the type (1.4) depicted in Figure 8.1. These two programs are based on the same number of scenarios, and they have the same optimal value ℓ^* . Hence, by applying the scenario theory, one attains the same guarantees for the two programs. Which one do we prefer? They show an evident difference: In the program on the left, most of the sampled cost functions have a value close to ℓ^* when $\nu = \nu^*$; this is not so for the program on the right, and

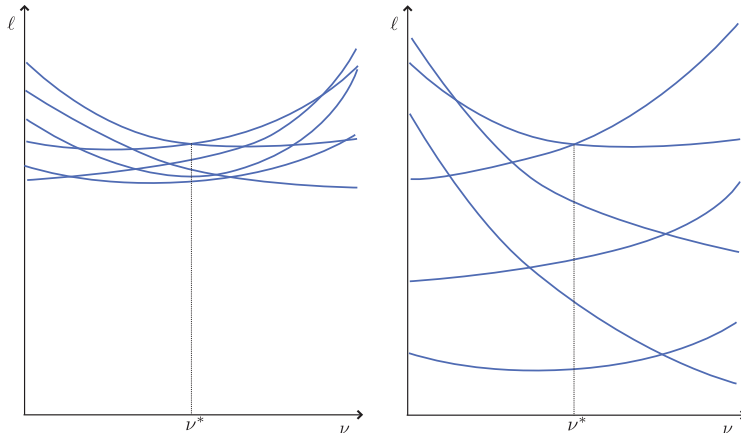


Figure 8.1. Two instances of scenario programs $\min_{\nu \in \mathbb{R}^{d-1}} [\max_{i=1, \dots, N} \ell(\nu, \delta_i)]$.

the values are much more scattered. Therefore, in this second program one expects that a fair chance exists that the performance will significantly outdo ℓ^* in a future case, which can hardly happen for the problem on the left.

From the discussion above, it appears that the cost functions that do not concur to determining the solution in a scenario program still carry useful information on the performance the scenario solution achieves in new situations. Can this information be included in the scenario theory? This issue has been studied in [34], and here we give a brief account of the main result.

Start with the following definitions.

Definition 8.1 (empirical costs). Consider the cost values $\ell(\nu^*, \delta_i)$, $i = 1, \dots, N$, achieved by the solution ν^* when it is applied to the scenarios δ_i , and sort them in decreasing order: $\ell_1^* \geq \ell_2^* \geq \dots \geq \ell_N^*$. Values ℓ_k^* are called the empirical costs.

Definition 8.2 (risk). For any given $\nu \in \mathbb{R}^{d-1}$ and $\ell \in \mathbb{R}$, the risk associated with (ν, ℓ) is $R(\nu, \ell) = \mathbb{P}\{\delta \in \Delta : \ell(\nu, \delta) > \ell\}$. The risk of the k th empirical cost ℓ_k^* is defined as $R_k = R(\nu^*, \ell_k^*)$.

The following generalized nondegeneracy assumption is made.

Definition 8.3 (nondegeneracy). For every $N \geq d$, it holds with probability 1 that

$$\ell_d^* \neq \ell_{d+1}^* \neq \dots \neq \ell_N^*.$$

For fully supported problems (see Definition 5.4), $\ell^* = \ell_d^*$ with probability 1. Instead, $\ell^* > \ell_d^*$ happens with nonzero probability for non-fully supported problems. In any case, $\ell^* \geq \ell_d^*$ holds with probability 1. We know from Theorem 3.7 that the distribution of $R(\nu^*, \ell^*)$ is dominated by a beta distribution $B(d, N - d + 1)$. We next present Theorem 8.4, taken from [34], which generalizes this result. Indeed, as a corollary of Theorem 8.4 we have that R_d has distribution $B(d, N - d + 1)$, from which we recover Theorem 3.7 since the inequality $\ell^* \geq \ell_d^*$ implies that the violation of (ν^*, ℓ^*) is less than or equal to that of (ν^*, ℓ_d^*) .

Theorem 8.4. The joint probability distribution function of $R_d, \dots, R_N$ is an ordered Dirichlet distribution with parameters $(d, \underbrace{1, 1, \dots, 1}_{N-d})$, i.e.,

$$\begin{aligned} & \mathbb{P}^N \{R_d \leq \epsilon_d, R_{d+1} \leq \epsilon_{d+1}, \dots, R_N \leq \epsilon_N\} \\ &= \frac{N!}{(d-1)!} \int_0^{\epsilon_d} \alpha_d^{d-1} \int_0^{\epsilon_{d+1}} \dots \int_0^{\epsilon_N} \mathbf{1}_{\{0 \leq \alpha_d \leq \dots \leq \alpha_N \leq 1\}} d\alpha_N \dots d\alpha_{d+1} d\alpha_d. \end{aligned}$$

The marginals of Dirichlet distributions are beta distributions, and this is how the result that R_d has distribution $B(d, N - d + 1)$ is obtained in [34].

The main use of Theorem 8.4 is that it is employed to construct a region to which the whole cumulative distribution function of $\ell(\nu^*, \delta)$ belongs. While the reader is referred to [34] for the details, suffice here to say that one can remove the left and

²²Only empirical costs from ℓ_d^* onward are considered because some of the other costs have the same value by construction; see Figure 8.1.

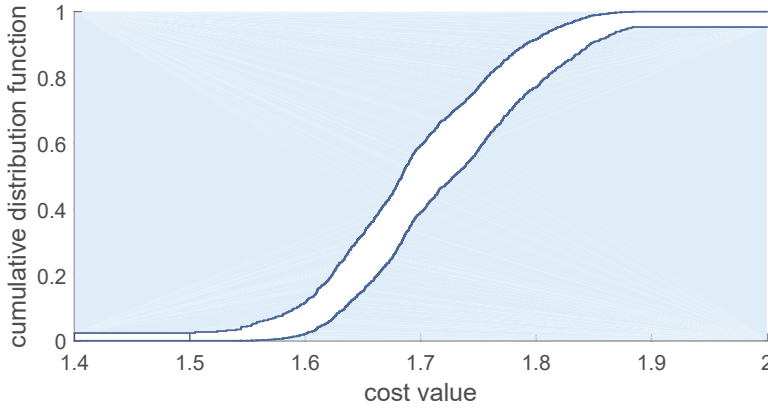


Figure 8.2. A “probability box” for the cumulative distribution function of $\ell(v^*, \delta)$. With confidence $1 - \beta$, the whole cumulative distribution function of $\ell(v^*, \delta)$ lies in the white area.

right tails of the distributions of $R_d, \dots, R_N$ to obtain that the cumulative distribution function of $\ell(v^*, \delta)$ at ℓ_k^* is near $1 - \frac{k}{N+1}$ for any k with high confidence. Based on this fact, a probability box like the one of Figure 8.2 is obtained. This box contains with high confidence the cumulative distribution function of $\ell(v^*, \delta)$, and offers important information on how the cost distributes when v^* is used.

8.2 ■ L_1 -regularization

The number N of scenarios needed to secure a desired robustness level ϵ increases with the number d of optimization variables and, in practice, may result in too many scenarios in applications where the number of variables is large. This difficulty can be alleviated by *regularization*, and here we focus on L_1 -regularization. L_1 -regularization is employed to reduce the effective number of optimization variables, and this reduction allows one to find robust solutions with substantially fewer samples. See, e.g., [87, 90, 47, 46, 32, 96] for discussions on L_1 -regularization.

A scenario program of the type (1.4) with L_1 -regularization is written as

$$\min_{v \in \mathbb{R}^{d-1}} \left[\max_{i=1, \dots, N} \ell(v, \delta_i) \right] \quad (8.1)$$

$$\text{subject to } \|Av - b\|_1 \leq r, \quad (8.2)$$

where A is a $(d-1) \times (d-1)$ matrix, b is a vector of dimension $d-1$, $\|\cdot\|_1$ is the 1-norm ($\|z\|_1 = \sum_j |z_j|$ where z_j are the components of z), and $r \in \mathbb{R}$ is a “constraining parameter.” Depending on the choice of A and b , (8.2) comprises “lasso constraint,” $\|v\|_1 \leq r$ (obtained for $A = I$ and $b = 0$; see Figure 8.3(a)), and “basalt column constraint,”

$$\left\| \begin{bmatrix} v_1 - v_2 \\ v_2 - v_3 \\ \vdots \\ v_{d-1} - v_1 \end{bmatrix} \right\|_1 \leq r \quad (8.3)$$

(which is obtained with the choices

$$A = \begin{bmatrix} 1 & -1 & 0 & 0 & \cdots \\ 0 & 1 & -1 & 0 & \cdots \\ & & \vdots & & \\ \cdots & 0 & 0 & 1 & -1 \\ -1 & 0 & \cdots & 0 & 1 \end{bmatrix},$$

and $b = 0$; see Figure 8.3 (b)). When lasso constraint is used, if the constraint binds the feasibility domain so that the solution for the unconstrained problem is excluded, the constrained solution has a tendency to move towards edges and vertices (*sparsity*). Likewise, in basalt column constraint, the solution has a tendency to set to zero the difference variables $v_j - v_{j+1}$ (and $v_{d-1} - v_1$) appearing in the constraint (8.3), that is, to favor solutions with equal adjacent components. This feature can be useful in various applications, such as the optimization of piecewise constant functions or signals where one wishes to moderate the number of jumps [69, 68].

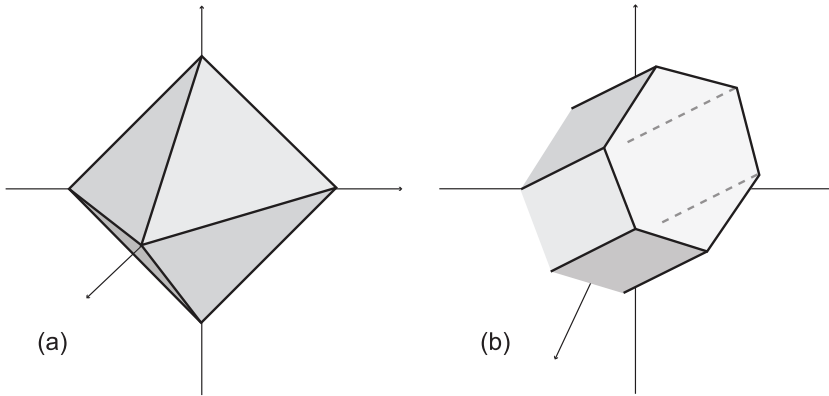


Figure 8.3. (a) *The lasso constraint.* (b) *The basalt column constraint.*

Paper [26] presents an algorithm where parameter r is modulated until the solution happens to belong to a q -dimensional subspace determined by setting to zero all components of $Av - b$ but q of them. Here parameter q is a user-chosen “complexity barrier” that is selected before running the scenario algorithm. Hence, regularization is used to slim down the complexity of the solution, and the algorithm selects variables, or combinations of them, that have a significant impact on the optimization problem. Once the q -dimensional subspace has been found, the algorithm proceeds to solve a standard unconstrained scenario optimization program confined to the selected subspace only. The generalization result that holds in this context is that (compare with (3.4))

$$\mathbb{P}^N \{V(v^*, \ell^*) > \epsilon\} \leq \binom{d-1}{d-1-q} \sum_{i=0}^q \binom{N}{i} \epsilon^i (1-\epsilon)^{N-i}, \quad (8.4)$$

where the upper limit q in the summation is the equivalent of $d-1$ in formula (3.4) (keep in mind that v confined to a q -dimensional subspace, which, augmented with ℓ , gives a total number of $q+1$ variables) and the binomial in front of the

summation accounts for all of the potential choices of the q -dimensional subspace. Since the summation factor goes to zero very fast as N increases, the summation kills the binomial and formula (8.4) proves useful in many applications; see again [26] for examples.

8.3 ■ Fast algorithm for the scenario approach

In the preceding section, we saw that L_1 -regularization can be used to reduce the number of scenarios that are needed to achieve a given level of robustness. An alternative approach towards the same goal consists in slightly moving the scenario solution away from the constraints into the feasibility domain of the scenario program. As the solution gains a margin of feasibility, it is expected to have improved violation properties. However, we must report that attempts to establish theoretical results in this direction have so far met with mixed success.

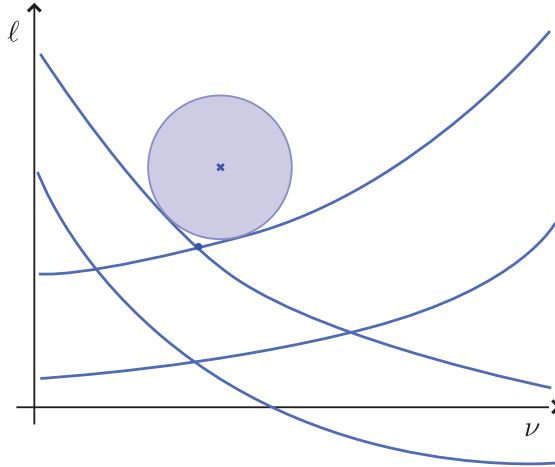


Figure 8.4. A ball-solution as compared to a point-solution. The center of the ball-solution (cross) is expected to have improved violation properties with respect to the point-solution (dot) since it keeps a margin from the constraints.

One first attempt is as follows. Referring to problem (1.4), instead of searching for a minimizer that just satisfies the constraints, one can think of seeking a minimizer whose distance from the constraints is not less than a given margin M . This is visualized in Figure 8.4. One can think of this approach as that of throwing a ball of radius M into the feasibility domain and waiting until the ball gets stuck at the bottom of the domain. The center of the ball in the final position is the solution. Now, a ball centered in a point (ν, ℓ) with radius M is all in Θ_δ if $(\nu, \ell) + r \in \Theta_\delta$, $\forall r : \|r\| \leq M$, which is the same as $(\nu, \ell) \in \cap_{r: \|r\| \leq M} (\Theta_\delta - r)$. Since the intersection of convex sets is convex, we can define $\Theta'_\delta = \cap_{r: \|r\| \leq M} (\Theta_\delta - r)$ to see that the constraint with margin becomes a standard convex constraint expressed as $(\nu, \ell) \in \Theta'_\delta$. Hence, the standard scenario theory can be applied to establish violation limits for the constraint $(\nu, \ell) \in \Theta'_\delta$ in relation to the solution of the problem with margin (the solution marked with a cross in Figure 8.4) and, since the number of optimization variables and scenarios for this problem is the same as in the original problem with no margin, the guarantees coincide with those that are valid for the solution of the approach without margin (marked with a dot in Figure 8.4) to satisfy the constraint

$(\nu, \ell) \in \Theta_\delta$. On the other hand, $(\nu, \ell) \notin \Theta'_\delta$ means that not all of the ball is in the feasibility domain of Θ_δ , but when this happens, the center of the ball can still be a feasible point for Θ_δ . This way, one establishes that the center of the ball has violation guarantees for the initial constraint $(\nu, \ell) \notin \Theta_\delta$ that are at least as good as the violation guarantees of a standard scenario solution. Yet any attempt to show that it is strictly better has so far failed, and establishing an improved violation result for the center of the ball remains an open problem. A final remark, which may matter to someone who wants to undertake research in this direction, is that setting the value of M in advance (as we have done here) independently of the problem makes little sense because any given problem can be resized arbitrarily by a change of variables. Therefore, M is better selected based on particularities of the problem at hand, for example, one may want to evaluate the distance that separates constraints before setting the value of M .

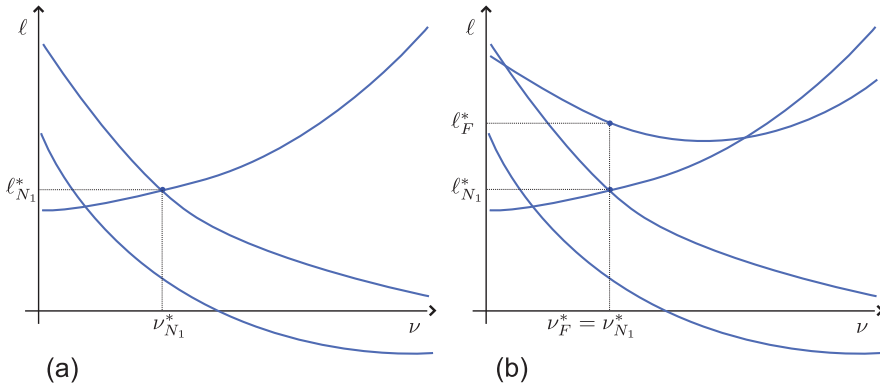


Figure 8.5. Illustration of FAST. (a) Optimization with a moderate number N_1 of scenarios. (b) Detuning step used to boost the violation guarantees.

A second, alternative approach has been developed in paper [33] leading to an algorithm called FAST. Admittedly, however, this approach does not have the same beauty as the first attempt that we have sketched above. In FAST, one first considers a moderate number N_1 of scenarios δ_i and solves a scenario program with $N = N_1$, so generating a decision $\nu_{N_1}^*$ and an optimal value $\ell_{N_1}^*$; refer to Figure 8.5(a). Here, however, $\ell(\nu_{N_1}^*, \delta) \leq \ell_{N_1}^*$ is not guaranteed with the desired probability $1 - \epsilon$ since N_1 is too low for this to hold. Then, a *detuning step* is carried out where N_2 additional scenarios are sampled and the smallest value ℓ_F^* such that $\ell(\nu_{N_1}^*, \delta_i) \leq \ell_F^*$, $i = 1, \dots, N_1 + N_2$, is computed; see Figure 8.5(b). The outcome of FAST is $\nu_F^* = \nu_{N_1}^*$ and the value ℓ_F^* . In [33], the following theorem is proven. The theorem can be used to devise guarantees on the risk of the returned solution to exceed the cost ℓ_F^* .

Theorem 8.5. The violation of (ν_F^*, ℓ_F^*) satisfies relation

$$\mathbb{P}^{N_1+N_2} \{V(\nu_F^*, \ell_F^*) > \epsilon\} \leq (1 - \epsilon)^{N_2} \cdot \sum_{i=0}^{d-1} \binom{N_1}{i} \epsilon^i (1 - \epsilon)^{N_1-i}. \quad (8.5)$$

FAST has pros and cons, as described in the following.

Pros: Reduced sample size requirements. FAST provides a cheaper way (in terms of sample complexity) to find solutions to medium- and large-scale problems. In practice, one normally chooses $N_1 = Kd$, where K is a user-selected number typically set to 20; then, a simple lower bound to N_2 so that $\mathbb{P}^{N_1+N_2}\{V(v_F^*, \ell_F^*) > \epsilon\} \leq \beta$ is $\frac{1}{\epsilon} \ln \frac{1}{\beta}$, which is obtained with elementary computations from (8.5) after neglecting the term $\sum_{i=0}^{d-1} \binom{N_1}{i} \epsilon^i (1-\epsilon)^{N_1-i}$. Hence, a simple formula to estimate the overall number of scenarios needed with FAST is

$$Kd + \frac{1}{\epsilon} \ln \frac{1}{\beta}.$$

A comparison with the formula

$$\frac{2}{\epsilon} \left(\ln \frac{1}{\beta} + d - 1 \right)$$

given in Theorem 1.1 for the classical scenario approach shows that with FAST, the multiplicative dependence of the required number of scenarios on $\frac{1}{\epsilon}$ and d , which is critical when d is medium/large, is replaced by a computationally advantageous additive dependence on $\frac{1}{\epsilon}$ and d .

Cons: Suboptimality of FAST. Figure 8.5(b) represents the solution obtained using FAST. In the figure, $\ell_{N_1}^*$ is the cost value for the problem with N_1 scenarios, and ℓ_F^* is the cost value after the introduction of N_2 extra scenarios in the detuning step. An inspection of the figure reveals that the feasibility region of all constraints contains a part that outperforms ℓ_F^* . Although in the classical approach additional scenarios beyond $N_1 + N_2$ must be introduced to achieve the same level of violation as FAST, so that the number of scenarios that are used is $N \geq N_1 + N_2$, still the solution of the classical approach can outperform, and often does, ℓ_F^* . On the other hand, it is interesting to note that this suboptimality can be estimated. Letting ℓ_N^* be the cost value of the classical scenario approach obtained with the same scenarios as for FAST plus additional scenarios, it certainly holds that $\ell_F^* - \ell_N^* < \ell_F^* - \ell_{N_1}^*$. Consequently, the maximum possible suboptimality of FAST compared with the classical scenario approach is given by $\ell_F^* - \ell_{N_1}^*$.

8.4 ■ Wait-and-judge

As was discussed in Chapter 5, the main result (3.4) of the scenario approach is obtained by recognizing that the violation of the solution of fully supported problems (Definition 5.4) has the worst possible probability distribution, which coincides with 1 minus the right-hand side of (3.4). However, optimization problems encountered in applications are often not fully supported. In fact, in scenario programs with many variables, it is not rare that way fewer support constraints are found than there are optimization variables. When a problem is not fully supported, one can object against applying the bound in (3.4), which is tight only for fully supported problems, and wish to incorporate in the evaluation of the reliability that less than d support constraints have been seen.²³

²³If it is *a priori* known that the number of support constraints is always less than a given value $h < d$, then a theory akin to that in Chapter 3 with h in place of d can be applied and recent works studying specific problem structures for which it holds that $h < d$ are [80, 98]. However, in general, the number of support constraints is a random variable, and *a priori* judgements are bound to be conservative.

In Theorem 3.7, ϵ is a deterministic constant, set in advance prior to seeing any δ_i . The new perspective that we introduce here is that ϵ becomes a function of the number of support constraints that have been found in the instance of the scenario program at hand. To this aim, let $\epsilon(k)$ be a function that takes value in $[0, 1]$, where k is an integer ranging over $\{0, 1, \dots, d\}$. After computing the solution θ^* , one also evaluates the number of support constraints s^* and makes the statement that $V(\theta^*) \leq \epsilon(s^*)$. In this way, the bound on the violation is *a posteriori* determined, and it is adjusted to the number of support constraints.

This setup is studied in paper [29]. The following nondegeneracy assumption on program (3.1) is a key element in the analysis.

Assumption 8.6 (nondegeneracy). *For every N , it holds with probability 1 that the solution with all constraints in place coincides with the solution where only the support constraints are kept.*

Under this assumption, in [29] the following result is proven:

$$\mathbb{P}^N \{V(\theta^*) > \epsilon(s^*)\} \leq \gamma^*, \quad (8.6)$$

where γ^* is obtained from the following variational problem:

$$\begin{aligned} \gamma^* &= \inf_{\xi(\cdot) \in \mathcal{C}^d[0,1]} \xi(1) \\ \text{subject to } & \frac{1}{k!} \frac{d^k}{dt^k} \xi(t) \geq \binom{N}{k} t^{N-k} \cdot \mathbf{1}_{\{t \in [0, 1-\epsilon(k)]\}}, \quad t \in [0, 1], \\ & k = 0, 1, \dots, d, \end{aligned} \quad (8.7)$$

where $\mathbf{1}_A$ indicates the indicator function of set A ($\mathbf{1}_A = 1$ over A and $\mathbf{1}_A = 0$ otherwise), $\mathcal{C}^d[0, 1]$ is the class of d times differentiable functions with continuous d th derivative over the interval $[0, 1]$, and $\frac{d^k}{dt^k}$ with $k = 0$ means that no derivative operator is applied. See [29] for a discussion on the variational problem (8.7) and for approximate formulas to calculate its solution.

One is interested in making γ^* very small, for example, $\gamma^* = 10^{-6}$, which means that $V(\theta^*) \leq \epsilon(s^*)$ holds with very high confidence $1 - \gamma^*$. When $\epsilon(k)$ is chosen to be constant, $\epsilon(k) = \epsilon$ for all k , the γ^* given by (8.7) turns out to be coincident with the right-hand side of (3.4), and Theorem 3.7 is therefore recovered as a particular case of result (8.6), a fact proven in [29, Corollary 1]. On the other hand, function $\epsilon(k)$ can be selected so that it largely improves over the constant ϵ , i.e., $\epsilon(k)$ is significantly smaller than ϵ for most values of k while keeping γ^* equal to the right-hand side of (3.4). Hence, the flexibility introduced by adjusting ϵ to k often permits one to formulate stronger conclusions on the violation of θ^* , and we give an example in the next subsection.

Before that, we make a remark that we believe can be of interest at a philosophical level. Note that the left-hand side of equation (8.6) can also be written as

$$\begin{aligned} \mathbb{P}^N \{V(\theta^*) > \epsilon(s^*)\} &= \mathbb{P}^N \left(\bigcup_{k=0}^d \{V(\theta^*) > \epsilon(k) \text{ and } s^* = k\} \right) \\ &= \sum_{k=0}^d \mathbb{P}^N \{V(\theta^*) > \epsilon(k) \text{ and } s^* = k\}. \end{aligned} \quad (8.8)$$

To proceed, one can think of bounding each term in the summation in (8.8) with $\sum_{i=0}^{k-1} \binom{N}{i} \epsilon(k)^i (1 - \epsilon(k))^{N-i}$ (which is obtained from (3.4)) with k in place of d , i.e., as though the problem was in k variables, and $\epsilon(k)$ in place of ϵ , and write

$$\mathbb{P}^N \{V(\theta^*) > \epsilon(s^*)\} \leq \sum_{k=0}^d \sum_{i=0}^{k-1} \binom{N}{i} \epsilon(k)^i (1 - \epsilon(k))^{N-i}. \quad (8.9)$$

However, as shown in Appendix A of paper [29], equation (8.9) is incorrect. This is the sign of a deep fact: *a posteriori* observing k in dimension d is not the same as working in dimension k , or, said differently, simple solutions (supported by k constraints) to complex problems (in dimension $d > k$) are not as guaranteed as solutions to simple problems (in dimension k). While this is a result that carries an important qualitative significance with philosophical implications, still the quantitative gap between the guarantees that can be given after seeing k in dimension d as compared to having k variables is minor, a fact discussed in detail in paper [29], and function $\epsilon(k)$ can in fact be chosen so that it is close for all k to the result that is valid in dimension k .

8.4.1 ■ An example

$N = 1000$ points $p_i \in \mathbb{R}^{100}$ are independently sampled from an unknown probability density, and one constructs the hyper-sphere S of smallest volume that contains all the points. This is obtained by solving the following program, where c is the center and r is the radius of S :

$$\begin{aligned} & \min_{q \in \mathbb{R}^{100}, r \in \mathbb{R}} r \\ & \text{subject to } \|p_i - c\| \leq r, \quad i = 1, \dots, N. \end{aligned} \quad (8.10)$$

We want to provide estimates on the probabilistic mass contained in the hyper-sphere S , or, which is the same, on the probability that one next point sampled independently of the initial set of 1000 points is in S .

In this problem, we identify a point p in $\mathbb{R}^{100}$ with the uncertainty parameter δ , the p_i 's are the scenarios δ_i 's, and program (8.10) is a scenario program of the type (3.1). A new point p falls outside the hyper-sphere constructed by resolving (8.10) if $\|p - c^*\| > r^*$, where (c^*, r^*) is the solution of (8.10). The probability for this to happen is the violation $V(c^*, r^*)$ of the solution (c^*, r^*) .

The optimization variables are the radius r and the 100 coordinates defining the center c , which yields $d = 101$ and $N = 1000$. With these values, an application of the result in (8.6) gives that $V(c^*, r^*) \leq \epsilon(s^*)$ holds with high confidence $1 - 10^{-6}$ with the function $\epsilon(k)$ that is profiled in Figure 8.6. Upon resolving program (8.10), we found 28 support constraints. This is the number of points p_i that are on the surface of S . Since $\epsilon(28) = 6.71\%$, we conclude that the probabilistic volume outside the hyper-sphere does not exceed 6.71%.

Some remarks are in order.

- (i) Figure 8.6 profiles a function $\epsilon(k)$ that satisfies (8.6) for $N = 1000$, $d = 101$, and $\gamma^* = 10^{-6}$. Using instead equation (3.4) with the same values for N , d , and $\beta = \gamma^*$, one finds $\epsilon = 15.17\%$, which is also profiled in Figure 8.6 for the sake of comparison. As it appears, resorting to (3.4) in the case at hand leads to a weaker conclusion by a factor more than 2.

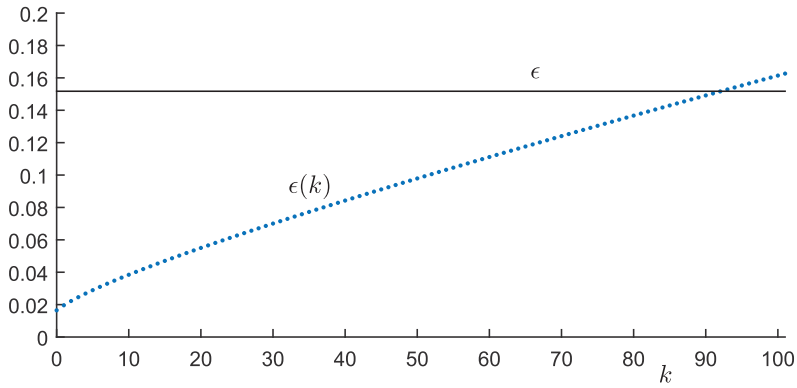


Figure 8.6. Function $\epsilon(k)$ and ϵ .

- (ii) Like Theorem 3.7, the use of the result in (8.6) does not require any knowledge of the distribution according to which the points in $\mathbb{R}^{100}$ are sampled.
- (iii) In our example, which is by simulation, some post-experiment analysis is possible because we actually generated the points and their distribution is therefore known. This analysis highlights some important features of the method. The points were generated from a Gaussian distribution with zero mean and identity covariance matrix. The probabilistic mass outside the hyper-sphere found by solving (8.10) was 3.67%, below the value $\epsilon(28) = 6.71\%$. We then performed a repetition of 500 trials of the same experiment and the number of support constraints was always between 20 and 43. Each time the probabilistic mass was below the value $\epsilon(s^*)$ as expected since the confidence is $1 - 10^{-6}$. On average, the probabilistic mass outside the sphere was in a ratio of 0.44 with $\epsilon(s^*)$. A margin between the real mass and $\epsilon(s^*)$ is required because the mass outside the hyper-sphere is subject to stochastic fluctuation, while the result holds with high confidence.
- (iv) In the first simulation run of the example, we found 28 support constraints. Given a fully supported problem in $d = 28$ dimensions, it is not difficult to augment the optimization domain with 73 dummy variables to make the total number of optimization variables equal to 101, which is the same as the number of variables we have in this example, while the number of support constraints remains 28. For these problems, it is immediate to show that result (3.4) can be applied with $d = 28$, and, moreover, (3.4) holds with an equality. Therefore, any result that is valid distribution-free for all problems with 101 variables cannot possibly return a value for $\epsilon(28)$ that outdoes the ϵ obtained from (3.4) with $d = 28$. Interestingly, setting the right-hand side of (3.4) to 10^{-6} gives $\epsilon = 5.97\%$, a value not too different from $\epsilon(28) = 6.71\%$ obtained from (8.6). This result is interpreted that *a posteriori* observing that there are 28 support constraints leads to certificates on the violation that are not too different from *a priori* knowing that the support constraints are always deterministically equal to 28.

8.5 ■ Expected shortfall

In this section, we consider again uncertain cost functions of the type $\ell(\nu, \delta)$.

Throughout this book, our attention has been focused on minmax solutions of a scenario program built from a finite sample of δ values or on the solution of the same program after some of the scenarios have been removed and max is performed over the remaining scenarios. This corresponds to seeking a minimizer that violates no more than a given proportion of scenarios. In economics, this approach is called value at risk (VaR). At times, one is interested not only in the proportion of scenarios that are violated, but also in the values that the solution achieves corresponding to these scenarios. The larger the values, the higher the jeopardy. This leads to alternative measures of risk. A well-known approach consists in minimizing the average of the violated scenarios, an approach that in economics is called conditional value at risk (CVaR), or expected shortfall (ES), with minor differences between the two that we ignore here. A study of this approach has been performed in [76], where it has been shown that the results valid for the classical scenario approach can be carried over to this setup.

For any ν , denote by $\ell_i(\nu)$, $i = 1, \dots, N$, the values attained by $\ell(\nu, \delta_1), \dots, \ell(\nu, \delta_N)$ taken in decreasing order:

$$\ell_1(\nu) \geq \ell_2(\nu) \geq \dots \geq \ell_N(\nu).$$

The scenario ES approach prescribes that one minimizes

$$\min_{\nu \in \mathbb{R}^{d-1}} \left[\frac{1}{k} \sum_{i=1}^k \ell_i(\nu) \right], \quad (8.11)$$

where the average is taken over the worst k ($k \leq N$) values. ν_{ES}^* is the minimizer of (8.11). Assuming that $N \geq k + d$, one can further define

$$\bar{\ell} = \ell_{k+d}(\nu_{ES}^*),$$

which is the so-called “shortfall threshold.” In typical cases, the interpretation of $\bar{\ell}$ is that it separates the values corresponding to shortfall situations (those achieving the values that concur in forming ES) from functions attaining a lower value at the minimizer; see Figure 8.7 for an example.

Under a nondegeneracy condition, in [76] it is proven that the probability that one next value exceeds $\bar{\ell}$ for $\nu = \nu_{ES}^*$ is still governed by a beta distribution, the same beta distribution that is obtained in the standard scenario approach but in dimension $k - 1 + d$. In particular, this implies that $\mathbb{E}[V(\nu_{ES}^*, \bar{\ell})] = \frac{k-1+d}{N+1}$, which we mention explicitly since in the next section we provide a numerical example where this result is used.

8.5.1 ■ Numerical example: Portfolio optimization

Suppose that n_a assets $A^{[1]}, \dots, A^{[n_a]}$ are available for trading. On period i , the asset $A^{[j]}$ may gain or lose value in the market, and the ratio $\delta_i^{[j]} = (P_i^{[j]} - P_{i-1}^{[j]}) / P_{i-1}^{[j]}$, where $P_i^{[j]}$ is the closing price of asset $A^{[j]}$ on period i , is called the *rate of return* of asset $A^{[j]}$ on period i . To cope with uncertainty, investors diversify among assets. Thus, if an investor has \$1 to invest, s/he will invest fractions $\nu^{[1]}, \dots, \nu^{[n_a]}$ of her/his dollar on $A^{[1]}, \dots, A^{[n_a]}$ (we assume that $\nu^{[j]} \geq 0$ for all j , and $\sum_{j=1}^{n_a} \nu^{[j]} = 1$). The vector

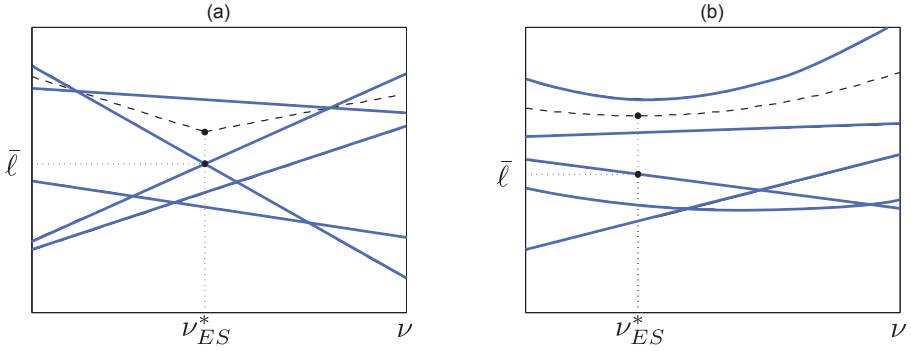


Figure 8.7. The shortfall threshold $\bar{\ell}$. Take $k = 2$. The solid lines represent the functions $\ell(\nu, \delta_i)$, and the dashed line is $\frac{1}{2}(\ell_1(\nu) + \ell_2(\nu))$, from which ν_{ES}^* is determined. (a) $\bar{\ell}$ is at the boundary of the shortfall situations. (b) $\bar{\ell}$ is one step further down than the boundary of the shortfall situations; this is a consequence of the fact that two functions $\ell(\nu, \delta_i)$ only determine the minimizer.

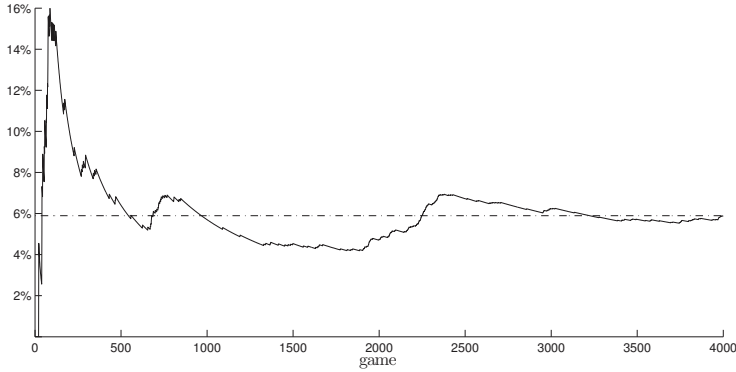


Figure 8.8. Solid line (—) = average number of times for when $\ell_{j+N+1}(\nu_{ES,j}^*) \ge \bar{\ell}_j$; dashed-dotted line (---) = 5.9% obtained from the theory.

$\nu := (\nu^{[1]}, \dots, \nu^{[n_a]})$ is called a “portfolio.” Letting $\delta_i := (\delta_i^{[1]}, \dots, \delta_i^{[n_a]})$ be the vector of the rates of return, the scalar product $\delta_i \cdot \nu = \sum_{j=1}^{n_a} \delta_i^{[j]} \nu^{[j]}$ is the rate of return of the portfolio on period i . If $\delta_i \cdot \nu$ is positive, the investor raises on period i her/his capital of $\delta_i \cdot \nu$ \$ for each dollar invested.

Define

$$\ell(\nu, \delta_i) := -\delta_i \cdot \nu,$$

which quantifies the portfolio loss on period i . Suppose now that the investor has observed a record of N vectors $\delta_1, \dots, \delta_N$ on previous periods. Then, s/he can choose a portfolio ν by minimizing the empirical expected shortfall $\frac{1}{k} \sum_{i=1}^k \ell_i(\nu)$.

We next consider the 5002 closing prices from November 11, 1995, to October 1, 2015, of $n_a = 10$ companies in the S&P 500 index²⁴ and apply the ES approach in a sliding window fashion. Precisely, we solve in succession 4000 optimization problems with $N = 1000$ periods, from j to $j + N - 1$, $j = 1, \dots, 4000$, each. Setting

²⁴These were the 10 companies in the index with the highest market capitalization at the beginning of 2015, namely, AAPL, XOM, MSFT, JNJ, WMT, WFC, GE, PG, JPM, CVX.

$k = 50$, we compute $v_{ES,j}^*$ and $\bar{\ell}_j$ for $j = 1, \dots, 4000$. Figure 8.8 gives the average number of times for when $\ell(v_{ES,j}^*, \delta_{j+N}) \geq \bar{\ell}_j$, compared with the value $\frac{k-1+n_a}{N+1} \simeq 5.9\%$.²⁵

8.6 ■ Nonconvex scenario optimization

Theorem 5.2 states that in a convex scenario program in dimension d there are at most d support constraints. These are constraints that, if one of them is removed, the solution improves (Definition 5.1). The fact that the number of support constraints is bounded by the dimension of the optimization variable played a central role in Chapter 3. Moving on to nonconvex optimization, this result is no longer true. Figure 8.9 displays a nonconvex program in dimension $d = 2$ with three support constraints. This fact poses a major stumbling block in an attempt to extend the theory of Chapter 3 to nonconvex optimization.²⁶

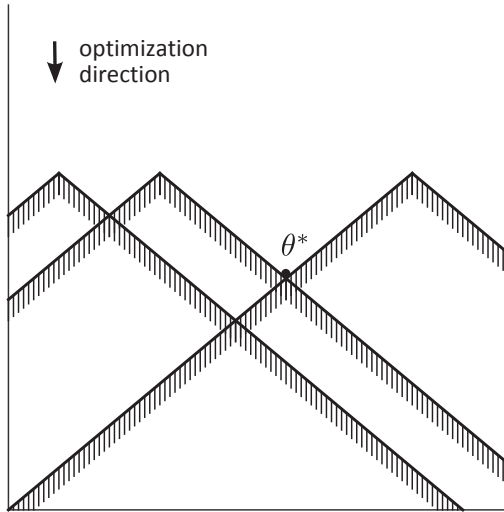


Figure 8.9. A nonconvex scenario program with three constraints. All three of the constraints are of support.

The wait-and-judge theory of Section 8.4, on the other hand, suggests that one can *a posteriori* count the number of support constraints to make judgements about the violation of the solution. While this approach has only been applied to *convex* optimization in Section 8.4, it may still offer a handle to address *nonconvex* problems for which no *a priori* bound on the number of support constraints is available. How far can we go in applying the wait-and-judge approach to a nonconvex setup?

An analysis of the results in [29] shows that the wait-and-judge theory has really nothing to do with convexity, the only standing condition being the nondegeneracy Assumption 8.6. As a consequence, it is shown in [29] that the key ideas of the wait-and-judge theory carry over to optimization problems defined over generic

²⁵In this problem, optimization takes place over the simplex in $\mathbb{R}^{n_a}$ defined as $\{v \in \mathbb{R}^{n_a} : v^{[j]} \geq 0 \text{ for all } j, \sum_{j=1}^{N_a} v^{[j]} = 1\}$, which is a subset of an affine subspace of dimension $n_a - 1$; hence, $d - 1 = n_a - 1$ in this context.

²⁶In specific cases, with suitable problem formulations, it is still possible to develop a theory akin to that presented in Chapter 3; attempts in this direction are [64, 50, 55].

sets where convexity is not required, a fact that we briefly discuss in the following. Before doing so, however, we feel it is advisable to alert the reader to the fact that this does not mean that nonconvex setup has been tamed and it is now completely understood. The reason for this is that the nondegeneracy assumption turns out to be restrictive for nonconvex problems.

Let Θ be a generic set. For example, Θ can be an infinite dimensional vector space or just a set without an algebraic structure. Let $f(\theta)$ be a real-valued function defined over Θ , and let Θ_δ be subsets of Θ . No restrictions apply to f and Θ_δ . For example, $f(\theta)$ is not required to be a convex function, nor are Θ_δ convex sets. In this context, the scenario optimization program is

$$\begin{aligned} \min_{\theta \in \Theta} f(\theta) \\ \text{subject to } \theta \in \bigcap_{i=1, \dots, N} \Theta_{\delta_i}. \end{aligned} \quad (8.12)$$

In (8.12), all we know is that the number of support constraints is *a priori* bounded by N , the total number of constraints. No other limitations exist. Correspondingly, one considers a function $\epsilon(k)$ that ranges over $k = 0, 1, \dots, N$. Under the nondegeneracy Assumption 8.6, it is shown in paper [29] that one can extend the result in (8.6) and show that

$$\mathbb{P}^N \{V(\theta^*) > \epsilon(s^*)\} \leq \gamma^*, \quad (8.13)$$

where γ^* is this time obtained from the following variational problem:

$$\begin{aligned} \gamma^* = \inf_{\xi(\cdot) \in \mathcal{P}_N} \xi(1) \\ \text{subject to } \frac{1}{k!} \frac{d^k}{dt^k} \xi(t) \geq \binom{N}{k} t^{N-k} \cdot \mathbf{1}_{\{t \in [0, 1-\epsilon(k)]\}}, \quad t \in [0, 1], \\ k = 0, 1, \dots, N, \end{aligned}$$

where $\mathcal{P}_N$ is the class of polynomials of degree N .

An example illustrates this result.

Example 8.7 (convex hull in $\mathbb{R}^2$). Points p_i , $i = 1, \dots, N$, are independently sampled from a probability distribution $\mathbb{P}$ on $\mathbb{R}^2$, and the problem of constructing the smallest convex set that contains all the points is considered:

$$\begin{aligned} \min_{C \in \mathcal{C}} \mu(C) \\ \text{subject to } p_i \in C, \quad i = 1, \dots, N, \end{aligned} \quad (8.14)$$

where μ is the Lebesgue measure on $\mathbb{R}^2$ and $\mathcal{C}$ is the collection of all convex sets of $\mathbb{R}^2$. Program (8.14) is a scenario program with $\Theta = \mathcal{C}$, $f(\theta) = \mu(C)$, $\delta_i = p_i$, and $\Theta_i = \{C \in \mathcal{C} : p_i \in C\}$. Its solution C^* is the convex hull of points p_i , $i = 1, \dots, N$, and the problem is nondegenerate if and only if $\mathbb{P}$ has no concentrated mass on isolated points. As a matter of fact, when the points p_i are all distinct, the support constraints are those obtained in correspondence of the p_i 's at the vertices of the convex hull, and the convex hull of the vertex points coincides with the convex hull of all points.

We want to evaluate the probability mass that is left outside the convex hull. This is the same as assessing the violation $V(C^*)$.

We consider two probability distributions $\mathbb{P}$. Suppose first that $\mathbb{P}$ is the uniform distribution on the boundary of a circle. In this case, the convex hull is a polygon inscribed in the circle with vertices coincident with the points p_i ; see Figure 8.10(a) for an instance with $N = 7$. Hence, the number of support constraints is N , i.e., $s_N^* = N$, with probability 1, and an application of result (8.13) gives that $\epsilon(s^*) = \epsilon(N) = 1$; see [29] for a detailed derivation. This is the correct evaluation of the violation since every polygon inscribed in the circle leaves outside a probability mass equal to 1.

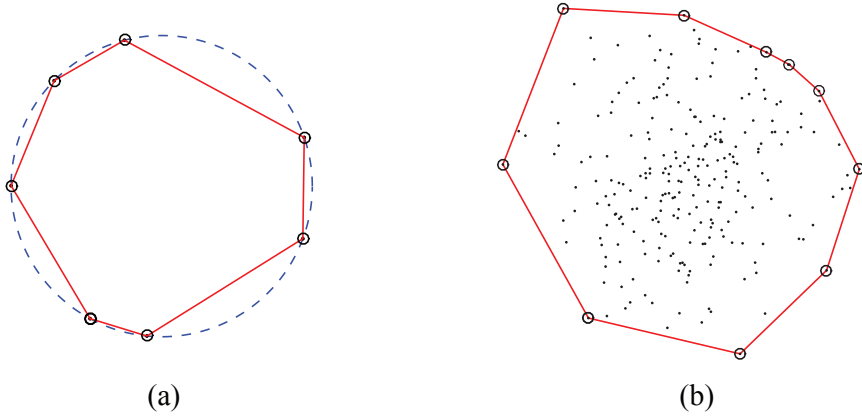


Figure 8.10. Two convex hulls of N points. (a) Points are sampled from the boundary of a circle. (b) Points are sampled from a Gaussian distribution.

Suppose instead that the points are sampled from a Gaussian distribution with zero mean and identity covariance matrix. See Figure 8.10(b) for an instance with $N = 250$, where the number of support constraints is 10. Setting $\gamma^* = 10^{-6}$, result (8.13) gives in this case that $\epsilon(10) = 0.147$, that is, the probabilistic mass outside the obtained convex hull is no more than 14.7% (see again [29] for computational details). ■

Let us now go back to inspect more closely the nondegeneracy Assumption 8.6. In convex problems, for a constraint to be of support, it needs to be active. Hence, degeneracy is an anomalous condition requiring that more than the constraints needed to determine the solution meet at the solution point. In nonconvex problems, instead, this is not the case (Figure 8.11 shows an example), so degeneracy cannot be seen as a pathological situation. In actual effects, constructing nonconvex problems in $\mathbb{R}^d$ that are not degenerate and, unlike convex problems, exhibit more than d support constraints with nonzero probability is not an easy task, and only contrived examples are presently available.

A conjecture that is under consideration at the time this book is being written is that degeneracy can possibly only help violation. This means that an upper bound on $\mathbb{P}^N\{V(\theta^*) > \epsilon(s^*)\}$ for nondegenerate problems is also an upper bound on the same quantity for all problems, including those that are degenerate. If so, result (8.13) would apply to all nonconvex problems, nondegenerate and degenerate. At the present state of knowledge, however, this fact remains a conjecture, and indeed proving or disproving it would set an important advance in the study of nonconvex scenario problems.

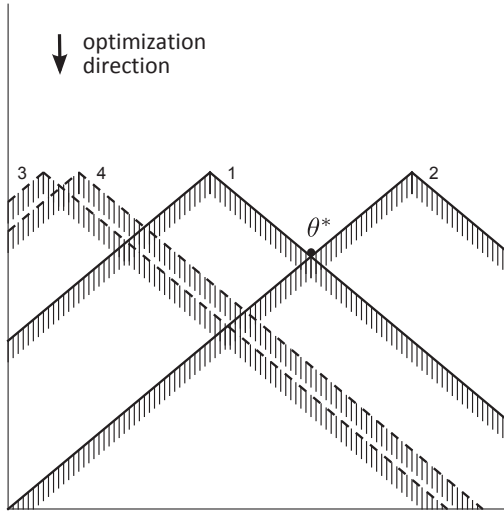


Figure 8.11. A nonconvex scenario program with four constraints. Constraints 3 and 4 are not of support since removing 3 while maintaining 1, 2, and 4 or removing 4 while maintaining 1, 2, and 3 does not change the solution. However, if only the support constraints 1 and 2 are maintained, then the solution moves to a lower value on the extreme left of the optimization domain.

On a different count, an alternative approach introduced in [31] exists to attack nonconvex problems. This approach is not as beautiful as the wait-and-judge theory, and it does not offer results as tight as those of the wait-and-judge approach, but it has the great advantage of being generally applicable. We sketch this approach in the following.

We start with the definition of support set.

Definition 8.8. A support set for program (8.12) is a subset $\{\Theta_{i_1}, \dots, \Theta_{i_k}\}$ of constraints such that program (8.12) with only these constraints in place has the same solution as the program with all constraints.

A support set is said to be *irreducible* if no element of it can be removed without changing the solution. In general, multiple irreducible support sets can be found for the same program (for instance, in Figure 8.11 both constraints 1, 2, and 3 and constraints 1, 2, and 4 are irreducible support sets). A support set whose cardinality is no more than the cardinality of any other support set is called *minimal*. Finding a minimal support set is in general difficult. However, in paper [31] it is shown that support sets that are not necessarily minimal, but still have relatively small cardinality, can be constructed at low computational cost.

Denote by σ^* the cardinality of a support set obtained by means of any algorithm (so, the support set need not be minimal, nor irreducible). The following result is proven in [31]: Let $\epsilon(k)$ be any given function from $\{0, 1, \dots, N\}$ to $[0, 1]$ such that $\epsilon(N) = 1$; then, it holds that

$$\mathbb{P}^N \{V(\theta^*) > \epsilon(\sigma^*)\} \leq \sum_{k=0}^{N-1} \binom{N}{k} (1 - \epsilon(k))^{N-k}. \quad (8.15)$$

Based on this result, it is easy to show that the function

$$\epsilon(k) = \begin{cases} 1 & \text{if } k = N, \\ 1 - \sqrt[N-k]{\frac{\beta}{N^{(N)}}} & \text{otherwise} \end{cases} \quad (8.16)$$

gives that $\mathbb{P}^N \{V(\theta^*) > \epsilon(\sigma^*)\} \leq \beta$, for any given $\beta \in [0, 1]$. Figure 8.12 profiles a plot of this $\epsilon(k)$ for various values of N and $\beta = 10^{-6}$.

Results (8.15) and (8.16) hold without any nondegeneracy assumption. On the other hand, they do not exploit the intimate structure of optimization programs, so this theory is not as tight as the wait-and-judge approach. In particular, result (3.4) cannot be recovered from (8.15) as it instead can be done using the wait-and-judge approach.

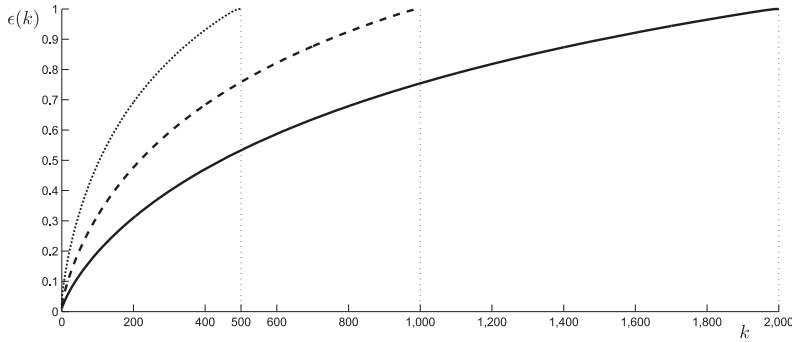


Figure 8.12. Plot of $\epsilon(k)$ in (8.16) for $N = 500$ (dotted line), $N = 1000$ (dashed line), and $N = 2000$ (continuous line) ($\beta = 10^{-6}$).

8.6.1 ■ An example: Control with quantized input

Consider the discrete-time uncertain linear system

$$x_{t+1} = Ax_t + Bu_t, \quad (8.17)$$

where $x_t \in \mathbb{R}^2$ is the state variable, $u_t \in \mathbb{R}$ is the control input, $B = [0 \ 0.5]^T$, and $A \in \mathbb{R}^{2 \times 2}$ is uncertain with independent Gaussian entries with mean

$$\bar{A} = \begin{bmatrix} 0.8 & -1 \\ 0 & -0.9 \end{bmatrix}$$

and standard deviation 0.02 each. Here, we identify a matrix A with a δ . The system is initialized in $x_0 = [1 \ 1]^T$. Moreover, due to actuation limitations, the input is chosen from a discrete and bounded set: $u_t \in U = \{-5, \dots, -1, 0, 1, \dots, 5\}$.

The control objective is that of driving the system state close to the origin in $T = 8$ instants by choosing a suitable input sequence $u_0, \dots, u_{T-1}$. Since $x_T = A^T x_0 + \sum_{t=0}^{T-1} A^{T-1-t} B u_t$, if we let

$$R = [B \ AB \ \dots \ A^{T-1}B]$$

and

$$\mathbf{u} = [u_{T-1} \ u_{T-2} \ \dots \ u_0]^T,$$

the problem can be formulated as that of deciding $\mathbf{u}$ so as to make $\|A^T x_0 + R\mathbf{u}\|_\infty = \|x_t\|_\infty$ as small as possible, where $\|\cdot\|_\infty$ stands for the ∞ -norm. Finite-horizon, open-loop problems like this one are common as single steps of more complex receding-horizon model predictive control (MPC) schemes; other times, they arise as stand-alone problems in sensorless environments in which no feedback is possible (e.g., positioning of an end-effector when no exteroceptive sensors are available). The example here is a toy version of these problems.

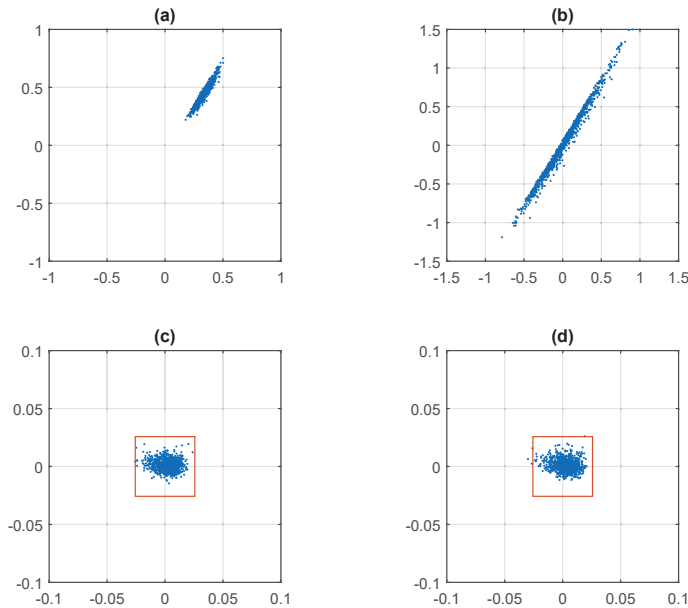


Figure 8.13. Final state for 1000 systems.

Figure 8.13(a) shows the final states x_8 for $N = 1000$ extractions A_i when no control action is applied ($u_t = 0$ for $t = 0, \dots, 7$). One can note that relying on the state contraction property due to the system stability is not enough to reach the origin in eight time instants. We next computed the optimal control sequence $\hat{\mathbf{u}}$ for the nominal system $(\bar{A}, B)$ and found $\hat{\mathbf{u}} = [-2 \ 3 \ -2 \ 4 \ 3 \ -5 \ 2 \ -5]^T$. When this $\hat{\mathbf{u}}$ is applied to the same A_i 's as before, the final states x_8 in Figure 8.13(b) are obtained. The nominal control action $\hat{\mathbf{u}}$ produces no benefit as it moves the state faster towards the origin, but it also produces a large dispersion around it owing to uncertainty.

Consider instead the scenario approach. Precisely, the $N = 1000$ scenarios A_i were used to construct the scenario program

$$\begin{aligned} \min_{h \in \mathbb{R}, \mathbf{u} \in U^8} \quad & h \\ \text{subject to} \quad & \|(A_i)^T x_0 + R_i \mathbf{u}\|_\infty \leq h, \quad i = 1, \dots, N, \end{aligned} \quad (8.18)$$

which is a mixed-integer program aiming at a control sequence $\mathbf{u}$ that minimizes the largest deviation (for the various A_i) of x_8 from the origin. The optimal solution we obtained was $(h^*, \mathbf{u}^*)$ with $h^* = 0.0257$ and $\mathbf{u}^* = [1 \ -1 \ -4 \ 3 \ 5 \ -4 \ -2 \ 4]^T$. Figure 8.13(c) displays the final states x_8 for the 1000 A_i 's used in (8.18) (note the different scale on the axes of this figure as compared to 8.13(a) and 8.13(b)). The same figure also represents the ∞ -box of size $h^* = 0.0257$.

The final states plotted in Figure 8.13(c) refer to situations that have been explicitly used in (8.18) to determine $(h^*, \mathbf{u}^*)$. A natural question to ask is how well $\mathbf{u}^*$ performs when it is applied to a new matrix A . For this program we found a support set with cardinality $\sigma^* = 3$ (how it was found is explained in detail in paper [31]). Choosing $\beta = 10^{-6}$ and using the function $\epsilon(k)$ in (8.16), we find $\epsilon(\sigma^*) = \epsilon(3) = 0.0323$. According to the result (8.15), with confidence $1 - 10^{-6}$, the optimal solution $(h^*, \mathbf{u}^*)$ is $\epsilon(\sigma^*)$ -feasible, which in the present context means that

$$\|x_T\|_\infty = \|A^T x_0 + R\mathbf{u}^*\|_\infty > h^*$$

happens with probability no more than $\epsilon(\sigma^*)$. For our situation this becomes

$$\mathbb{P}\{\|x_8\|_\infty > 0.0257\} \leq 3.23\%,$$

and x_8 is in the box in Figure 8.13(c) with probability at least 96.77%. Figure 8.13(d) shows the final state reached by 1000 new simulations, each with a newly extracted A .

Bibliography

- [1] T. Alamo, R. Tempo, and E.F. Camacho. Randomized strategies for probabilistic solutions of uncertain feasibility and optimization problems. *IEEE Transactions on Automatic Control*, 54:2545–2559, 2009. (Cited on p. 4)
- [2] T. Alamo, R. Tempo, A. Luque, and D.R. Ramirez. Randomized methods for design of uncertain systems: Sample complexity and sequential algorithms. *Automatica*, 51:160–172, 2015. (Cited on p. 41)
- [3] B.D.O. Anderson and J.B. Moore. *Optimal Control. Linear Quadratic Methods*. Prentice-Hall, Englewood Cliffs, NJ, 1989. (Cited on p. 50)
- [4] P. Apkarian and H.D. Tuan. Parameterized LMIs in control theory. *SIAM Journal on Control and Optimization*, 38:1241–1264, 2000. (Cited on pp. 22, 26)
- [5] P. Apkarian, H.D. Tuan, and J. Bernussou. Continuous-time analysis, eigenstructure assignment, and H_2 synthesis with enhanced linear matrix inequalities (LMI) characterizations. *IEEE Transactions on Automatic Control*, 46:1941–1946, 2001. (Cited on p. 26)
- [6] K.J. Åström. *Introduction to Stochastic Control Theory*. Academic Press, London, UK, 1970. (Cited on p. 2)
- [7] B.R. Barmish. *New Tools for Robustness of Linear Systems*. Macmillan, New York, 1994. (Cited on p. 51)
- [8] B.R. Barmish and C.M. Lagoa. The uniform distribution: A rigorous justification for its use in robustness analysis. *Mathematics of Control, Signals, and Systems*, 10:203–222, 1997. (Cited on p. 23)
- [9] G. Becker and A. Packard. Robust performance of linear parametrically varying systems using parametrically-dependent linear feedback. *Systems and Control Letters*, 23:205–215, 1994. (Cited on p. 26)
- [10] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski. *Robust Optimization*. Princeton University Press, Princeton, NJ, 2009. (Cited on p. 2)
- [11] A. Ben-Tal and A. Nemirovski. Robust convex optimization. *Mathematics of Operations Research*, 23:769–805, 1998. (Cited on p. 2)
- [12] A. Ben-Tal and A. Nemirovski. Robust optimization - methodology and applications. *Mathematical Programming*, 92:453–480, 2002. (Cited on p. 2)
- [13] P. Beraldi and A. Ruszczyński. A branch and bound method for stochastic integer problems under probabilistic constraints. *Optimization Methods and Software*, 17:359–382, 2002. (Cited on p. 44)
- [14] D. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, Belmont, MA, 2005. (Cited on p. 2)

- [15] D. Bertsimas, D.B. Brown, and C. Caramanis. Theory and applications of robust optimization. *SIAM Review*, 53:464–501, 2011. (Cited on p. 2)
- [16] L. Blackmore, M. Ono, A. Bektassov, and B.C. Williams. A probabilistic particle-control approximation of chance constrained stochastic predictive control. *IEEE Transactions on Robotics*, 26:502–517, 2010. (Cited on p. 4)
- [17] S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, Philadelphia, 1994. (Cited on pp. 22, 51)
- [18] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004. (Cited on p. 22)
- [19] S. Boyd and Q. Yang. Structured and simultaneous Lyapunov functions for system stability problems. *International Journal of Control*, 49:2215–2240, 1989. (Cited on p. 22)
- [20] G. Calafiore and M.C. Campi. The scenario approach to robust control design. *IEEE Transactions on Automatic Control*, 51:742–753, 2006. (Cited on pp. 4, 26)
- [21] G. Calafiore and F. Dabbene. A probabilistic framework for problems with real structured uncertainty in systems and control. *Automatica*, 38:1265–1276, 2002. (Cited on p. 8)
- [22] G. Calafiore, F. Dabbene, and R. Tempo. Randomized algorithms for probabilistic robustness with real and complex structured uncertainty. *IEEE Transactions on Automatic Control*, 45:2218–2235, 2000. (Cited on p. 8)
- [23] G.C. Calafiore and L. Fagiano. Robust model predictive control via scenario optimization. *IEEE Transactions on Automatic Control*, 58:219–224, 2013. (Cited on p. 26)
- [24] M.C. Campi. Classification with guaranteed probability of error. *Machine Learning*, 80:63–84, 2010. (Cited on pp. 82, 84, 85)
- [25] M.C. Campi, G. Calafiore, and S. Garatti. Interval predictor models: Identification and reliability. *Automatica*, 45:382–392, 2009. (Cited on pp. 67, 76, 78)
- [26] M.C. Campi and A. Carè. Random convex programs with L_1 -regularization: Sparsity and generalization. *SIAM Journal on Control and Optimization*, 51:3532–3557, 2013. (Cited on pp. 92, 93)
- [27] M.C. Campi and S. Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19:1211–1230, 2008. (Cited on pp. 38, 39, 62)
- [28] M.C. Campi and S. Garatti. A sampling-and-discarding approach to chance-constrained optimization: Feasibility and optimality. *Journal of Optimization Theory and Applications*, 148:257–280, 2011. (Cited on p. 45)
- [29] M.C. Campi and S. Garatti. Wait-and-judge scenario optimization. *Mathematical Programming*, 167:155–189, 2018. (Cited on pp. 96, 97, 101, 102, 103)
- [30] M.C. Campi, S. Garatti, and M. Prandini. The scenario approach for systems and control design. *Annual Reviews in Control*, 33:149–157, 2009. (Cited on p. 27)
- [31] M.C. Campi, S. Garatti, and F.A. Ramponi. A general scenario theory for non-convex optimization and decision making. *IEEE Transactions on Automatic Control*, published online, DOI:10.1109/TAC.2018.2808446. (Cited on pp. 104, 107)
- [32] E.J. Candes and M.B. Wakin. An introduction to compressive sampling. *IEEE Signal Processing Magazine*, 21:21–30, 2008. (Cited on p. 91)

- [33] A. Carè, S. Garatti, and M.C. Campi. FAST—Fast algorithm for the scenario technique. *Operations Research*, 62:662–671, 2014. (Cited on p. 94)
- [34] A. Carè, S. Garatti, and M.C. Campi. Scenario min-max optimization and the risk of empirical costs. *SIAM Journal on Optimization*, 25:2061–2080, 2015. (Cited on p. 90)
- [35] A. Charnes and W.W. Cooper. Chance constrained programming. *Management Science*, 6:73–79, 1959. (Cited on p. 4)
- [36] E. Cinquemani, M. Agarwal, D. Chatterjee, and J. Lygeros. Convexity and convex approximations of discrete-time stochastic control problems with constraints. *Automatica*, 47:2082–2087, 2011. (Cited on p. 4)
- [37] P. Colaneri, J.C. Geromel, and A. Locatelli. *Control Theory and Design: An RH_2 and RH_∞ viewpoint*. Academic Press, San Diego, CA, 1997. (Cited on pp. 2, 22)
- [38] D.A. Cox, J.B. Little, and D. O’Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, New York, 2007. (Cited on p. 26)
- [39] N. Cristianini and J. Shawe-Taylor. *An introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, Cambridge, UK, 2000. (Cited on p. 80)
- [40] M.C. de Oliveira, J. Bernussou, and J.C. Geromel. A new discrete-time robust stability condition. *Systems and Control Letters*, 37:261–265, 1999. (Cited on p. 26)
- [41] M.C. de Oliveira, J.C. Geromel, and J. Bernussou. Extended H_2 and H_∞ norm characterization and controller parameterization for discrete-time systems. *International Journal of Control*, 75:666–679, 2002. (Cited on p. 26)
- [42] D. Dentcheva. Optimization models with probabilistic constraints. In G. Calafiore and F. Dabbene, editors, *Probabilistic and Randomized Methods for Design under Uncertainty*, pages 47–96, Springer, London, 2006. (Cited on p. 4)
- [43] D. Dentcheva, A. Prékopa, and A. Ruszczyński. Concavity and efficient points of discrete distributions in probabilistic programming. *Mathematical Programming*, 89:55–77, 2000. (Cited on p. 44)
- [44] D. Dentcheva, A. Prékopa, and A. Ruszczyński. On convex probabilistic programming with discrete distribution. *Nonlinear Analysis*, 47:1997–2009, 2001. (Cited on p. 44)
- [45] L. Devroye, L. Györfi, and G. Lugosi. *A Probabilistic Theory of Pattern Recognition*. Springer-Verlag, New York, 1996. (Cited on p. 80)
- [46] D. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52:1289–1306, 2006. (Cited on p. 91)
- [47] D. Donoho. For most large underdetermined systems of equations, the minimal ℓ^1 -norm is the sparsest solution. *Communications on Pure and Applied Mathematics*, 59:797–829, 2006. (Cited on p. 91)
- [48] J.C. Doyle. Guaranteed margins for LQG regulators. *IEEE Transaction on Automatic Control*, 23:756–757, 1978. (Cited on p. 2)
- [49] D. Dua and E. Karra Taniskidou. *UCI Machine Learning Repository*, <http://archive.ics.uci.edu/ml>, University of California, Irvine, School of Information and Computer Sciences, 2017. (Cited on p. 86)

- [50] P.M. Esfahani, T. Sutter, and J. Lygeros. Performance bounds for the scenario approach and an extension to a class of non-convex programs. *IEEE Transactions on Automatic Control*, 60:46–58, 2015. (Cited on p. 101)
- [51] W.J. Frable. *Thin Needle Aspiration Biopsy*, volume 14 of *Major Problems in Pathology*. W.B. Saunders, Philadelphia, 1983. (Cited on p. 86)
- [52] P. Gahinet, P. Apkarian, and M. Chilali. Affine parameter-dependent Lyapunov functions and real parametric uncertainty. *IEEE Transactions on Automatic Control*, 41:436–442, 1996. (Cited on p. 26)
- [53] S. Garatti and M.C. Campi. L_∞ layers and the probability of false prediction. In *Proceedings of the 15th IFAC Symposium on System Identification (SYSID)*, Saint-Malo, France, 2009, pp. 1187–1192. (Cited on p. 73)
- [54] R.W.M. Giard and J. Hermans. The value of aspiration cytologic examination of the breast. *Cancer*, 69:2104–2110, 1992. (Cited on p. 86)
- [55] S. Grammatico, X. Zhang, K. Margellos, P.J. Goulart, and J. Lygeros. A scenario approach for non-convex control design. *IEEE Transactions on Automatic Control*, 61:334–345, 2016. (Cited on p. 101)
- [56] M. Grant and S. Boyd. Graph implementations for nonsmooth convex programs. In V. Blondel, S. Boyd, and H. Kimura, editors, *Recent Advances in Learning and Control*, volume 371 of *Lecture Notes in Control and Information Sciences*, pages 95–110, Springer-Verlag, London, 2008. (Cited on pp. 7, 22, 51)
- [57] M. Grant and S. Boyd. *CVX: Matlab Software for Disciplined Convex Programming, Version 2.1*. <http://cvxr.com/cvx>, 2017. (Cited on pp. 7, 22, 51)
- [58] M. Green and D.J.N. Limebeer. *Linear Robust Control*. Prentice-Hall, Upper Saddle River, NJ, 1995. (Cited on p. 2)
- [59] O. Güler. *Foundations of Optimization*. Springer, New York, 2010. (Cited on p. 57)
- [60] P.R. Kumar and P. Varaiya. *Stochastic Systems: Estimation, Identification and Adaptive Control*. Prentice-Hall, Upper Saddle River, NJ, 1986. (Cited on p. 2)
- [61] C.M. Lagoa and B.R. Barmish. Distributionally robust Monte Carlo simulation: A tutorial survey. In *Proceedings of the 15th IFAC World Congress*, pages 151–162, Barcelona, Spain, 2002. (Cited on p. 52)
- [62] P. Li, M. Wendt, and G. Wozny. A probabilistically constrained model predictive controller. *Automatica*, 38:1171–1176, 2002. (Cited on p. 4)
- [63] J. Löfberg. YALMIP: A toolbox for modeling and optimization in MATLAB. In *Proceedings of the CACSD Conference*, pages 284–289, Taipei, Taiwan, 2004. (Cited on pp. 7, 22)
- [64] J. Luedtke and S. Ahmed. A sample approximation approach for optimization with probabilistic constraints. *SIAM Journal on Optimization*, 19:674–699, 2008. (Cited on p. 101)
- [65] J. Luedtke, S. Ahmed, and G.L. Nemhauser. An integer programming approach for linear programs with probabilistic constraints. *Mathematical Programming*, 122:247–272, 2010. (Cited on p. 44)
- [66] J. Mohammadpour and C.W. Scherer, editors. *Control of Linear Parameter Varying Systems with Applications*. Springer, New York, 2012. (Cited on p. 22)

- [67] M. Morari and E. Zafriou. *Robust Process Control*. Prentice Hall, Englewood Cliffs, NJ, 1989. (Cited on p. 27)
- [68] H. Ohlsson, L. Ljung, and S. Boyd. Segmentation of ARX-models using sum-of-norms regularization. *Automatica*, 46:1107–1111, 2010. (Cited on p. 92)
- [69] N. Ozay, M. Sznaier, C. Lagoa, and O. Camps. A sparsification approach to set membership identification of a class of affine hybrid systems. In *Proceedings of the IEEE Conference on Decision and Control*, pages 123–130, Cancun, Mexico, 2008. (Cited on p. 92)
- [70] I.R. Petersen and D.C. McFarlane. Optimal guaranteed cost control and filtering for uncertain linear systems. *IEEE Transaction on Automatic Control*, 39:1071–1077, 1994. (Cited on p. 51)
- [71] B.T. Polyak and R. Tempo. Probabilistic robust design with linear quadratic regulators. *Systems & Control Letters*, 43:343–353, 2001. (Cited on p. 50)
- [72] M. Prandini, S. Garatti, and J. Lygeros. A randomized approach to stochastic model predictive control. In *Proceedings of the 51st IEEE Conference on Decision and Control*, Maui, HI, 2012, pp. 7315–7320. (Cited on p. 26)
- [73] A. Prékopa. *Stochastic Programming*. Kluwer, Boston, 1995. (Cited on p. 4)
- [74] A. Prékopa. The use of discrete moment bounds in probabilistic constrained stochastic programming models. *Annals of Operations Research*, 85:21–38, 1999. (Cited on p. 4)
- [75] A. Prékopa. Probabilistic programming. In A. Ruszczyński and A. Shapiro, editors, *Stochastic Programming*, volume 10 of *Handbooks in Operations Research and Management Science*, pages 267–352, Elsevier, London, 2003. (Cited on p. 4)
- [76] F.A. Ramponi and M.C. Campi. Expected shortfall: Heuristic and certificates. *European Journal of Operational Research*, 267:1003–1013, 2018. (Cited on p. 99)
- [77] A. Ruszczyński. Probabilistic programming with discrete distributions and precedence constrained knapsack polyhedra. *Mathematical Programming*, 93:195–215, 2002. (Cited on p. 44)
- [78] A. Ruszczyński and A. Shapiro, editors. *Stochastic Programming*, volume 10 of *Handbooks in Operations Research and Management Science*. Elsevier, London, 2003. (Cited on p. 3)
- [79] G. Schildbach, L. Fagiano, C. Frei, and M. Morari. The scenario approach for stochastic model predictive control with bounds on closed-loop constraint violations. *Automatica*, 50:3009–3018, 2014. (Cited on p. 26)
- [80] G. Schildbach, L. Fagiano, and M. Morari. Randomized solutions to convex programs with multiple chance constraints. *SIAM Journal on Optimization*, 23:2479–2501, 2013. (Cited on p. 95)
- [81] B. Schölkopf and A.J. Smola. *Learning with Kernels*. MIT Press, Cambridge, MA, 2002. (Cited on p. 80)
- [82] A. Shapiro, D. Dentcheva, and A. Ruszczyński. *Lectures on Stochastic Programming: Modelling and Theory*. SIAM, Philadelphia, 2009. (Cited on pp. 3, 4)
- [83] A.N. Shiryaev. *Probability*. Springer, New York, 1996. (Cited on pp. 60, 65)
- [84] T. Söderström. *Discrete-Time Stochastic Systems*. Springer, London, 2002. (Cited on p. 8)

- [85] W.N. Street, W.H. Wolberg, and O.L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In *IS&T/SPIE 1993 International Symposium on Electronic Imaging: Science and Technology*, volume 1905, pages 861–870. San Jose, CA, 1993. (Cited on p. 86)
- [86] R. Tempo, G. Calafiore, and F. Dabbene. *Randomized Algorithms for Analysis and Control of Uncertain Systems*. Springer-Verlag, London, 2005. (Cited on pp. 8, 22, 23, 50, 51)
- [87] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B (Statistical Methodology)*, 58:267–288, 1996. (Cited on p. 91)
- [88] K.C. Toh, M.J. Todd, and R.H. Tutuncu. Sdpt3 — a MATLAB software package for semidefinite programming. *Optimization Methods and Software*, 11:545–581, 1999. (Cited on p. 51)
- [89] A. Trofino and C.E. de Souza. Biquadratic stability of uncertain linear systems. *IEEE Transactions on Automatic Control*, 46:1303–1307, 2001. (Cited on p. 26)
- [90] J. Tropp. Greed is good: Algorithmic results for sparse approximation. *IEEE Transactions on Information Theory*, 50:2231–2242, 2004. (Cited on p. 91)
- [91] R.H. Tutuncu, K.C. Toh, and M.J. Todd. Solving semidefinite-quadratic-linear programs using sdpt3. *Mathematical Programming Series B*, 95:189–217, 2003. (Cited on p. 51)
- [92] J.S. Tyler and F.B. Tuteur. The use of a quadratic performance index to design multivariable control systems. *IEEE Transaction on Automatic Control*, 11:84–92, 1966. (Cited on p. 50)
- [93] V.N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. (Cited on pp. 40, 80)
- [94] V.N. Vapnik. *Statistical Learning Theory*. Wiley, New York, 1998. (Cited on pp. 40, 80)
- [95] M. Vidyasagar. *Theory of Learning and Generalization: With Applications to Neural Networks and Control Systems*. Springer-Verlag, London, 1997. (Cited on p. 40)
- [96] H. Xu and S. Mannor. Robustness and generalization. *Machine Learning*, 86:391–423, 2012. (Cited on p. 91)
- [97] G. Zames. Feedback and optimal sensitivity: Model reference transformations, multiplicative seminorms, and approximate inverses. *IEEE Transaction on Automatic Control*, 26:301–320, 1981. (Cited on p. 2)
- [98] X. Zhang, S. Grammatico, G. Schildbach, P.J. Goulart, and J. Lygeros. On the sample size of random convex programs with structured dependence on the uncertainty. *Automatica*, 60:182–188, 2016. (Cited on p. 95)
- [99] K. Zhou, J.C. Doyle, and K. Glover. *Robust and Optimal Control*. Prentice-Hall, Upper Saddle River, NJ, 1996. (Cited on p. 2)

Index

- assumption
 - convexity, 34
 - existence and uniqueness, 38
 - nondegeneracy, 96
- average approach, 2, 4
- ball-solution
 - definition of, 61
- beta distribution, 39
- chance-constrained approach, 3, 4
- chance-constrained optimization, 4, 44
- Chebyshev layer, 70, 73
- classification, 79
- conditional value at risk (CVaR), 99
- confidence parameter, 12, 20, 72
- constraint removal, 44
- control
 - robust, 21
 - with quantized input, 105
- convex function, 6
- convex hull, 102
- convexity assumption, 34
- CVaR (conditional value at risk), 99
- data-driven optimization, 9
- degenerate problem, 62, 103
- deterministic robustness, 49
- Dirichlet distribution, 90
- distribution-free result, 14, 41, 72, 84, 98
- disturbance compensation, 27
- empirical costs
 - definition of, 90
- empirical frequency, 43, 44
- expected shortfall, 99
- empirical, 100
- fully supported problem, 58, 73
 - definition of, 58
- GEM (guaranteed error machine), 81
- guaranteed error machine (GEM), 81
- Helly's lemma, 57
- histogram, 85
- IMC (internal model control), 27
- internal model control (IMC), 27
- interval uncertainty, 51
- linear matrix inequality (LMI), 22
- LMI (linear matrix inequality), 22
- machine learning, 10, 79
- mixed-integer program, 106
- model
 - beyond the use of, 53
 - region estimation, 67
- pole placement, 25
- portfolio optimization, 11, 99
- prediction, 68
- prior information, 14, 70, 72
- prior knowledge, 14
- probability box, 89
- procedure
 - constraint removal, 44
 - greedy removal, 44
 - optimal constraint removal, 44
 - random removal, 44
- quadratic stability, 21
- quadratic stabilization, 24
- region estimation model (REM), 67
 - identification, 70
 - reliability, 68
 - size, 68
- regularization, 91
- REM (region estimation model), 67, 68, 70
- risk, 3, 16, 90
 - empirical, 18
- risk parameter, 12, 72
- risk-performance trade-off, 16
- sample complexity, 13, 14, 95
- scenario optimization
 - definition of, 6
 - fast algorithm, 93
 - generalization theorem for, 38
 - generalization theory of, 12
 - nonconvex, 101
 - wait-and-judge, 95, 101
 - with constraint removal, 44, 52
- scenario program, 7, 20, 34
- scenarios, 7
- statistical learning, 79
- support constraint, 57
 - definition of, 57
- support set
 - definition of, 104
- value at risk (VaR), 11, 99
- Vapnik–Chervonenkis dimension, 40

VaR (value at risk), 11, 99	probability, 33	violation set
violation, 33	set, 33	definition of, 33
cumulative distribution of,	violation parameter, 20	
39	violation probability	
expected value of, 42	definition of, 33	worst-case approach, 2, 4

This book is about making decisions driven by experience. In this context, a scenario is an observation that comes from the environment, and scenario optimization refers to optimizing decisions over a set of available scenarios. Scenario optimization can be applied across a variety of fields, including machine learning, quantitative finance, control, and identification.

The scenario approach has been given a solid mathematical foundation in recent years, addressing fundamental questions such as: How should experience be incorporated in the decision process to optimize the result? How well will the decision perform in a new case that has not been seen before in the scenario sample? And how robust will results be when using this approach?

This concise, practical book provides readers with

- an easy access point to make the approach understandable to nonexperts;
- numerous examples and diverse applications from a broad range of domains, including systems theory, control, biomedical engineering, economics, and finance; and
- an overview of various decision frameworks in which the method can be used.

Practitioners can find “easy-to-use recipes,” while theoreticians will benefit from a rigorous treatment of the theoretical foundations of the method, making it an excellent starting point for scientists interested in doing research in this field.

Accessible to experts and nonexperts alike, *Introduction to the Scenario Approach* will appeal to scientists working in optimization, practitioners working in myriad fields involving decision-making, and anyone interested in data-driven decision-making.

Marco C. Campi is a professor at the University of Brescia, Italy, where he has taught topics related to data-driven and inductive methods for many years. He is a distinguished lecturer of the Control Systems Society and chair of the Technical Committee IFAC on Modeling, Identification, and Signal Processing. Professor Campi has held visiting and teaching appointments at several institutions and has served in various capacities on the editorial boards of *Automatica*, *Systems and Control Letters*, and the *European Journal of Control*. Professor Campi is a Fellow of IEEE, a member of IFAC, and a recipient of the Giorgio Quazza prize and the IEEE CSS George S. Axelby outstanding paper award. He has delivered plenary addresses at major conferences, including Optimization, CDC, MTNS, and SYSID.



Simone Garatti is an associate professor in automatic control at the Polytechnic University of Milan, Italy, where he received his Ph.D. in information technology engineering. Professor Garatti has been a visiting scholar and invited lecturer at various prestigious universities, where he has shared his research in optimization and scenario theory. In 2006, he won a fellowship for the short-term mobility of researchers provided by the National Research Council of Italy (CNR). His scientific interests include data-driven optimization, data-driven control systems design, system identification, and randomized algorithms for problems in systems and control. He is the author of more than 70 contributions in international journals, books, and proceedings.



For more information about MOS and SIAM books, journals, conferences, memberships, or activities, contact:



Society for Industrial
and Applied Mathematics
3600 Market Street, 6th Floor
Philadelphia, PA 19104-2688 USA
+1-215-382-9800 • Fax +1-215-386-7999
siam@siam.org • www.siam.org



Mathematical
Optimization Society

Mathematical Optimization Society
3600 Market Street, 6th Floor
Philadelphia, PA 19104-2688 USA
+1-215-382-9800 x319
Fax +1-215-386-7999
service@mathopt.org • www.mathopt.org

MO26

ISBN 978-1-611975-43-7



9781611975437